

Contents

Why the Computer Seems Unfair to Arabic — and How We Fixed It Without

Retraining the AI	4
The big, patient, plain-language version	4
Part 0 — The whole story in about one minute	5
Part 1 — The problem, told slowly, assuming nothing	7
1.1 What is a “digital library that answers questions”?	7
1.2 Why “offline” and “on a laptop” matter so much	8
1.3 Two languages, one computer, and the seed of the problem	9
1.4 The very first disadvantage: Arabic gets chopped into more pieces .	9
1.5 What could “the computer is unfair to Arabic” even mean?	10
1.6 The mystery, set up — and a confession	11

Part 2 — Every Word You Need, Explained Simply **12**

What a computer “search” is, in the first place	12
A “score” — the number the computer gives to a guess	13
The plain meaning	13
The everyday picture	13
One honest complication you should know now	13
The most important warning about scores	14
“Tokens” and why Arabic breaks into more pieces	14
The plain meaning	14
The everyday picture	14
Why Arabic gets the worse end of this	15
The one number to remember here: 1.27 times	15
Why this matters down the line	15
A “relevance cutoff” or “threshold” — a doorway height	16
The plain meaning	16
The everyday picture	16
Why the height of the doorway is the whole problem	16
“Score compression” — a ruler with too few marks	17
First, the idea everybody believes — and why it is wrong	17
The plain meaning of compression	17
The everyday picture: a ruler with too few marks	18
Now translate the actual numbers, slowly	18
Why a narrow band is a problem even when the marks are higher . . .	19
“Separation” and “discriminability” — the gap between good and bad . . .	20
The plain meaning	20
The everyday picture	20
Tying it back	20
“Tenant” — each user’s private shelf	20
The plain meaning	20
The everyday picture	21
Why tenants matter for this book	21
“Frozen model” — we do not retrain the AI	21
The plain meaning	21
The everyday picture	22

Why “frozen” is the backbone of this whole book	22
“Cross-script” — Arabic letters and Latin letters never match	22
The plain meaning	22
The everyday picture	23
Why this is a different, separate problem	23
The honest part you must hear now	23
“Tenant starvation” — a big neighbour hogging the shelf	24
The plain meaning	24
The everyday picture	24
Why it lands on Arabic — and the honest caveats	24
A quick recap, in one breath each	25
Part 4 — The four places where Arabic gets hurt	25
A quick reminder of the machine	26
Spot 1 — The shredder cuts Arabic into more pieces (tokenization fragmentation)	26
Spot 2 — The judge squeezes all Arabic scores into a narrow band (score compression)	28
Spot 3 — The single yes/no cutoff sits in the wrong place for Arabic (the mis-placed global threshold)	29
Spot 4 — A word-search across two alphabets always scores zero (the cross- script lexical zero)	30
The fifth spot — a small library buried under a giant one (minority-library starvation)	31
The four-spot picture, in one breath	32
Part 5 — The big surprise, explained very carefully	33
First, what a “score” even is, and why one number per pair	33
The story of two graders	33
Mapping the story onto Arabic and English	34
Why looking at the average fools you	35
Why the spread is what matters — what a “spread” buys you	35
Why a cutoff set on the wide grader is wrong for the narrow grader	36
Now the real numbers, translated into plain meaning	37
One honest wrinkle, stated plainly	38
Why this whole reframing matters — what it changes about the fix	38
Part 7 — Fix #2: The Cross-Script Glossary (How an Arabic Question Finds an English Book)	39
7.1 A second, completely different problem	39
7.2 Two ways the computer searches at the same time	40
7.3 Why an Arabic question can NEVER word-match an English page	41
7.4 The surprising asymmetry: English questions already work	42
7.5 The fix: a small living dictionary that grows itself	42
7.6 The result: from impossible to perfect, in 48 millionths of a second	43
7.7 The “living” part: it teaches itself new words, once, forever	44
7.8 The honest caveat: the meaning-matcher already does this	45
Part 8 — Fix #3: The Per-Library Shelf (How a Small Arabic Library Avoids Being Buried)	47
8.1 A third problem, and the most subtle one	47

8.2 What a “tenant” is, and how the shelves are shared	47
8.3 The hidden trap: how the word-matcher decides what is “important”	47
8.4 How the giant library buries the small one	48
8.5 Measuring the burial: 0.913 collapses to 0.461	48
8.6 Proving it is vocabulary, not language, and not size alone	49
8.7 The fix: give every library its own little shelf	50
8.8 The honest caveat: the modern search starves WORSE	51
8.9 Why this fix has to come first	52
8.10 The three fixes together	53
Part 9. What the experiments showed, every number in plain words	53
9.1 First result: Arabic words get chopped into more pieces	54
9.2 The headline result: Arabic is NOT scored lower	55
9.3 The real result: the Arabic scores are squeezed into a narrow band	55
9.4 Why the squeeze actually hurts: the irrelevant band creeps up too	56
9.5 The honest exception: the cosine irrelevant band reverses	56
9.6 The five libraries: each one needs its own cutoff	57
9.7 The decisive cross-check: the stricter Arabic bar is from the squeeze, not from lower scores	57
9.8 The gate at work: far fewer wrong answers slip through	58
9.9 The honest exception: the gate does NOT beat the answer-key oracle, and this F1 is not Arabic-specific	59
9.10 The glossary: from impossible to perfect for Arabic-to-English	59
9.11 The honest exception: a dense AI search already gets 1.00 too	60
9.12 Starvation: the small library buried under the giant one	60
9.13 The honest exception: dense AI search starves WORSE, and this is not about Arabic itself	61
Part 10. What we honestly cannot claim	61
10.1 The collection is tiny and overwhelmingly English	62
10.2 My test questions were idealised, not real student questions	62
10.3 The glossary does not beat dense AI search on recall	62
10.4 The cross-script disadvantage is one-directional	63
10.5 The gate’s fairness is partial and depends on which scorer you use	63
10.6 The starvation fix is not an Arabic cure, and dense starves worse	63
10.7 One machine, two specific models	63
10.8 I never tested whether the final answers actually got better	63
10.9 The one thing I do stand behind	64
Part 10.6 — I tested it much harder (more books, more models, real statistics): the honest result	64
Part 11 — Why this matters: fairness as engineering, not retraining	69
The one sentence to take away	69
The usual reflex, and why people reach for it	69
What we did instead: build distribution-aware machinery around the frozen model	70
The brutal honesty, restated, because this is the honest version	72
Why this is the right answer for a place like Karbala	72
How the idea travels to other languages	73

Part 12 — Plain glossary recap	74
Closing	76
Declarations	77
Part 3: How the Whole System Works, Told as a Story	77
The library, the librarian, and the rules of the building	78
The question begins its journey	79
Stage 1 — Normalize: tidying the question before the search	79
Stage 2 — Two searches at once: the keyword hunt and the meaning hunt .	80
Assistant One: the keyword librarian (BM25)	80
Assistant Two: the meaning librarian (embeddings and FAISS)	81
Why run both	82
Stage 3 — Merge the two lists: combining two opinions fairly (RRF)	82
Stage 4 — Rerank: the careful reader who grades each finalist	83
This is where the Arabic squeeze finally bites	84
Stage 5 — The gate: deciding yes or no, and why one fixed answer fails Arabic	85
The fix that needs no retraining: let the bar set itself, per library	86
Stage 6 — Answer: and the clever cross-language retry	88
The cousin problem: the big library that hogs the shelf	89
The whole journey, in one breath	90
Part 6 — Fix Number One: The Self-Calibrating Gate, Step by Step	91
What this part is about, and why it comes first	91
What is a “gate,” in plain words?	92
What is a “tenant”? Why “per-tenant”?	93
The big trick: how do you set the right bar with no answer key?	93
Homemade stick number one: a guaranteed-GOOD pair	93
Homemade stick number two: a guaranteed-BAD pair	94
Putting the two sticks together: find the gap, climb a quarter of the way	94
Why this automatically gives Arabic a stricter, correctly-placed bar — without	
ever knowing it is Arabic	95
Being honest: the fix is real but partial, and it depends on the scorer	97
Walking the numbers: what the gate actually buys us	98
The honest ceiling: it does NOT beat the answer-key bars	99
A note on the quarter-of-the-way dial	99
And one more honest scope note on that headline	100
The translate-only-on-a-miss fallback, in plain words	100
Pulling Part 6 together	100

Why the Computer Seems Unfair to Arabic — and How We Fixed It Without Retraining the AI

The big, patient, plain-language version

Author: Ayman Kazim Yousef, Department of Artificial Intelligence Engineering, AlSafwa University, Karbala, Iraq.

A note from the author before we begin. This is a second, much longer telling of

a research paper I wrote. There is a shorter, more formal version full of the usual scientific language — tables, symbols, technical terms. This version is the opposite. It is written slowly and completely, for someone who has never studied computers, search engines, or artificial intelligence. I assume you know nothing about any of it, and I promise to explain every single piece of jargon in ordinary words the first time it shows up, usually with a picture you can hold in your head. I would rather repeat myself and be clear than be brief and lose you. Every number in this document is a real number that I measured; none of it is invented for effect. And I will be relentlessly honest about what I did **not** prove, because the honest version of a result is the only version worth having. Let us start at the very beginning.

Part 0 — The whole story in about one minute

Here is everything, as plainly as I can put it. Read this part first. If you read nothing else, read this.

What I built. I helped build a small digital library that lives entirely on a single ordinary laptop, with no internet needed. Think of it as a librarian you can talk to. You type a question — in Arabic or in English — and the computer searches through a pile of books, finds the passages that look most relevant to your question, and uses them to answer you. It was built for students at my university in Karbala, Iraq, where many people read and ask questions in both Arabic and English, and where you cannot count on a fast internet connection or an expensive cloud computer doing the heavy lifting for you. Everything runs on the machine in front of the student.

The thing everybody believes. There is a very common belief about systems like this. People say: *the computer is unfair to Arabic because it gives Arabic a lower score*. The idea is that when the computer rates how well a passage answers your question, it hands Arabic passages smaller marks than English passages, so Arabic gets pushed down the list and loses. This belief is so common that I believed it myself, and I even wrote it down in an early draft of this very project. It sounds obviously true.

The surprise — it is wrong. I decided not to just believe it. I measured it. And it turned out to be false. When I compared the scores the computer gives to genuinely-relevant Arabic passages against the scores it gives to genuinely-relevant English passages, Arabic was **not** lower. If anything, Arabic was a touch **higher**. So the popular explanation for the unfairness is simply not what is happening.

What is actually happening. The real problem is different and, once you see it, more interesting. The computer is perfectly capable of giving Arabic good scores. What it cannot do well is *tell Arabic passages apart from each other*. For English, the computer uses a wide range of scores — it has lots of room to say “this one is excellent, this one is okay, this one is poor.” For Arabic, it squeezes all of its scores into a narrow band. It is as if the computer has a ruler with plenty of marks for English but only a few marks for Arabic. With a crowded, mark-poor ruler, a great Arabic answer and a so-so Arabic answer end up looking almost the same. The information that separates good from mediocre gets crushed together. I call this **score squeezing** (the formal name is *score compression*). The scores are not lower — they are *packed tighter*. **One**

honest warning up front, which I will explain fully in Part 10.6: after I first found this, I tested it much harder — I added four more Arabic books, used three different AI models instead of one, and ran real statistics. The rock-solid result is that the *shape* of Arabic’s scores is **genuinely, measurably different** from English’s on every model (this is not luck or a small-sample fluke). But *which way* it differs turns out to be **unpredictable**: when I tested it properly — four more Arabic books, three different models, a fair clean comparison through the system’s own pipeline — the squeezing did *not* hold up. On the model the system actually uses, a varied Arabic library came out *wider* than English, not tighter (the early “squeezing” was really just a side-effect of my first sample being one small, very uniform book). So the honest headline is: “Arabic’s score-shape is significantly different from English’s, but in no fixed direction — it flips with the library, the genre, and the model.” And that is precisely why the fix (a per-library cutoff that *measures* each library’s actual shape and never assumes a direction) is the right one — the unpredictability is the whole reason it has to work that way.

Why squeezing causes trouble. The system has to draw a line somewhere — a cutoff that says “above this, accept the passage as a real answer; below this, throw it away as off-topic.” Picture a doorway with a fixed height. Anything tall enough to fit under the bar gets through. Now, that bar was placed using English’s wide, roomy range of scores. When you take that same bar and apply it to Arabic’s tightly-squeezed range, it lands in the wrong spot. It is the right height for an English-sized doorway and the wrong height for an Arabic-sized one. So the unfairness is real — but it lives in the *shape* of the scores, not in their *level*.

The fix — and what makes it unusual. Here is the part I am most proud of. I did not retrain the AI. Retraining means feeding a model huge amounts of new data to change its inner workings, which takes enormous computing power, costs money, and was simply impossible on a student’s laptop. Instead, I left the AI models completely frozen — untouched, exactly as they came — and built three pieces of ordinary engineering *around* them. This is the heart of the paper, and I will repeat it many times: **fairness here is achieved as engineering, not as retraining.**

The three fixes are:

1. **A self-adjusting cutoff for each library** that watches how that particular library’s own scores are spread out and places the accept/reject line to match. Because Arabic-heavy libraries have a squeezed, narrow spread, this method automatically gives them a stricter, better-fitted line — *and it never once looks at what language the text is in*. The fairness falls out on its own. On a fair test this lifted the share of accepted answers that were actually on-topic from 71 out of 100 up to 94 out of 100, while never throwing away a single correct answer, and it used no human-labeled examples at all.
2. **A small bilingual dictionary that grows by itself** so that an Arabic question can find the right English book even when the two share not a single letter. It lifts the chance of an Arabic question lexically finding its English match from zero up to certain, in about 48 millionths of a second, with no AI model running at that moment. I will be honest later that a modern AI search method already does this job too — so the dictionary’s real advantage is being faster, cheaper,

and easier to inspect, not doing something nothing else can do.

3. **A separate small shelf for each library** so that a tiny library (often the Arabic one) does not get buried when a giant library sits on the same shelf and hogs all the space. This rescues the small library’s results from collapsing. I will be honest that this problem is not really about Arabic at all — it is about sharing vocabulary with a big neighbour — and that the fancy AI search actually suffers from it *worse*, not better.

The honest size of all this. Everything I measured was on a small, English-heavy collection of books — only 63 Arabic chunks against 3,760 English chunks — on one laptop, with two specific frozen AI models. These results show you the *direction* of a problem and the *mechanism* behind it. They are not universal numbers you can quote about all Arabic everywhere. I will say this again and again, because overselling an Arabic-fairness result would be a betrayal of the whole point.

That is the entire story. The rest of this document just walks through it slowly, gently, and completely, so that every claim above makes full sense to you. Let us begin with what the system even is.

Part 1 — The problem, told slowly, assuming nothing

1.1 What is a “digital library that answers questions”?

Let me start with the most basic thing: what did I actually build, and what is it for?

Imagine you walk into a library. You do not know exactly which book has what you need. So you go to the librarian and ask a question: “How does compound interest work?” or “What is entropy?” A good librarian does two things. First, they find the right books and the right pages — the passages that actually talk about your question. Second, they read those passages and give you an answer in plain words, instead of just dumping ten books in your arms.

What I helped build is a *computer* that plays this librarian. You type a question. The computer hunts through a pile of stored books, pulls out the passages that look most relevant, and then writes you a short answer based on those passages. That is the whole idea. In the technical world this kind of system has a long name — “retrieval-augmented generation,” sometimes shortened to RAG — but you do not need that name. Just hold the picture of a librarian who first *finds* the right pages and then *explains* them.

I need to define two of those everyday words carefully, because the rest of the paper leans on them.

- **Retrieval** simply means *finding and pulling out* the relevant passages from the pile of books. When I say “the retrieval stage,” I mean the part of the system whose only job is to go fetch the passages that match your question. This is the librarian walking to the right shelf.
- **A passage**, or what I will often call a **chunk**, is just a small piece of a book — a

paragraph or so. Books are too big to handle whole, so the system chops each book into bite-sized chunks, and it searches and scores those chunks one at a time. When I say “the computer scores a chunk,” I mean it looks at one little paragraph and judges how well it answers your question.

So the simplest summary is this: you ask a question, the system finds the best chunks, and then it answers you using them. My research is almost entirely about that *finding* step — the retrieval — because if the system fetches bad chunks, then no matter how cleverly it writes the final answer, the answer is built on bad evidence. Garbage in, garbage out. If the librarian brings you the wrong pages, a beautifully worded summary of the wrong pages is still wrong.

1.2 Why “offline” and “on a laptop” matter so much

I keep saying this library runs *offline*, entirely on one ordinary laptop, with no internet. That is not a small detail; it shapes everything I was allowed to do. Let me explain why.

Offline means the computer does not phone a giant server somewhere on the internet for help. Everything happens on the machine in front of you. There are real reasons to want this. Internet in many places is slow, expensive, or unreliable. Sending a student’s questions and reading material off to a company’s far-away computer raises privacy worries. And a system that needs the internet simply stops working the moment the connection drops. An offline system keeps working in a classroom with no signal, on a bus, in a home with a spotty connection. For students in Karbala, where I am, this is not a luxury feature — it is the difference between a tool that works and one that does not.

On a modest laptop means the computer doing the work is small and cheap, not a room full of expensive machines. A normal student laptop has a modest processor (the “brain chip,” called a CPU) and no specialized AI accelerator (the expensive chip, called a GPU, that big companies use to train and run large AI models quickly). My whole system had to run on just the ordinary brain chip.

Here is the consequence that drives the entire paper. Because everything runs offline on a small machine, **I could not retrain the AI models**. Let me unpack “retrain.”

An AI model, in the sense I mean here, is a program that has *learned* its behaviour from being shown an enormous mountain of examples. To *retrain* it — to change what it has learned — you have to show it a new mountain of examples and let it grind through them, which takes massive computing power, a great deal of electricity, lots of time, and usually those expensive specialized chips. None of that exists on a student laptop in Karbala. So retraining the AI was off the table from the start. Whatever fix I found, it had to work *around* the AI models exactly as they came, leaving their insides untouched. I call models in this untouched state **frozen** — frozen meaning “not changed, not retrained, exactly as shipped.” This single constraint — *the models are frozen and I cannot retrain them* — is why my three fixes are all engineering tricks layered around the models instead of changes to the models themselves. Remember the phrase: **fairness as engineering, not retraining**. You will see it many times.

1.3 Two languages, one computer, and the seed of the problem

Now the part that makes this specifically about Arabic.

The students who use this library read and ask questions in *two* languages: Arabic and English. Their books are a mix of both. A single library might hold an English textbook on machine learning and an Arabic book on basic programming, side by side. So the computer has to handle both languages at once, and — this is important — it tries to do so with *one shared brain*.

What does “one shared brain” mean? The AI model that scores how relevant a passage is was not built separately for Arabic and for English. It is a single model that was taught roughly a hundred languages all at once, sharing the same internal capacity among all of them. Picture one teacher responsible for a hundred students at the same time. The teacher only has so much attention to go around. The students who get the most practice and the most class time — the popular, well-resourced languages, with English at the front of the line — get taught best. The students who get less time — and Arabic, despite being spoken by hundreds of millions of people, gets far less attention than English inside these models — are taught less well. This well-known trade-off has a name in the research world, the “curse of multilinguality,” but you only need the teacher-with-too-many-students picture: shared attention, unequally spread, with English favoured.

So before I measured anything, there was good reason to expect Arabic to come off worse than English somehow. The question was never *whether* Arabic was disadvantaged. The question was *in what exact way*, and *whether I could fix it without retraining the frozen shared brain*.

1.4 The very first disadvantage: Arabic gets chopped into more pieces

There is one disadvantage to Arabic that shows up before the computer scores anything at all, right at the front door, and it is worth understanding because it sets up everything that follows.

Computers do not read words the way we do. Before any AI model can work with text, the text is sliced into small units called **tokens**. A token is just a piece of a word — sometimes a whole short word, sometimes a chunk of a longer one, sometimes a single fragment. The slicing is done by a part of the system called a **tokenizer**, which you can think of as a paper-cutter with a fixed set of pre-approved shapes it is allowed to cut. If a word matches one of its shapes, it stays whole. If it does not, the cutter has to chop the word into several smaller approved pieces.

Here is the catch. The cutter’s set of approved shapes was designed mostly around English. So English words usually match a shape and stay fairly whole. Arabic words often do not match, so they get chopped into more, smaller pieces. There is a real linguistic reason: Arabic packs a lot into one written word — it glues on little prefixes and suffixes for “the,” “and,” “in,” “to,” and for “his,” “her,” and so on, and it usually leaves out the short vowels. An English-centric cutter has no neat shape for many of these glued-together Arabic words, so it shatters them.

I measured exactly how badly. Using the relevance model’s own cutter on my own

collection of books, an average English word got sliced into **1.536 pieces**, while an average Arabic word got sliced into **1.946 pieces**. Divide one by the other and Arabic is chopped into **1.27 times** as many pieces as English. In plain terms: for the same amount of meaning, Arabic is shattered into about a quarter again as many fragments as English.

Why does this matter? Two reasons, in everyday terms. First, the computer only has so much room to pay attention at once — a fixed budget of tokens it can hold in mind. If Arabic uses up more tokens to say the same thing, it spends more of that limited budget on the same idea, leaving less room for everything else. Second, when a meaningful word gets shattered into many little fragments, some of its meaning can get smeared across the pieces and become harder for the model to grasp cleanly. So Arabic starts the race a step behind, simply because it arrives in more, smaller, messier pieces. I want to be careful and honest here: this chopping penalty is a real disadvantage on its own, but I am *not* claiming I proved it directly causes the squeezing problem I will describe next. I measured both as facts. The chopping is the most natural explanation for *why* the squeezing happens, but I treat it as supporting evidence, not as a proven cause. Honesty first.

1.5 What could “the computer is unfair to Arabic” even mean?

Now we arrive at the central mystery. People say the computer is “unfair to Arabic.” But what would that actually *mean*, concretely? To solve a problem you have to pin down exactly what the problem is, so let me lay out the two very different things that sentence could mean.

To do that, I need to explain what a **relevance score** is. When you ask a question and the computer considers a particular chunk of a book, it produces a single number that says, roughly, “here is how well this chunk answers your question.” A high number means “great match.” A low number means “off-topic, ignore it.” The part of the system that produces this number is called the **scorer**. Everything downstream — whether a chunk is accepted as a real answer or thrown away — depends on this one number. So if there is unfairness to Arabic, it almost certainly lives somewhere in these numbers.

Now, here are the two possible meanings of “unfair to Arabic,” and they are genuinely different:

Meaning One — Arabic scores lower. This is the popular belief. It says: when the computer scores a genuinely relevant Arabic chunk, it hands out a *smaller* number than it would for a genuinely relevant English chunk. Picture two students, one Arabic and one English, who both wrote excellent essays. The teacher gives the English student 90 out of 100 and the Arabic student only 70 out of 100, for work of equal quality. That is a *level* problem — the whole scale is shifted down for Arabic. If this were the real problem, the obvious fixes would be things like adding a few points to every Arabic score to make up the gap, or — most naturally — retraining the model on more Arabic so it stops marking Arabic down.

Meaning Two — Arabic scores get squeezed together. This is a completely different kind of unfairness. It says: the computer might give relevant Arabic chunks

perfectly good numbers — even as good as English’s — but it cannot *spread them out*. It crams all its Arabic judgments into a narrow band, so a brilliant Arabic chunk and a merely-okay Arabic chunk both land on almost the same number. The teacher in this version is not marking Arabic *down* — the Arabic essays might even average a hair higher — but the teacher can only bring themselves to give every Arabic essay something between 78 and 82, while English essays get spread all the way from 40 to 95. That is a *spread* problem, not a level problem. And it calls for a completely different fix.

Here is why telling these two apart is the whole ballgame. **If the problem is Meaning One (lower scores), you fix it one way. If it is Meaning Two (squeezed scores), you must fix it a totally different way.** Guess wrong about which one it is, and your fix solves a problem you do not have while ignoring the one you do. So before doing anything clever, I had to actually find out which meaning was true. Not guess. Not assume. Measure.

1.6 The mystery, set up — and a confession

I want to set the mystery up honestly, including my own mistake, because the mistake is part of the lesson.

When I first started this project, I assumed Meaning One. I assumed Arabic scored lower. It is the natural assumption — everybody makes it, it sounds obviously right, and an earlier draft of my own work said exactly that. I was ready to “fix” a lower-scores problem.

But I am a careful person, and before building a fix I wanted to *see* the problem with my own eyes, in real numbers. So I ran a measurement, which I will describe gently in a later part. The short version is that I gathered a batch of Arabic chunks paired with questions they truly answer, and a matching batch of English chunks paired with questions *they* truly answer, and I let the frozen scorer rate all of them. Then I looked at the numbers two ways: their *level* (are the Arabic numbers lower on average?) and their *spread* (are the Arabic numbers packed into a narrower band?).

The result overturned my assumption. On the level question, Arabic was **not** lower — it came out a touch *higher*. The popular belief, and my own earlier draft, were wrong. But on the spread question, Arabic was dramatically narrower — packed into a band less than half as wide as English’s. The unfairness was real, but it was Meaning Two, not Meaning One. It was squeezing, not lowering.

I am telling you this confession on purpose, because it is the spine of the whole paper and I will hold to it from here on: **everyone assumes the computer is unfair to Arabic by scoring it lower; I measured it; that is wrong; the real unfairness is that the computer squeezes all Arabic scores into a narrow band, so it cannot tell a great Arabic match from a so-so one as clearly as it can for English.** Every part that follows builds on that one corrected idea.

And that corrected idea immediately tells us where the real damage happens. Remember that the system must draw a single line — a cutoff — to decide which chunks to accept and which to throw away. That line was placed using English’s wide, generous spread of scores. When Arabic’s scores are squeezed into a narrow band instead,

the very same line falls in the wrong place for Arabic. It is a doorway built for an English-shaped range, used on an Arabic-shaped range. *That* mis-placed line is the unfairness a student would actually feel — good Arabic answers wrongly rejected, or off-topic ones wrongly accepted — and it is the first thing my three fixes set out to repair, without ever retraining the frozen brain that does the squeezing.

That is the problem, told as slowly and completely as I know how. In the next parts I will show you exactly how I measured the squeezing, exactly what the numbers mean, and exactly how three plain pieces of engineering put the line back where it belongs — for Arabic and English alike — on a small laptop, offline, with no model ever retrained.

Part 2 — Every Word You Need, Explained Simply

Before we can talk about what went wrong and how I fixed it, we have to share a language. This part of the book is a small dictionary. I will take one idea at a time. I will say what each word means in the plainest words I can find, and then I will give you a picture from everyday life — a ruler, a doorway, a bookshelf — so that the idea sticks in your head and you never have to look it up again.

You do not need any background. You do not need to know mathematics, computers, or anything about how search engines work. I am going to assume you know nothing, and I am going to build everything up from the ground.

Read this part slowly. If you understand the words in this part, the rest of the book will feel easy, because the rest of the book is just these same words used together. I will repeat the important ideas more than once on purpose. Repetition is how a thing moves from “I read it” to “I know it.”

What a computer “search” is, in the first place

Let us start even lower than the dictionary, with the simplest possible picture, so that every later word has somewhere to live.

Imagine you own a big pile of books. A friend walks in and asks you a question: “Where do I find the part about how plants drink water?” Your job is to go to the pile, find the page that best answers the question, and hand it to your friend.

A computer “search system” — the thing this whole book is about — does exactly that job, but with thousands of pages and in a fraction of a second. You type a question. The computer goes through a huge pile of text and tries to hand back the passage that best answers you.

That is all it is. A question goes in. A passage comes out. Everything else in this book is about one stubborn problem: **the computer does this job well when the question and the books are in English, and noticeably worse when they are in Arabic.** Not hopelessly worse. Just worse in a specific, measurable, fixable way. The rest of this dictionary gives you the exact words to describe *how* it is worse, because

the “how” turns out to be surprising — it is not the “how” that almost everybody guesses.

A “score” — the number the computer gives to a guess

The plain meaning

When the computer looks at your question and at a candidate passage from the pile, it does not simply say “yes, this matches” or “no, it doesn’t.” Instead it gives the pair a **number**. This number is called a **score** (sometimes a “relevance score”). The bigger the number, the more the computer believes this passage answers your question. A small number means “probably not what they wanted.” A large number means “this looks like a strong answer.”

So the computer’s real behaviour is: take the question, take a passage, and produce one number that says *how good a match this is*. Do that for every candidate passage. Then the passage with the highest number wins, and that is the one handed back to you.

The everyday picture

Think of a strict but fair teacher grading short answers on a test. The teacher reads the student’s answer, reads the question, and writes a mark in the margin — say, 7 out of 10, or 2 out of 10. The teacher is not saying “right” or “wrong.” The teacher is saying *how right*. A 9 is a great answer. A 3 is a weak one. The mark is a measure of how well the answer fits the question.

The computer’s score is exactly that margin mark. The score is the computer’s grade for “how well does this passage answer this question.”

One honest complication you should know now

A real teacher’s marks are easy to read because we all agree what “out of 10” means. The computer’s scores are not so tidy. In this system there are actually two different kinds of grade the computer can hand out, and they live on two different scales:

- One kind of score is called a **cross-encoder** score (do not worry about the name yet; I define it later). Its grades can be almost any number, big or small, positive or negative — there is no fixed top mark. A very strong match might get a 10. A very poor one might get a minus 7. The scale just runs as far as it needs to.
- The other kind is called a **cosine** score. Its grades are squeezed to sit between minus 1 and plus 1, so it is more like a fraction. A strong match sits up near 0.8; a poor one sits down near 0.1.

The reason I mention both now is simple: throughout this book I check every important finding *twice*, once with each kind of grade, so that you can trust it is a real pattern and not a fluke of one grading scheme. If a thing shows up on both rulers, it is real.

Keep that habit in mind — “did it show up on both kinds of score?” — because I will keep coming back to it.

The most important warning about scores

Here is a thing that trips up almost everyone, including an earlier version of me. **A single score, on its own, tells you almost nothing.** If I tell you “this passage scored 8,” you cannot tell whether that is good or bad until you know what the *other* scores look like. Is 8 the top mark anyone ever gets, or do good answers usually get 40? A grade only has meaning *next to other grades*. The computer’s grades are like that: uncalibrated, which is just a long word meaning “the raw number has no fixed meaning until you compare it to the rest.” Hold on to this. The entire central discovery of this book depends on it. We are not going to ask “is Arabic’s score high or low?” We are going to ask “how are Arabic’s scores *spread out* compared to English’s?” — and that is a completely different question, with a completely different and more useful answer.

“Tokens” and why Arabic breaks into more pieces

The plain meaning

Before the computer can grade anything, it has to chop the text into small bites it can chew. It does not read whole words the way you do. It breaks each word into pieces. These pieces are called **tokens**. A token might be a whole short word, or it might be just a fragment of a longer word — a chunk of letters.

For example, the English word “cat” might stay whole as one token: cat. But a longer or rarer word like “unbelievable” might get broken into three tokens: un, believ, able. The computer then works with those pieces instead of the original word.

Why does it break words up at all? Because there are far too many possible words in the world to give each one its own slot. By keeping a fixed bag of common pieces and gluing them together, the computer can handle any word — even one it has never seen — by spelling it out of familiar fragments. It is a sensible trick. But the trick has a cost, and the cost falls unevenly on different languages.

The everyday picture

Think of a child building words out of letter blocks. If the child has a block that says “CAT,” the word “cat” takes one block. Easy. One piece. But suppose the child has no block for a long, unusual word and has to spell it out one or two letters at a time. Now that single word eats up five or six blocks. Same word, same meaning — but it cost far more blocks to build, because the toy box did not happen to contain the right big pieces for it.

That is exactly what happens to languages the bag of pieces was not really designed for.

Why Arabic gets the worse end of this

The bag of common pieces that these computer models use was built mostly from English and other European text. So it is full of pieces that fit English words neatly. It is *not* full of pieces shaped for Arabic.

Arabic, on top of that, builds its words in a way that creates a lot of variety from a small core. A single written Arabic word often carries, glued onto it, what English would write as several separate small words — a “the,” an “and,” a “to,” a “my,” all stuck onto one word with no spaces. And Arabic usually leaves out the short vowel marks, so the same string of letters can be stretched in several directions. The result is a huge number of distinct surface forms, and the English-built bag of pieces simply does not have a tidy block for most of them. So it falls back to chopping Arabic words into many small, less meaningful fragments.

The one number to remember here: 1.27 times

I measured this directly, using the very same chopping tool the computer’s grader actually uses, on the actual library this system runs on. Here is what came out:

- An English word, on average, breaks into about **1.536** pieces. So a typical English word is one piece, and now and then a longer one needs two — averaging just over one and a half.
- An Arabic word, on average, breaks into about **1.946** pieces — almost two pieces per word.

Put the two side by side: 1.946 against 1.536. Divide one by the other and you get about **1.27**. That single number is worth memorising, because it comes back again and again.

What does 1.27 actually mean, in plain words? It means that for the very same amount of meaning, Arabic gets shattered into about **a quarter more pieces** than English. If English carries an idea in four blocks, Arabic carries the same idea in about five. This has a real name in the field: it is called **fertility** — the average number of pieces a word breaks into. Higher fertility is worse, because “fertile” here means “produces more fragments,” and more fragments means the meaning is smeared thinner across more, smaller, less informative bites.

Why this matters down the line

Picture trying to recognise a friend’s face when the photo has been cut into four pieces, versus when the same photo has been cut into five smaller pieces. The five-piece version is a little harder to reassemble; each piece carries a little less of the face. Arabic arrives at the computer’s grader already cut into more, smaller pieces. The grader has to put the meaning back together from flimsier fragments. I will argue later that this is *one likely reason* the grader ends up less sure of itself about Arabic. I want to be honest right away, though: I measured the fragmentation, and I measured the unsureness, but I did not prove that the first *causes* the second. They sit next to each other, and the connection is natural and believable, but I am not claiming a chain of proof. I am claiming two measured facts and a sensible suspicion that links them.

A “relevance cutoff” or “threshold” — a doorway height

The plain meaning

Once the computer has scored the candidate passages and found the best one, it has to make a final yes-or-no decision: is even this best passage *actually good enough* to use? Because sometimes the honest answer is “nothing in the pile really answers your question,” and in that case the right move is to say so, not to shove a weak passage at you and pretend.

To make that yes-or-no call, the computer needs a line in the sand. A score above the line means “accept this — it is good enough to answer with.” A score below the line means “reject — nothing here is good enough.” That line is called the **threshold**, or the **relevance cutoff**, or just the **cutoff**. It is a single number. Beat it, you are in. Fall short, you are out.

The everyday picture

Picture the height bar at the entrance to a fast carnival ride — the painted hand or the bar that says “you must be at least this tall to ride.” A child walks up. If the top of their head clears the bar, they ride. If not, they wait. The bar does not care about anything except one measurement: are you tall enough or not.

The threshold is that bar. The passage’s score is the child’s height. Clear the bar, the passage is accepted and used. Fall under it, the passage is turned away and the computer admits it has no good answer.

Why the height of the doorway is the whole problem

Now here is the question that this entire book turns on: **where exactly do you paint that line?**

Paint it too low — set the bar too short — and almost anything gets through, including weak, off-topic passages. Your friend keeps getting handed pages that don’t really answer them. Paint it too high — set the bar too tall — and you turn away good passages that really would have helped, and the computer keeps saying “I have nothing” when it actually did.

So far, so reasonable. Here is the trap that this book is really about. The old system used **one single doorway height for everyone and everything** — one bar, painted once, used for every user and every language. And that one bar was painted by looking at how English scores behave. For English, that height sat in a sensible place. The trouble is that, as we will see, Arabic’s scores live on a differently shaped scale, so the very same bar — the same physical height — lands in the *wrong place* for Arabic. The bar did not move. The ground under it did. That mismatch is the heart of the unfairness, and the doorway-height picture is the picture to keep in your mind for it.

“Score compression” — a ruler with too few marks

This is the most important idea in the entire book. I am going to spend the most time on it, go slowly, and come at it from several angles, because if you understand this one thing, you understand the discovery. Everything else is a consequence of it.

First, the idea everybody believes — and why it is wrong

Almost everyone, when you say “the computer is unfair to Arabic,” pictures the same thing. They picture the computer handing Arabic *lower marks*. They imagine that a good Arabic passage gets, say, a 6 where the same-quality English passage would get an 8. Lower scores. The Arabic answer pushed down the page. That is the folk story. It is the story I myself believed and wrote down in an early draft. It feels obviously right.

I measured it. It is wrong. When I actually put guaranteed-good Arabic question-and-passage pairs in front of the grader and read off the marks, Arabic did *not* score lower than English. If anything, Arabic scored a tiny bit *higher*. On the cross-encoder grade, good Arabic pairs averaged about **10.534** and good English pairs about **10.448** — Arabic is a hair above. On the cosine grade, Arabic averaged about **0.805** and English about **0.754** — again, Arabic a touch above. The folk story is not just unproven; it is the opposite of what the numbers say. I will keep repeating this, because it is so easy to slide back into the wrong belief: **Arabic does not score lower. It scores equal, or slightly higher.**

So if the marks are not lower, where on earth is the unfairness? It is real — I will show you it is real — but it is not where everyone points. It is in the *spread* of the marks, not their *height*. That spread is what “score compression” names.

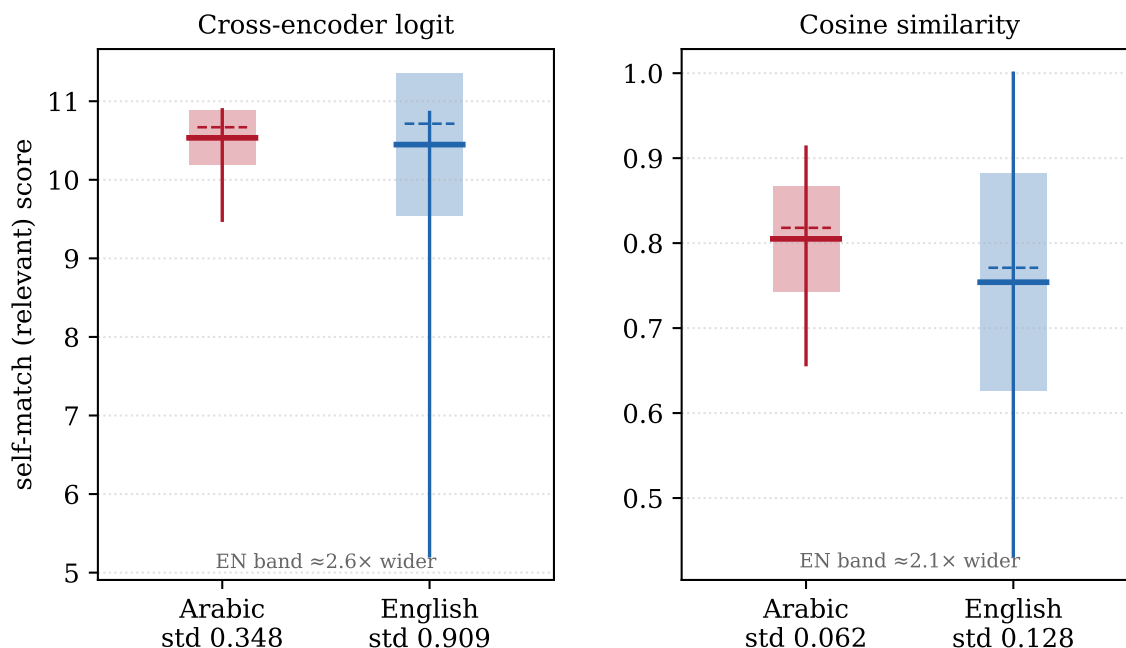
The plain meaning of compression

Score compression means this: the computer’s grades for Arabic are all crammed together into a narrow band, while its grades for English are spread out across a wide range.

Read that twice. It is not about the grades being low. It is about the grades being *close together*. For English, a great match and a so-so match get clearly different marks — the marks are spread far apart, so you can tell them apart at a glance. For Arabic, a great match and a so-so match get marks that are bunched up close together — so it is much harder to tell them apart, because the grader is using only a tiny slice of its scale for all of them.

The everyday picture: a ruler with too few marks

Figure 2. Score compression: Arabic relevant scores occupy a much narrower band — not a lower one (box = mean $\pm$ SD, whisker = min-max, dashed = median; Arabic mean is equal-or-higher)



Here is the picture to burn into your memory. Imagine two rulers.

The **English ruler** is a fine ruler, marked all the way along with many little lines — millimetres, lots of them. With this ruler you can measure two things and confidently say “this one is 3 millimetres longer than that one.” It has plenty of marks, so it can tell small differences apart. It can separate “good” from “great” from “okay” cleanly, because each of those lands on a different mark.

The **Arabic ruler** is a crude ruler. It has only a handful of marks, far apart, crammed into a short stretch. When you measure two Arabic things, they both fall between the same two distant marks, so the ruler shrugs and says “both about the same.” It *cannot* tell the small differences apart, not because the things are bad, but because **the ruler simply does not have enough notches in that region to register the difference.**

That is score compression. The Arabic grades are read off a ruler with too few marks. The grader is not being stingy with Arabic — it is being *imprecise* about Arabic. It hands out marks that are all jammed into a narrow band, so it loses the ability to clearly say “this Arabic answer is better than that one.” The grades are squeezed. Compressed. The dynamic range — the working distance from the lowest useful mark to the highest — has collapsed.

Now translate the actual numbers, slowly

In the field, the standard way to put a number on “how spread out are these grades” is a measurement called the **standard deviation**. You do not need the formula. You only need the meaning: **a big standard deviation means the grades are spread**

far apart; a small standard deviation means the grades are bunched close together. It is just a number for the width of the band. Big number, wide band. Small number, narrow band. That is the whole of it.

Now the measurements, in plain words:

- On the cross-encoder grade, the spread of good English marks had a standard deviation of about **0.909**. The spread of good Arabic marks had a standard deviation of about **0.348**. Compare them: 0.909 against 0.348. The English band is roughly **2.6 times wider** than the Arabic band. Put it in words: the English ruler has about two-and-a-half times as much room to work in. The Arabic marks are packed into a band less than half as wide. The grader has far fewer notches to tell a great Arabic match from a merely-okay one.
- On the cosine grade — the second ruler, the cross-check — the spread of good English marks was about **0.128** and of good Arabic marks about **0.062**. Compare: 0.128 against 0.062. The English band is roughly **2.1 times wider**. The same story, on a totally different scale, with a different grader. That is why I trust it. Two rulers, same verdict: **Arabic's good grades are squeezed into a band roughly two to two-and-a-half times narrower than English's.**

You can even see it in the extremes. On the cross-encoder grade, the English good marks stretched all the way down to about **5.214** at the low end and up to about **10.857** at the high end — a long reach, a wide spread, with room to spare. The Arabic good marks were all bunched up between about **9.485** and **10.891** — a short, tight huddle. Same idea: English has a long ruler; Arabic has a stubby one.

Why a narrow band is a problem even when the marks are higher

This is the subtle turn, so let me walk it carefully. You might think: if Arabic's marks are a touch *higher* and just packed close together, who cares? Higher is good, right?

Here is why it is a problem. Remember the doorway — the threshold, the bar you have to clear to be accepted. To place that bar well, you need a clear *gap* between the marks of the good passages and the marks of the bad passages. You want the good ones up high, the bad ones down low, and a comfortable empty space in between where you can paint your line. The wider that empty gap, the easier it is to place the bar correctly, and the more forgiving the system is of small errors.

Compression eats that gap. When the good Arabic marks are all squeezed into a narrow band, they sit closer to the bad marks. And — I measured this too — the *bad* Arabic marks are also pulled up and squeezed: the clearly-irrelevant Arabic pairs averaged about **-6.69**, while the clearly-irrelevant English pairs sat lower, around **-7.673**. So on both ends the bands move toward each other. The good band drops toward the bad; the bad band rises toward the good. The empty gap between “good” and “bad” — the only space a doorway can safely stand in — **shrinks for Arabic**. That is the real harm. Not lower marks. *A smaller margin between good and bad*. This is the next dictionary word, and it deserves its own entry.

“Separation” and “discriminability” — the gap between good and bad

The plain meaning

Separation (sometimes called the **separating gap**) is the empty space between the band of good marks and the band of bad marks. **Discriminability** is just a longer word for the same underlying ability: how well can the grader *discriminate* — that is, tell apart — a good passage from a bad one? High discriminability means good and bad get clearly different marks with a wide gap between them. Low discriminability means good and bad get marks that are close together, with little or no gap.

These two words are two sides of one coin. Lots of separation means high discriminability means easy to tell good from bad. Little separation means low discriminability means hard to tell good from bad.

The everyday picture

Imagine sorting ripe fruit from unripe fruit by colour. If the ripe ones are deep red and the unripe ones are bright green, you can sort them in the dark with one eye shut — the colours are far apart, the separation is huge, and you can draw your dividing line anywhere in the big empty space between red and green. That is high discriminability.

Now imagine the ripe ones are deep red and the unripe ones are a slightly-less-deep red. Now you are squinting. The two piles almost touch. Wherever you draw your dividing line, you are going to put some fruit in the wrong pile, because there is barely any gap to aim at. That is low discriminability — and that is Arabic’s situation. The “good” and “bad” piles for Arabic are pushed close together, so any line you draw is on shakier ground.

Tying it back

This is the operational meaning of all that compression talk. Compression (a narrow band of marks) causes shrunken separation (good and bad sit close together) which causes low discriminability (the grader can’t tell them apart cleanly) which causes the doorway to be hard to place (no comfortable gap to stand the bar in). One word leads to the next. The grader is not *meaner* to Arabic. It is *less decisive* about Arabic — less sure, fuzzier, working with a ruler that has too few marks and a gap that is too thin. That single sentence — *less decisive, not less generous* — is the whole discovery in eight words.

“Tenant” — each user’s private shelf

The plain meaning

A **tenant** is one user’s own private collection inside the larger system. The word comes from renting: a tenant rents an apartment and lives in it privately, separate from the other tenants in the building. In this system, each user gets their own private set of books, walled off from everyone else’s. The system keeps them strictly separate,

so one user can never see another user's books, and one user's search only ever digs through their own shelf.

So whenever you read "tenant," just think: *one particular user, and the private pile of books that belongs only to them.*

The everyday picture

A storage-unit facility. One big building, but inside it are many separate locked units, each rented by a different person. Your unit holds your things. Your neighbour's unit holds theirs. You have a key to yours and only yours. The building is shared; the contents of each unit are private. Each rented unit is a tenant.

Why tenants matter for this book

Tenants matter because different users keep different kinds of books, and that changes the shape of their scores. One user's shelf might be all English machine-learning books. Another's might be all Arabic. Another's a mix. And here is the payoff that the rest of the book builds on: because each tenant's shelf has its own character, each tenant's scores have their own spread — their own ruler. A shelf full of Arabic books will show the squeezed, narrow-band ruler we just described. A shelf full of English books will show the wide ruler. So instead of forcing one doorway height on everyone, the smart move is to let *each tenant get its own doorway, set to fit that tenant's own ruler*. That per-shelf doorway is the first of my three fixes, and "tenant" is the word that makes it possible to even talk about it. I will sometimes label specific tenants T1, T2, T3, and so on, simply as names — T3, for instance, is a mostly-Arabic shelf, and it ends up needing one of the strictest doorways of all, exactly because its ruler is the most squeezed.

"Frozen model" — we do not retrain the AI

The plain meaning

The "AI" at the centre of this system — the thing that reads a question and a passage and produces a score — is what's called a **model**. You can think of a model as a giant, already-trained machine: somebody, somewhere, fed it mountains of text long ago and tuned its millions of internal settings until it could grade text reasonably. Those internal settings are baked in. When we say the model is **frozen**, we mean exactly this: **we do not change a single one of those internal settings. We leave the machine precisely as it came. We do not retrain it.**

Retraining a model means going back in and adjusting its millions of internal settings using new data — a heavy, slow, expensive process that needs powerful hardware, lots of data, and real risk of breaking what already worked. A frozen model is the opposite: we accept the machine exactly as shipped, warts and all, and we build our fixes *around* it, in the ordinary plumbing that surrounds it.

The everyday picture

Imagine you bought a kitchen oven whose thermostat reads a little wrong — it runs hot. You have two choices. One: open the oven up, rewire the thermostat, recalibrate the internal electronics — risky, slow, and you might ruin the oven. Two: leave the oven exactly as it is, and simply learn that “when it says 200, it’s really 220, so I set the dial to 180.” You did not touch the oven’s insides at all. You built a small, reversible correction in the *world around* the oven. You wrote a note and you adjusted how you *use* it.

A frozen model is the oven left untouched. All three of my fixes are notes-on-the-fridge: they correct the system’s behaviour without ever opening the machine.

Why “frozen” is the backbone of this whole book

I want this to be impossible to miss, because it is the proudest claim in the book. **None of my fixes retrain the AI.** Every fix lives in the ordinary engineering around the frozen model — a small calculation here, a dictionary file there, a way of organising the shelves. This matters for very concrete, human reasons. This system runs offline, on a modest laptop, for students in Karbala, Iraq. There is no giant computer farm to retrain a hundred-language model on. There is barely enough Arabic data to retrain anything without making it worse. And a fix you can read, check, and undo is far safer and more trustworthy than a mysterious change buried inside millions of machine settings that nobody can inspect.

So the slogan of the book — and you will see it again — is: **fairness as engineering, not retraining.** We do not make the AI fairer by rebuilding the AI. We make the whole *system* fairer by being clever in the plumbing around a frozen AI. Frozen model is the word that lets me say that with a straight face.

“Cross-script” — Arabic letters and Latin letters never match

The plain meaning

A **script** is the set of letters a language is written in. English is written in the Latin script — a, b, c, and so on. Arabic is written in the Arabic script — a completely different set of shapes, written right to left. **Cross-script** describes any situation where the question is written in one script and the passage you are hunting for is written in the other: an Arabic-script question trying to find an English-script passage, or the reverse.

Here is the brutal, simple fact about cross-script matching by plain letter-overlap: **the two scripts share no letters at all.** Not a few. Zero. An Arabic letter is never the same shape as a Latin letter. So if your search method works by looking for shared letters or shared words between the question and the passage — and the simplest, oldest search methods do exactly that — then an Arabic question and an English passage have *nothing* in common to match on. The overlap is not small. It is exactly, structurally, unavoidably zero.

The everyday picture

Imagine two people trying to find each other's matching socks, but one person's socks are labelled only with numbers and the other's only with colours. You ask, "do any of your labels match any of mine?" The honest answer is: they *can't*, ever, because numbers and colours are not the same kind of thing. There is no possible overlap. Not because anyone did anything wrong — the two labelling systems simply do not share a single symbol. That is cross-script with plain letter-matching: a guaranteed zero, by the very nature of the two alphabets.

Why this is a different, separate problem

Notice this is a completely different harm from score compression. Compression was about a grader being *fuzzy* — it gave grades, just bunched-up grades. Cross-script letter-matching is about getting *no signal at all* — a flat zero, before any grading even happens. I call it a "structural" floor, meaning it is built into the structure of the situation and cannot be tuned away by adjusting a doorway, because there is no score there to adjust. You cannot pick a better height for a bar when nobody can reach the bar at all.

The honest part you must hear now

I will be straight with you, the way I am throughout the book. This zero-overlap problem is real, but it only bites the *oldest, simplest* kind of search — the kind that matches plain letters. There is a more modern kind of search (I'll describe it later) that understands *meaning* rather than letters, and it crosses scripts just fine on its own — it can already match an Arabic question to the right English book. So my fix for the cross-script problem, which is a little growing bilingual dictionary, is *not* the only way to cross scripts, and I do not claim it beats the meaning-based search on how many books it finds. What my dictionary fix buys is that it is astonishingly cheap and fast, it needs no heavy AI model running, you can read and edit every entry by hand, and it quietly learns new words as users hit walls. Speed, transparency, and self-teaching — not a recall miracle. I want you to know that before you are impressed by it, not after.

And one more honest wrinkle, so the picture is complete: the problem only runs in *one* direction in practice. An Arabic question struggling to find an English passage — that's the hard case, the one stuck at zero. But an English question looking for an Arabic passage already works, because Arabic technical books tend to sprinkle English terms straight into their text — the English word "entropy," the English formula symbols — sitting right there in Latin letters for an English question to land on. So the wall is one-sided. I report that asymmetry plainly rather than averaging it into a misleadingly tidy single figure.

“Tenant starvation” — a big neighbour hogging the shelf

The plain meaning

This is the last word in the dictionary, and it is about a fairness problem that, oddly, is not really about language at all — but it lands hardest on the small Arabic users, so it belongs here.

Tenant starvation is what happens when one user’s collection is enormous and another user’s is tiny, and they are searched through one shared pile. The huge collection crowds out the tiny one. When you search, the candidates that come back are dominated by the big collection simply because it is *big* — there is so much of it that it fills up all the slots, and the small user’s genuinely-good passages get shoved off the page before anyone even looks at them. The small user is “starved” of the chance to have their own results seen.

The everyday picture

Picture a shared bookshelf in a flat. One flatmate owns nine hundred books and crams them onto the shelf. Another flatmate owns ten books. You ask everyone to “grab the most relevant books on cooking,” and people reach onto the shared shelf and pull the first armful they touch. Almost every book they grab belongs to the flatmate with nine hundred, simply because nearly everything on the shelf is theirs. The flatmate with ten books has cookbooks too — good ones — but they are buried, one in ninety, and nobody’s hand lands on them. The small collection is starved, not because its books are worse, but because the big neighbour is hogging the shelf.

Why it lands on Arabic — and the honest caveats

In this system, the small, newly-added, easily-buried collection is *often* the Arabic one, simply because the Arabic libraries tend to be the smaller and newer ones, while a big English collection dominates the shared shelf. So in practice the starved user is frequently an Arabic user, and the fix — giving each user their own private shelf to search instead of one shared pile — clearly helps Arabic users. In my measurements, the small user’s good results collapsed badly under a dominant neighbour, recovering less than half of what they should; giving them a private shelf restored it completely.

But I have to be honest about what is really going on, because it would be easy to oversell this as “an Arabic-bias mechanism” and it is not, quite. The starvation is triggered by two collections **sharing the same vocabulary**, not by one of them being Arabic. If the small collection used totally different words from the big one, it would not get starved at all — I checked, and it stayed fine. It is the *shared academic words* that let the big collection’s many copies drown out the small one’s few. So this is better described as “a small collection that talks about the same topics as a big neighbour gets buried.” It just happens that, here, that small collection is usually the Arabic one. And one more honest point: a fancier meaning-based search does not rescue you here — in fact it starves *worse* than the simple one. The only real cure is the private shelf. I call this a fairness *precondition*: it is not the Arabic-grading bias itself, but if you don’t fix it, none of the other fixes even get a chance, because the good Arabic

passage was thrown away before the grader ever saw it.

A quick recap, in one breath each

You now have every word you need. Let me lay them out one last time, each in a single plain sentence, so they settle:

- **Score** — the computer’s grade for how well a passage answers a question; meaningless alone, meaningful only next to other grades.
- **Token** — a small piece a word is chopped into; Arabic words break into about **1.27 times more pieces** than English ones (this is called fertility), like a word built from more, smaller letter-blocks.
- **Threshold / cutoff** — the doorway height a passage’s grade must clear to be accepted; the old system used one height for everyone, painted from English, and it lands wrong for Arabic.
- **Score compression** — the central discovery: Arabic’s grades are squeezed into a band roughly **two to two-and-a-half times narrower** than English’s, like a ruler with too few marks, so the grader cannot tell a great Arabic match from an okay one as clearly.
- **Separation / discriminability** — the gap between the good grades and the bad grades; compression shrinks that gap, so it gets hard to tell good from bad — the grader is *less decisive about Arabic, not less generous to it*.
- **Tenant** — one user’s private shelf of books; each tenant has its own ruler, which is why each deserves its own doorway.
- **Frozen model** — the AI left completely untouched; we fix everything in the plumbing around it, never inside it — *fairness as engineering, not retraining*.
- **Cross-script** — an Arabic question and an English passage share zero letters, so plain letter-matching gives an unavoidable zero; a meaning-based search already crosses scripts, so my dictionary fix buys speed and transparency, not a recall miracle, and only the Arabic-to-English direction is stuck in the first place.
- **Tenant starvation** — a big collection hogs the shared shelf and buries a small one; it usually hurts the small Arabic user, but it is really caused by shared vocabulary, not by language, and only a private shelf cures it.

Keep these nine words close. From here on, the book simply uses them together to tell one honest story: the unfairness to Arabic is a *squeezed ruler*, not a *low grade*, and it can be fixed with careful plumbing around a frozen machine — no retraining required.

Part 4 — The four places where Arabic gets hurt

Before we can fix a problem, we have to know exactly where it lives. It is not enough to wave our hands and say “the computer is unfair to Arabic.” Unfair *where*? At which step? In what way? A doctor does not say “you are sick”; a doctor says “the infection is in your left lung.” This part of the paper is the careful diagnosis. We are going to walk the whole machine, step by step, and point at each spot where an Arabic question

gets a worse deal than an English one. There turn out to be **four** such spots, plus a **fifth** that hurts the same people but is not really about Arabic at all (I will explain why I keep it separate).

Let me first remind you, very plainly, what the machine actually does, because every one of the four spots lives at a particular step inside it.

A quick reminder of the machine

Imagine you are a student. You have a private digital library — a set of books that belong to you and only you. You type a question. The computer has to find the few sentences, out of thousands, that best answer you, and then hand those sentences to a small writing assistant that composes a reply. To find those sentences, the machine does several things in order:

1. **It chops your question and every passage into little pieces.** Computers do not read whole words the way you do. They cut text into “tokens” — small chunks, sometimes a whole word, sometimes half a word, sometimes a single letter. This chopping is called **tokenization**. Think of it as feeding text through a paper shredder before the machine can look at it.
2. **It searches two different ways at once.** One way is **word-matching** (the technical name is BM25). This is the old, simple, dictionary-style search: it looks for passages that literally contain the same words as your question. The other way is **meaning-matching** (a “dense” search using something called an embedding). This one tries to find passages that *mean* the same thing even if they use different words. The machine runs both and merges their results.
3. **It re-grades the top candidates with a careful judge.** A heavier, slower model called the **cross-encoder** (or “reranker”) looks at your question and each candidate passage together, as a pair, and gives that pair a **score** — a single number saying “this is how relevant I think this passage is to this question.” Higher number, more relevant. This is the step where the all-important *scores* are produced.
4. **It decides whether the best passage is good enough.** Finally there is a **gate** — a simple yes/no rule. It takes the highest score from the judge and compares it to a **cutoff** (also called a threshold). If the score clears the cutoff, the passage is “good enough” and is used to answer you. If it does not clear the cutoff, the machine decides it has nothing good and answers from general knowledge instead. The cutoff is exactly like the height bar at the entrance to a fairground ride: clear the bar, you get on; fall short, you do not.

Those four steps — shred, search, grade, gate — are where everything happens. And bias against Arabic can sneak in at four of these places. Let me take them one at a time, slowly, each with an everyday picture and the plain meaning of its number.

Spot 1 — The shredder cuts Arabic into more pieces (tokenization fragmentation)

Where it lives: the very first step, the shredding (tokenization).

The plain story. The shredder that cuts text into tokens was not built around Arabic. It was built mostly from English text, with other languages added in afterward. So its little pre-made pieces fit English words nicely but fit Arabic words badly. When it meets an English word, it can usually swallow it whole or in two clean halves. When it meets an Arabic word, it often has to fall back on lots of tiny scraps, because it has no ready-made piece that matches.

Why does Arabic give the shredder so much trouble? Because of how Arabic words are built. A single written Arabic word frequently carries, glued onto it, what English would write as several separate little words: “and,” “the,” “in,” “to,” and a possessive ending like “his” or “their.” Where English writes *and in the house* as four separate words, Arabic can pack the equivalent into one word with all those bits stuck on. On top of that, Arabic usually leaves out the short vowels in writing, so the same set of letters can be several different words, and the shredder has even less to grab onto. The result: one Arabic word gets shattered into more pieces than one English word.

An everyday picture. Imagine two people each tearing a sheet of paper into pieces, and you have to reassemble the sheets later. The first person tears English: a few big clean strips. The second person tears Arabic: many small confetti-like scraps. Both started with the same amount of writing, but the Arabic sheet now exists as far more, far smaller fragments. Putting the meaning back together from confetti is harder than from a few clean strips.

The number, and what it means. We measured this directly, using the *judge’s own shredder* — not some outside tool, but the exact shredder the relevance judge actually uses — on our own library. The result:

- English: on average **1.536 pieces per word**. (Roughly: most English words come out in one or two pieces.)
- Arabic: on average **1.946 pieces per word**. (Roughly: most Arabic words come out in about two pieces, often more.)

Divide one by the other and you get **1.27**. In plain words: **Arabic words are broken into about 1.27 times as many pieces as English words** — about a quarter more fragments for the same amount of meaning. That is the “fragmentation penalty.”

Why does extra fragmentation hurt? Two reasons, both simple. First, the machine has a fixed-size memory for how much text it can look at in one go (think of it as a desk that can only hold so many scraps of paper at once). If Arabic uses up more scraps per word, fewer actual words fit on the desk — so the machine effectively gets to read *less* of an Arabic passage than of an English one before the desk is full. Second, when meaning is spread thinly across many tiny scraps, each scrap carries less of the message, and the machine has to work harder to glue the message back together. More chances to glue it back wrong.

The honest caveat. This fragmentation is real and we measured it. But I want to be careful: I am *not* claiming this 1.27 number is some universal truth about Arabic everywhere. It is what we measured on *our* library with *this particular* shredder. Other libraries, other shredders, would give somewhat different numbers. The studies we lean on for the general idea — that English-built shredders treat non-English, richly-built languages unfairly [4, 36, 38] — survey many languages and do not publish

a single headline number for Arabic. So I use them for the *idea* and report our own 1.27 as our own *measured fact*, not as a law of nature.

And one more honesty point, which matters a lot for the rest of the paper: this fragmentation is the *first* harm, but it is **not the harm I spend the most effort fixing**. I treat it mainly as a *plausible cause* of the deeper harm that shows up two steps later — the squeezed scoring range. I am careful *not* to mix up the two. Fragmentation happens at the shredder; the squeeze happens at the judge. I can measure both. I cannot *prove* that the first causes the second — I can only say the direction makes sense. I will not claim more than that.

Spot 2 — The judge squeezes all Arabic scores into a narrow band (score compression)

Where it lives: the grading step, where the careful judge hands out scores.

This is the heart of the whole paper, so I will only introduce it here and then give it an entire part of its own (Part 5), where I will slow right down and work through it with a fully detailed example. For now, just the shape of the idea and the number.

The plain story. You would expect — almost everyone expects — that the judge is unfair to Arabic by giving Arabic *lower* scores. Lower scores would mean: the judge thinks Arabic passages are worse matches, ranks them below English ones, and lets fewer of them through. That is the obvious story. **It is wrong.** We measured it, and it is wrong.

What the judge actually does is stranger and more interesting. It does *not* give Arabic lower scores. If anything it gives Arabic relevant passages scores a tiny bit *higher*. But it gives all of them scores that are bunched together in a **narrow band** — a small range — whereas for English it spreads the scores out over a **wide range**. The judge is generous to Arabic on average but *indecisive* about Arabic. It cannot tell a brilliant Arabic match from a merely okay one nearly as cleanly as it can for English, because it crams all its Arabic verdicts into a tiny span of numbers.

An everyday picture (the short version — the long version is Part 5). Picture two teachers grading essays. One teacher uses the whole range from 0 to 100: terrible essays get 30, average ones get 65, brilliant ones get 95. You can instantly tell the students apart. The second teacher likes everyone and gives every single essay something between 78 and 85. A brilliant essay gets 84; a so-so essay gets 79. The second teacher is not being *mean* — the grades are actually a touch *higher* on average. But the second teacher is far less *useful*, because the grades are squashed into a tiny band and you can barely tell the students apart. That second teacher is how the judge treats Arabic.

The number, and what it means. We measure “how spread out” a set of scores is with a quantity called the **standard deviation** — a single number that says how wide the band of scores is. Big standard deviation, wide spread, lots of room to tell things apart. Small standard deviation, narrow spread, everything bunched together. Here is what we found for the scores of *guaranteed-relevant* passages (we will explain in Part 5 how we guaranteed they were relevant):

- For the careful judge (the cross-encoder): Arabic’s spread is **0.348**, English’s spread is **0.909**. English’s band is about **2.6 times wider**.
- For the meaning-matcher (the cosine score): Arabic’s spread is **0.062**, English’s spread is **0.128**. English’s band is about **2.1 times wider**.

In plain words: **the Arabic scores are packed into a band less than half as wide as the English scores**. The judge has fewer “notches” to work with for Arabic — fewer distinct levels with which to separate a great match from a so-so one. It is a ruler with fewer marks on it.

And the part that surprises everyone — the averages. The *average* score for relevant Arabic passages is **10.534**; for English it is **10.448**. Arabic’s average is *higher*, not lower (by 0.087). In the meaning-matcher, Arabic averages **0.805** and English **0.754** — again Arabic higher. So if you only checked the averages, you would conclude there is no Arabic problem at all. You would be completely wrong. The problem is hiding in the *spread*, not the average. This is the single most important sentence in the whole paper, and Part 5 is devoted to making it crystal clear.

I will leave the full working to Part 5. Here I have only planted the flag: the second spot where Arabic gets hurt is that its scores are squeezed into a narrow band — *score compression* — and this is the central finding.

Spot 3 — The single yes/no cutoff sits in the wrong place for Arabic (the mis-placed global threshold)

Where it lives: the gate, the final yes/no decision.

The plain story. Remember the gate is a height bar: a passage’s score has to clear the cutoff to be used. Now, where do you set that bar? The deployed system originally set *one single bar for everybody* — one cutoff used for every user and every language. It was hand-picked, and (naturally, since the library is mostly English) it was picked to sit in a sensible place for *English’s* wide spread of scores.

Here is the trap. A bar that sits in exactly the right place for a *wide* spread of scores sits in the *wrong* place for a *narrow* spread of scores. Because Arabic’s scores are squeezed into a narrow band (Spot 2), and that whole narrow band sits slightly higher up, a single bar chosen for English’s roomy geometry lands at the wrong relative height inside Arabic’s cramped geometry. The “good” Arabic passages and the “bad” Arabic passages are all packed close together near the top, and a bar set with English in mind cuts through them in the wrong place — either letting through junk or, in other settings, blocking good matches. The cutoff was calibrated on the wrong shape.

An everyday picture. Imagine a single doorway height that two families have to use. The first family is very mixed in height — toddlers to tall adults — so a doorway bar set at a certain height neatly separates “kids” from “grown-ups.” The second family are all nearly the same height, all clustered just under the ceiling. Put the *same* bar, chosen for the first family, in front of the second family, and it either lets everyone through or stops everyone, because they are all bunched together and the bar was placed for a *spread-out* group, not a *clustered* one. One doorway height cannot serve both families. One cutoff cannot serve both score-shapes.

The number, and what it means. When we let the system pick the *right* cutoff for each user separately (we will explain how in the mitigations), the correct cutoffs were genuinely different from one user to the next. On the careful judge, across five users, the right cutoffs ranged from **-3.29** (loosest) to **-1.39** (strictest) — about 1.9 units apart. The two users whose libraries *contain Arabic* needed the **strictest** cutoffs of all five: the Arabic-plus-math user at **-1.59**, the mixed Arabic-and-English user at **-1.39**. The purely-English users sat looser, at -2.93, -2.75, and -3.29. No single bar can sit correctly for everyone when the right bars are that far apart — and the Arabic-containing libraries are precisely the ones a one-size English bar would mis-serve.

The crucial cross-check (so we do not fool ourselves). A skeptic could say: “Of course the Arabic users got a higher, stricter cutoff — their scores must just be higher.” We checked, and that is *not* the reason. We looked at the bottom edge of each user’s relevant-score band — call it the floor of the “good” scores. The Arabic users’ floors are **not** lower than the English users’ floors; they are actually a touch *higher* (Arabic-plus-math 10.615, mixed 10.584, versus English 10.596 and 10.521). So the stricter Arabic cutoff is **not** caused by lower scores. It is caused entirely by the *tighter band* — the squeeze from Spot 2. This is the precise machinery by which “score compression” (a squeezed spread) — and not “lower scores” — forces the cutoff into the wrong place. I want to underline this because it is the thread that ties Spot 2 to Spot 3: the *spread*, not the *average*, drives the cutoff problem.

Spot 4 — A word-search across two alphabets always scores zero (the cross-script lexical zero)

Where it lives: the word-matching half of the search step (BM25).

The plain story. Half of the search is plain word-matching: find passages that contain literally the same words as the question. Now think about what happens when the question is in Arabic and the passage is in English. Arabic is written in Arabic letters. English is written in Latin letters. They **share no letters at all**. So a word-by-word match between an Arabic question and an English passage finds *nothing* — not “a little,” but *exactly zero*. There is no overlap to find, by the very nature of the two alphabets. This is a hard floor, not a soft weakness.

An everyday picture. Imagine trying to find matching cards between two decks, where one deck is printed in an alphabet you have never seen and the other in your own. You are told to match cards only when the *printed symbols* are identical. You will match nothing — not because the cards have nothing in common in meaning, but because the symbols simply never line up. Two languages that share no letters can never word-match, however close their meanings.

The number, and what it means. On our real bilingual library, an Arabic question trying to find the right English book by word-matching alone scored a recall of **0.00** — it found the right book *zero* percent of the time. Zero. That is the absolute floor of this kind of bias: you cannot fix it by adjusting a cutoff, because there is no signal there to adjust. There is nothing to threshold when the score is structurally zero.

The honest caveats (this one needs two). First, the harm runs in *one direction*

only. Going the *other* way — an English question looking for the right Arabic book by word-matching — already worked perfectly, recall **1.00**, with no help at all. Why? Because Arabic technical books tend to embed English technical terms right in the text, verbatim — formulas, code, names, loanwords written in Latin letters. So an English query finds those embedded English words inside the Arabic book. The disadvantage is therefore Arabic-question-finding-English-book only, and I report that lopsidedness honestly rather than averaging it into a single rosier number.

Second — and this is the most important honesty point about Spot 4 — the *meaning-matcher* (the dense search) already crosses the two alphabets perfectly well. A modern meaning-based encoder reaches recall **1.00** at the book level for Arabic questions finding English books, all by itself, with no dictionary, because it matches on meaning rather than letters. So it would be **false** to shout “the computer fails Arabic across scripts!” The word-matching half fails, structurally. The meaning-matching half does not. That means our fix for Spot 4 (a little bilingual dictionary, described later) does *not* win by improving recall — the meaning-matcher already maxes out recall. Its value is elsewhere: it is far cheaper and faster (about **48 microseconds**, millionths of a second, with no AI model loaded at all, versus tens of milliseconds per query plus minutes of one-time setup for the dense encoders), it is fully transparent and editable, and it can learn a user’s own special terms. So Spot 4 is a real, structural zero — but once a meaning-matcher is present, it is more a *cost-and-clarity* problem than a *recall* problem. I will not overstate it.

The fifth spot — a small library buried under a giant one (minority-library starvation)

I separate this one because, strictly speaking, **it is not bias against Arabic**. It hurts the same people, so it belongs in the paper, but the cause is not language. Let me explain it and then explain why I refuse to label it “Arabic bias.”

The plain story. Suppose two users share the same underlying search machinery, and one user’s library is enormous while the other’s is tiny. In our real setting one library held about 98.5% of everything and the other held about 1.5% — roughly 3,725 passages versus 58. Now suppose the two libraries talk about *overlapping subjects*, so they share a lot of academic vocabulary. When the small library’s owner searches, the word-matching search ranks results across *everybody’s* combined pile first and only afterward keeps the ones that belong to that user. Because the giant library owns almost all the passages that contain any given academic word, its passages fill up all the top spots, and the small library’s genuinely-relevant passages get pushed off the page *before* the machine even narrows down to the right owner. The small library starves.

An everyday picture. Picture a shared bookshelf where a giant neighbour has stacked 3,725 of their books and you have squeezed in 58 of yours, and you both write about similar topics so your spines use the same words. A librarian pulls “the top 5 books about this topic” off the *whole* shelf, then hands you only the ones that are yours. But the neighbour’s books so completely dominate the shelf that all five the librarian grabbed are theirs — and you get nothing, even though several of your 58 were perfect. The big neighbour hogged the shelf. That is starvation.

The number, and what it means. As the giant library’s dominance grew, the small library’s ability to recover its own correct top-5 passages collapsed: from **0.913** (recovering about 91% of its rightful results) at a 50/50 split down to **0.461** (recovering only about 46% — less than half) at the real 98.5% skew. Giving each library its *own private shelf to search* — a per-user sub-index — restored recovery to a perfect **1.0**.

Why this is not “Arabic bias” (three honest points).

1. **The cause is shared vocabulary, not language.** We ran a control where the small library used *distinct* vocabulary, not overlapping. It barely starved at all (it held steady around 0.91–0.97 across all the skews). So the trigger is *shared words under a dominant neighbour*, not Arabic-ness. It would harm *any* small library that shares vocabulary with a big one. In our bilingual academic setting, that small library is *often* the Arabic one — which is why it lands the same users — but the mechanism does not care what language anyone speaks.
2. **The meaning-matcher starves worse, not better.** You might hope “just use the modern meaning-based search and the problem goes away.” It does not. We measured the dense meaning-search under the same skew and it collapsed to **0.322** — *below* the word-search’s 0.461. So dense search is not a cure here; it is worse. Only giving each library its own private shelf fixes it.
3. **So I file this as a fairness precondition, not an Arabic-bias mechanism.** It is something you must fix so that everything else even has a chance to work — because no clever cutoff and no clever dictionary can rescue a passage that was thrown away before any of them ran. But it is honestly a “small-library-with-shared-words” problem that happens to land on Arabic users, not a property of Arabic itself.

The four-spot picture, in one breath

Let me gather the four real Arabic-bias spots into a single plain summary, because seeing them lined up is the whole point of this part:

- **Spot 1 (shredding):** Arabic words get cut into about $1.27\times$ more pieces. A first-order disadvantage, and the likely upstream cause of Spot 2.
- **Spot 2 (grading):** the judge squeezes Arabic scores into a band less than half as wide (about $2.1\text{--}2.6\times$ narrower), even though Arabic’s average is a touch *higher*, not lower. This is the central finding. *Score compression, not lower scores.*
- **Spot 3 (gating):** because Arabic’s band is squeezed and sits slightly high, one global yes/no cutoff chosen for English’s wide spread lands in the wrong place for Arabic — and the Arabic-containing libraries are exactly the ones that need the strictest, most different cutoffs.
- **Spot 4 (word-search):** an Arabic question word-matching an English passage scores a structural *zero*, because the two alphabets share no letters — though this is one-directional, and a meaning-matcher already crosses the gap, so it is more a cost/clarity issue than a recall issue.

And the fifth spot (a small library starved by a giant one) hurts the same users but is driven by shared vocabulary, not language, so I keep it honestly separate.

Spots 1, 2, and 3 are really one story told at three steps: a fragmentation penalty upstream, a squeezed score-band in the middle, and a mis-placed cutoff at the end, all linked. That linked story — and especially the surprise at its center — deserves its own slow, careful treatment. So let me now spend the next part doing exactly that.

Part 5 — The big surprise, explained very carefully

This is the most important part of the whole paper. If you remember nothing else, remember this. I am going to go slowly, repeat myself on purpose, and build the idea from the ground up with an everyday story, because the idea is genuinely counter-intuitive and almost everyone — including an earlier draft of this very project, written by me, which I now retract — gets it backwards at first.

Here is the one-sentence version, and then I will spend the rest of the part unpacking it:

Everyone believes the computer is unfair to Arabic because it gives Arabic *lower* scores. We measured it. That belief is wrong. Arabic relevant matches actually score a touch *higher*, not lower. The real unfairness is that the computer squeezes all Arabic scores into a *narrow band*, so it cannot tell a great Arabic match from a so-so one as clearly as it can for English — and a single yes/no cutoff set using English’s wide range therefore sits in the wrong place for Arabic.

Read that twice. Now let me earn it.

First, what a “score” even is, and why one number per pair

Recall the judge — the careful model that, given your question and one candidate passage, produces a single number saying how relevant the passage is to the question. Higher number, more relevant. That single number is the **score**.

Everything downstream hangs on that number. The machine ranks passages by it. The gate compares it to the cutoff. If the score is good, your answer gets grounded in real text from your library; if it is poor, the machine shrugs and answers from general knowledge. So the *quality* of that number — how trustworthy and how *informative* it is — decides whether you, the Arabic-reading student, get a good answer or a vague one.

Now, here is the key move. There are two completely different ways a scoring system can be unfair to a language. People almost always think of the first and almost never think of the second. The whole paper turns on telling them apart. So let me introduce them with a story that has nothing to do with computers.

The story of two graders

Imagine a school with two teachers who both grade the same kind of essay, on the same scale of 0 to 100.

Grader A uses the *whole* range. When Grader A reads a pile of essays, the weak ones come back around 40, the average ones around 65, the strong ones around 88, the brilliant ones around 96. Look at Grader A's pile of grades and you instantly see the shape of the class: there is a wide spread, low to high, and you can rank the students confidently. If you wanted to draw a line that says "everyone above this line passes with honours," you have plenty of room to place that line — say at 80 — and it cleanly separates the honours essays from the rest, because the grades are *spread out* and there is a real gap to cut through.

Grader B is a kindly soul who simply does not like giving low marks. Grader B reads the *same* pile of essays and gives every single one something between **78 and 85**. The brilliant essay gets an 84. The average essay gets an 80. The weak essay gets a 79. Notice three things about Grader B:

1. **Grader B is not stingy.** If anything, Grader B's grades are a little *higher* on average than Grader A's. The weak essay that Grader A scored 40, Grader B scored 79. Nobody could accuse Grader B of being harsh.
2. **But Grader B is nearly useless for telling students apart.** A one-point difference (84 versus 79) is doing the work that a fifty-point difference did for Grader A. The grades are crushed into a tiny band. You can *barely* tell the brilliant essay from the weak one, because they are only five points apart and the noise of grading easily covers five points.
3. **And here is the trap that ruins everything.** Suppose the school sets *one* honours line for *both* graders, and it sets that line by looking at Grader A's nicely-spread grades. Grader A's grades sensibly suggested a line at 80. Now apply that *same* line of 80 to Grader B's pile. Disaster. Grader B gave the average essay an 80 and the weak essay a 79 — so the line at 80 lets the average essay into honours and very nearly lets the weak one in too, while a genuinely brilliant essay at 84 is barely distinguished from them. The line was chosen for a *wide-spread* grader and is hopelessly mis-placed for a *narrow-spread* grader. It is in the wrong spot — not because Grader B was mean, but because Grader B's whole range is squashed and a line picked for the wide range lands in the wrong relative position inside the squashed one.

That is the entire idea of the paper, in a school. Hold onto it. Now I will map it, piece by piece, onto Arabic and English, and then drop in the real measured numbers.

Mapping the story onto Arabic and English

The judge in our machine plays *both* graders at once, depending on the language:

- When the judge scores **English** relevant passages, it behaves like **Grader A**. It uses a wide range. Strong matches score high, weaker matches score noticeably lower, and there is plenty of room to tell them apart. The English scores are *spread out*.
- When the judge scores **Arabic** relevant passages, it behaves like **Grader B**. It crams them all into a narrow band near the top. The strong Arabic match and the so-so Arabic match come out only a hair apart. The Arabic scores are *squeezed*.

And — exactly like Grader B — the judge is **not stingy** with Arabic. The Arabic scores are not lower. They are, if anything, a touch higher on average, just as Grader B's grades were a touch higher than Grader A's. So if you audited the judge by checking only the *averages*, you would see Arabic doing fine, even slightly better, and you would declare “no Arabic problem here.” You would be making the exact mistake the school would make if it judged Grader B only by Grader B's average grade — which looks perfectly healthy — while ignoring that Grader B can no longer tell students apart.

This is why I keep insisting: **the average fools you. The spread is what matters.** Let me say *why* the average fools you, in the plainest terms I can.

Why looking at the average fools you

An average is a single number that tells you where the *middle* of a pile sits. It tells you *nothing* about how *spread out* the pile is around that middle. Two completely different situations can have the very same average:

- Grader A's essays average, say, 72, spread from 40 to 96.
- Grader B's essays average about 81, all bunched from 78 to 85.

If a careless inspector only writes down “average grade” for each grader, the two graders look fine — even Grader B looks *better*, with a higher average. The inspector never notices that Grader B has lost the ability to *separate* students, because separation is not an average — it is a *spread*. The information that something is broken lives entirely in the spread, and the inspector threw the spread away by looking only at the average.

The same thing happened in our project. The naive belief — “Arabic scores lower” — is a statement about the *average* (it claims Arabic's average is below English's). We checked the averages, and the belief is simply false: Arabic's average is a touch higher. An inspector who stopped there would say “Arabic is fine.” But the real damage is in the *spread*, which the average completely hides. So the average does not just fail to reveal the problem — it actively *points the wrong way*, suggesting Arabic is slightly *better off*, when in truth Arabic is worse off in the way that actually matters for decisions.

This is the single most important reasoning step in the paper, so let me restate it three ways, because repetition here is worth it:

- **In one line:** the harm is in the spread, and the average hides the spread.
- **In the school:** Grader B's average looks healthy; Grader B's spread reveals the disaster.
- **In the machine:** Arabic's *average* score is fine (even a bit high); Arabic's *spread* of scores is squeezed, and that squeeze is the bias.

Why the spread is what matters — what a “spread” buys you

Let me make sure the word “spread” is fully concrete, because everything rests on it.

The point of a score is to let you *separate* good from bad. Separation needs *room*. If your scores run from 40 to 96, you have 56 points of room in which to place “good”

above “bad,” with a comfortable gap between them. If your scores run only from 78 to 85, you have 7 points of room — and into those 7 points you must somehow fit “brilliant,” “good,” “average,” and “weak,” all jammed together. There is no room to put a confident gap between them. The narrowness is the loss of separating power.

Here is a homely image for “spread.” Think of a ruler. A ruler with marks every centimetre lets you measure to the nearest centimetre. A ruler with marks only every ten centimetres cannot tell 3 cm from 7 cm — both round to “about 0 to 10.” A *wide* spread of scores is like a finely-marked ruler: many notches, fine distinctions possible. A *narrow, squeezed* spread is like a ruler with almost no marks: you can see *roughly* where something is, but you cannot tell two nearby things apart. The judge, for Arabic, is holding a ruler with too few marks. That is exactly what “less discriminative” means — *discriminative* here just means “able to tell things apart,” and a squeezed score-band cannot tell things apart, because it has too few effective notches.

So the spread matters because **the spread is the room you have to separate good from bad**, and separation is the entire job of a score. Take the room away — squeeze the spread — and the score still *exists*, still has a perfectly respectable average, but it can no longer do its one job well. That is the Arabic situation.

Why a cutoff set on the wide grader is wrong for the narrow grader

Now the third piece, which connects the squeeze to a real-world wrong decision. Remember the gate is a single height-bar cutoff: clear it, your passage is used; miss it, your passage is discarded.

Where should the bar go? Sensibly, it should sit in the *gap* between the good scores and the bad scores — high enough to keep the junk out, low enough to let the good stuff through. For Grader A, with grades nicely spread from 40 to 96 and a clear gap, you can place that bar confidently: somewhere in the empty middle. For English, the same — there is a wide gap between the spread-out “relevant” scores and the spread-out “irrelevant” scores, and a cutoff can sit comfortably inside it.

But now take *that very bar* — placed for the wide spread — and drop it onto the squeezed spread. Two things go wrong at once, and they compound:

1. **The good and the bad Arabic scores are bunched close together.** The squeeze does not only crush the “good” band; the “irrelevant” band also sits higher and tighter for Arabic, so the *gap* between good and bad — the empty space the bar is supposed to sit in — is itself much smaller. There is barely any gap to aim for.
2. **The bar was aimed using the wrong map.** Its position was chosen relative to English’s wide layout. Relative to Arabic’s cramped layout, that same position lands in the wrong spot — too low, so junk leaks through, or, in other circumstances, too high, so good matches get blocked. Either way, a bar placed by reading the wide map is mis-placed on the narrow map.

This is precisely Spot 3 from Part 4, and now you can see *why* it happens: it is a direct consequence of the squeeze in Spot 2. A single global cutoff cannot be right for both a wide spread and a narrow one, for the very same reason a single honours line

cannot be right for both Grader A and Grader B. The fix, which I describe fully in the mitigations, is to stop using one bar for everybody and instead let the machine *read each library's own score-spread and place that library's own bar inside its own gap* — give Grader B their own honours line, set from Grader B's own grades. But that is the cure; right now I am only making sure you see the *disease* with total clarity.

Now the real numbers, translated into plain meaning

I have told the story with made-up grades from 0 to 100 so the idea is clear. Let me now show you the *actual* measured numbers from our system and translate each one into exactly what it means in the grader story. The numbers are not on a 0-to-100 scale — the careful judge produces scores that can be negative or run past 10, and the meaning-matcher produces scores between roughly 0 and 1 — but the *story* is identical.

We measured the spread using the **standard deviation**, which is just a one-number summary of “how wide is this band.” Bigger standard deviation = wider band = more notches on the ruler = Grader A. Smaller standard deviation = narrower band = fewer notches = Grader B. Here is what we found for guaranteed-relevant passages:

The careful judge (cross-encoder): - English spread (standard deviation): **0.909** - Arabic spread (standard deviation): **0.348** - English is about **2.6 times wider** than Arabic.

What that means in plain words: the Arabic relevant scores are packed into a band less than half as wide as the English ones — in fact under 40% as wide. The judge has roughly two-and-a-half times *fewer effective notches* with which to tell a great Arabic match from a mediocre one than it has for English. It is the Grader-B ruler: too few marks to separate the students.

The meaning-matcher (cosine): - English spread: **0.128** - Arabic spread: **0.062** - English is about **2.1 times wider** than Arabic.

What that means: the same squeeze shows up in the completely different second scoring system. Arabic's band is again roughly half as wide. The fact that *both* scoring systems — built differently, on different principles — independently squeeze Arabic is what convinces me the squeeze is real and not a fluke of one model.

Now the averages, which are the part that refutes the naive belief:

The averages (where everyone expected Arabic to be lower): - Careful judge: Arabic averages **10.534**, English averages **10.448**. Arabic is **higher** by 0.087. - Meaning-matcher: Arabic averages **0.805**, English averages **0.754**. Arabic is **higher** by 0.051.

What that means: Arabic is *not* scored lower. In both scoring systems Arabic's average sits a little *above* English's. This is Grader B's healthy-looking average. An auditor checking only averages would conclude Arabic is doing fine, or even slightly better — and would completely miss the squeeze that is the actual problem. The averages do not just fail to reveal the bias; they gently point in the *wrong* direction.

And the part that makes the squeeze *bite*, the closing-in of the bad scores:

The irrelevant (“bad”) band sits closer for Arabic too (careful judge): - Arabic’s irrelevant scores average **−6.69**; English’s irrelevant scores average **−7.673**.

What that means: for English, the bad scores sit way down around -7.7 , far below the good scores up around 10.4 — a huge, comfortable gap to place a cutoff in. For Arabic, the bad scores are pulled *up* to about -6.7 , closer to the good band, *and* the good band is squeezed, so the gap between good and bad is smaller on both ends. Less room for the bar. This is the squeeze turning into a genuinely harder decision: not a lower good-score, but a *smaller margin* between good and bad. “Less discriminable” means exactly this — a smaller margin, a tighter call.

And finally, the cross-check that nails the diagnosis — the proof that the stricter Arabic cutoff comes from the *squeeze*, not from higher scores:

The bottom edge of each user’s good-score band (careful judge): - Arabic-plus-math user: **10.615** - Mixed Arabic-and-English user: **10.584** - English user 1: **10.596** - English user 2: **10.521**

What that means: the floor of the “good” Arabic scores is *not lower* than the floor of the “good” English scores — it is right alongside them, even a hair higher. So when the Arabic users end up with stricter cutoffs, it is provably *not* because their scores are lower. It is *only* because their band is *tighter*. Same floor, narrower band, so the same “place the bar a quarter of the way up the gap” rule lands the Arabic bar higher. The squeeze, and only the squeeze, drives the result. Lower scores have nothing to do with it — because there are no lower scores.

One honest wrinkle, stated plainly

I promised to be relentlessly honest, so here is the one place the clean story has a wrinkle. The squeeze is *clean and one-directional for the good (relevant) scores* — Arabic is tighter in *both* scoring systems, no exceptions. But for the *bad* (irrelevant) scores in the meaning-matcher only, the spread actually goes the *other* way: there English is slightly tighter than Arabic (English 0.097 versus Arabic 0.144). So I do *not* claim “Arabic is always the narrower band everywhere, no matter what.” That would be overselling. I claim the squeeze specifically and reliably for the **good/relevant** band — which is exactly the band that decides whether your real answer comes from your library — and I report the one exception rather than hide it.

Why this whole reframing matters — what it changes about the fix

Let me close this part by explaining *why* I have spent so long on a distinction between “lower” and “squeezed.” It is not pedantry. It completely changes what a sensible fix looks like.

If Arabic genuinely scored lower (the naive belief), the obvious fix would be to give Arabic a *bonus* — add some points to every Arabic score to lift it up to English’s level. That is a per-language patch: it bakes a “this is Arabic, add a bonus” rule into the code, it needs constant tuning, and it assumes a language-detector that is itself fallible. Worse, since Arabic does not actually score lower, a bonus would be *fixing a problem that does not exist* while ignoring the real one.

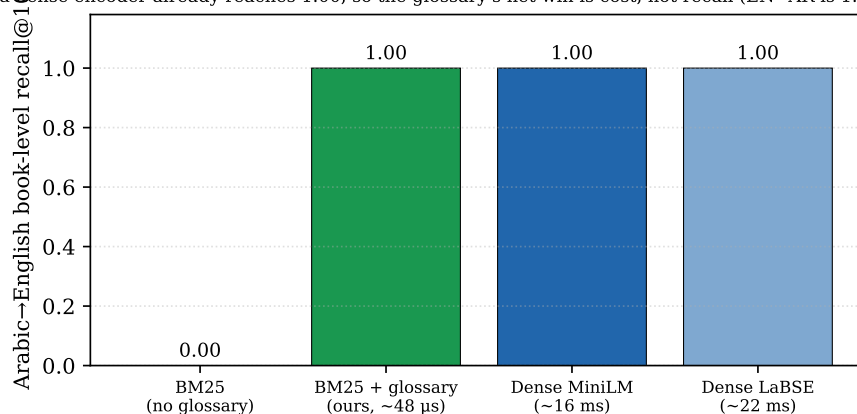
Because Arabic is actually *squeezed*, not lower, the fix is something quite different and, I think, more elegant. You do not push any scores around. You do not add a bonus. You do not even tell the machine which language it is looking at. Instead, you let the machine *read the shape of each library’s own scores* — how wide or narrow the band is — and place that library’s own cutoff to match its own shape. Give the narrow-band library a bar set from its own narrow band, exactly as you would give Grader B an honours line computed from Grader B’s own grades. A squeezed Arabic band then gets handled correctly *for the same reason* a spread-out English band does: not because anyone labeled it “Arabic,” but because the machine calibrated to the shape it actually saw.

That is the deep reason the central finding matters. “Arabic scores lower” would call for a clumsy, language-aware bonus. “Arabic scores are squeezed” calls for a clean, language-blind, shape-reading calibration — a fix that needs no retraining of the AI, no labels, no language detector, and that falls out automatically from simply respecting each library’s own score geometry. The honest motto of the whole project follows directly from getting this distinction right: **fairness as engineering, not retraining**. We do not rebuild the AI to be fair to Arabic. We build careful, ordinary machinery *around* the frozen AI so that fairness emerges on its own — and that is only possible because we correctly diagnosed the disease as a *squeeze*, not a *shortfall*.

Remember the one big idea: not lower, *narrower*. The average fools you; the spread is the truth. A bar set on the wide grader is wrong for the narrow grader. Everything in the rest of the paper is built on this single, carefully-measured surprise.

Part 7 — Fix #2: The Cross-Script Glossary (How an Arabic Question Finds an English Book)

Figure 6. Cross-script recall: the glossary lifts the structural lexical floor 0.00→1.00 — but a dense encoder already reaches 1.00, so the glossary’s net win is cost, not recall (EN→AR is 1.00 throughout)



7.1 A second, completely different problem

So far this paper has been about one big idea: the computer is not unfair to Arabic by giving it *lower* scores. It is unfair by *squeezing* Arabic scores into a narrow band, so it cannot tell a great match from a so-so one as sharply as it can for English. Fix #1 —

the self-calibrating gate — repaired that squeezing problem by setting a fair yes/no cutoff for each library based on that library’s own scores.

Now we turn to a *second* problem that is completely separate from the squeezing. It is not about scores being narrow. It is about a match that can never happen at all. This problem only shows up in one specific part of the search machine, and once you see it, it is almost embarrassingly simple. But it is real, and it has a clean, cheap fix.

Let me build it up slowly, because to understand the fix you first have to understand the part of the search engine where the problem lives.

7.2 Two ways the computer searches at the same time

When you type a question into our library system, the computer actually runs two different kinds of search side by side, and then blends the results. Think of it as asking two different librarians the same question and then combining their two answer lists into one.

Librarian One: the word-matcher. The first librarian is old-fashioned and literal. She matches *words*. If your question contains the word “entropy,” she goes and finds every page that literally contains the letters e-n-t-r-o-p-y. She does not understand meaning. She does not know that “entropy” and “disorder” are related ideas. She only knows: does this exact string of letters appear on this page or not? In the technical world this kind of search is called **BM25**. BM25 is just a careful, well-tuned recipe for “find pages that share words with the question, and rank the pages that share rarer, more distinctive words higher.” That is all it is — a smart word-overlap counter. We will keep calling her the word-matcher.

Librarian Two: the meaning-matcher. The second librarian is modern and clever. She does not match words; she matches *meaning*. She has been trained on enormous amounts of text in many languages, and she has learned that “entropy” and “disorder” point at roughly the same idea, and — crucially — that the English word “entropy” and the Arabic word that means entropy point at the same idea too. She turns every page and every question into a long list of numbers (a kind of “meaning fingerprint”), and she finds pages whose fingerprint is close to the question’s fingerprint. In the technical world this is called a **dense encoder** or **embedding model**. We will keep calling her the meaning-matcher.

The computer asks both librarians, gets two ranked lists, and merges them into one combined list. The blending step has a name — Reciprocal Rank Fusion — but the name does not matter here. What matters is this: **the final quality depends on both librarians doing their job**. If one librarian comes back with an empty list, the combined list is weaker, because half the search machine just contributed nothing.

Keep that picture. The cross-script problem is a problem with *Librarian One only* — the literal word-matcher. The meaning-matcher is fine. That asymmetry is the whole story of this Part, and it is also the source of the most honest caveat in the entire paper.

7.3 Why an Arabic question can NEVER word-match an English page

Here is the core difficulty, stated as plainly as I can.

Arabic is written in Arabic letters. English is written in Latin letters. **These two alphabets share no letters at all.** Not “few.” Not “some.” Zero. The Arabic letter that begins the word for “entropy” is not the same shape, not the same code inside the computer, not anything like the Latin letter “e.” To the literal word-matcher, an Arabic word and an English word are as different as a photograph and a sound recording.

Now think about what the literal word-matcher does. She matches *shared words*. So ask yourself: how many words does an Arabic question share with an English page?

Exactly zero. Always. By the very nature of the two alphabets.

This is not a question of the librarian being weak or badly trained. A perfect, infinitely smart literal word-matcher would *still* return zero, because there is genuinely nothing to match. The Arabic question is made of Arabic letters; the English page is made of Latin letters; there is no overlap to count. The word-matcher returns an empty list not because she failed, but because the task she does is *impossible* across two alphabets that share nothing.

An everyday analogy. Imagine you are looking for a book by matching the *exact shape of the handwriting* on its spine. If your search note is written in one alphabet and the book’s spine is written in a completely different alphabet, you will never find a shape-match — not because you searched badly, but because the two writing systems have no shapes in common. To bridge them you would need a translation step, a little card that says “this squiggle over here means that squiggle over there.” Without that card, the shape-matching method is dead on arrival.

The technical term for “query and page are written in different alphabets” is **cross-script**. (“Script” here just means alphabet/writing system.) And the technical way to say “the word-matcher returns nothing” is that the **lexical recall is zero**. “Lexical” means “to do with the literal words.” “Recall” means “of all the right pages out there, what fraction did we actually find?” A recall of zero means we found none of them.

We measured this on our real bilingual library — a library that genuinely contains both English technical books and Arabic technical books. We asked Arabic questions and checked whether the literal word-matcher could pull up the correct *English* books. The result was exactly what the alphabet argument predicts:

Arabic-question-to-English-book word-matching recall: 0.00.

Zero. Not low. Not nearly zero. Exactly zero, and zero *by construction* — meaning it is forced to be zero by the structure of the problem itself, before any model quality enters the picture. This is the lowest the bias can possibly go. You cannot tune a cutoff to fix it, because there is no signal there to tune. A cutoff decides “is this score good enough?” — but here there are no scores at all, because there are no matches at all. The list is empty.

This is a different kind of unfairness from the score-squeezing of Part 1. Score-squeezing is “the ruler has too few marks.” Cross-script zero is “the ruler is not even

touching the thing you are trying to measure.” Both hurt Arabic users, but they hurt in different places and need different fixes.

7.4 The surprising asymmetry: English questions already work

Before the fix, here is a genuinely surprising fact that we are careful to report honestly rather than hide.

The problem is **one-directional**. An Arabic question cannot word-match an English page — recall 0.00, as we just saw. But the *reverse* direction already works fine, completely unaided:

English-question-to-Arabic-book word-matching recall: 1.00.

A recall of 1.00 means “found all of them” — perfect. How can English questions find Arabic books when Arabic questions cannot find English books? The two alphabets still share nothing, so what is going on?

The answer is a small, real-world quirk of how technical books are written. **Arabic technical books are full of English words.** When an Arabic textbook on programming or mathematics discusses entropy, or logistic regression, or a variable named x , or a formula, it very often writes those technical terms *in English letters, right there on the Arabic page* — as identifiers, as formulae, as transliterated loan-words. The surrounding sentence is Arabic, but the technical term itself is sitting there in Latin letters.

So when an English question containing “entropy” arrives, the literal word-matcher scans the Arabic book, and — sure enough — finds the Latin-letter word “entropy” embedded in the Arabic text. Match found. The English-to-Arabic direction works because the Arabic books *already contain English words to match against*. The Arabic-to-English direction fails because English books almost never contain Arabic words to match against.

This is why we report this asymmetry instead of averaging the two directions into one tidy number. Averaging would say “cross-script recall is 0.50,” which would be misleading — it would hide the fact that one direction is already perfect and only the *other* direction is completely broken. The honest picture is: English questions are fine; **Arabic questions are the ones stranded at zero**. That is exactly the direction that hurts the Arabic-speaking student, which is why it is worth fixing.

7.5 The fix: a small living dictionary that grows itself

The fix is the simplest idea in the whole paper, and it has no AI model in it at all. It is a **bilingual glossary** — a little two-column dictionary. One column holds an Arabic technical term; the other column holds its English equivalent. “This Arabic word means entropy.” “This Arabic word means latent.” “This Arabic word means logistic.” And so on.

Here is how it is used. When an Arabic question comes in, before handing it to the literal word-matcher, the system looks up each Arabic term in the little dictionary. For every term it recognizes, it *adds* the English equivalent to the question. So a question

that started life as pure Arabic becomes a question that now also contains the English words “entropy,” “latent,” “logistic,” and whatever else was looked up.

Now hand *that* expanded question to the literal word-matcher. Suddenly she has English words to work with — and the English books are full of English words — so she finds them. The bridge is built. The translation card that the alphabet argument said we needed is exactly this little dictionary.

Two design choices make this safe and predictable, and both are worth understanding in plain terms.

It only ever ADDS words; it never removes them. The system takes the original question and *appends* the translations. It does not throw the Arabic away and replace it with English. This “only add, never remove” rule has a lovely guarantee baked in: **adding words can only ever help the word-matcher, never hurt her.** If a term is not in the dictionary, nothing is added for it, and the search is exactly as good (or as bad) as it was before. If a term *is* in the dictionary, the search can only improve, because the matcher now has one more true thing to match against. In technical language this property is called being “recall-monotone” — recall can only go up or stay the same, never down. In plain language: this fix has no downside. Worst case, it does nothing; it can never make matters worse.

It handles the messy glue of Arabic. Arabic sticks little words onto the front of bigger words — the equivalent of gluing “the,” “and,” “by,” “for,” “to,” “as” directly onto the start of a noun with no space. So the same root word can appear with several different prefixes stuck on, and a naive dictionary lookup would miss all the glued-on versions because they are not a literal match for the bare dictionary key. The glossary therefore peels these little prefixes off the front of each Arabic word before looking it up, so that the glued and un-glued forms all find the same dictionary entry. It also matches multi-word terms (some technical names are two or three Arabic words long) before it falls back to single words, so a phrase is recognized as a whole rather than chopped into unrelated pieces.

7.6 The result: from impossible to perfect, in 48 millionths of a second

When we put the glossary in front of the literal word-matcher and re-ran the same Arabic questions against the same library, the impossible became perfect:

Arabic-question-to-English-book word-matching recall: 0.00 → 1.00.

The literal word-matcher, who was previously returning an empty list every single time, now finds the correct English book every single time. The bridge works. And the English-to-Arabic direction stays at the 1.00 it already had, because the “only add, never remove” rule means the bridge cannot disturb the direction that was already fine.

And the cost? Almost nothing. The whole dictionary lookup-and-expand step takes about **48 microseconds** per question. A microsecond is a millionth of a second, so 48 microseconds is 48 millionths of a second — far faster than a single blink, far faster than you could ever notice. (To be exact, the typical time was 48.12 microseconds and even the slow cases stayed around 53.57 microseconds, measured over ten thousand

repeated runs, so it is reliably this fast, not just fast on average.) **And there is no AI model involved at all.** The glossary is just a lookup table — two columns of words sitting in a small file. No neural network has to wake up, load into memory, or run. It is the computational equivalent of glancing at a phrasebook.

The dictionary we actually deployed holds **308 Arabic-to-English term pairs** and **280 English-to-Arabic pairs** — a few hundred technical terms, enough to bridge the academic vocabulary our students actually use.

7.7 The “living” part: it teaches itself new words, once, forever

A fixed dictionary of a few hundred words is useful, but it has an obvious limit: the day a student asks about a technical term that is not in the dictionary, the bridge is missing again and that Arabic question falls back to recall zero. So the glossary does something more interesting. It **grows itself** from its own failures. This is the “continual self-extending” part of its name, and it is worth walking through slowly because it is genuinely clever and yet completely model-free in its ongoing operation.

Here is the idea. **A failed search is itself the signal to learn.** When an Arabic question for a single technical term comes in and finds *nothing* — an empty list, the tell-tale sign of a missing bridge — the system treats that emptiness as a teaching moment. It quietly does three things:

1. It calls a translation tool **one time** to translate that one unknown Arabic term into English.
2. It re-runs the search with the freshly translated English word added, so the student still gets their answer for the question they just asked.
3. It **writes the new pair into the dictionary** — “this Arabic term means this English word” — peeling off any glued-on prefix first so the stored key is the clean root.

From that moment on, the new term is a permanent dictionary entry. The next time anyone asks about that term, it is looked up instantly and for free, exactly like the original few hundred terms. **The translation tool fires at most once per term, ever — the first time the term is ever missed. After that, the word is known forever, and the lookup is pure dictionary, no model.**

An everyday analogy. Imagine a receptionist who keeps a phrasebook. The first time a visitor asks for something the phrasebook does not cover, the receptionist makes one phone call to a translator, helps the visitor, and *writes the new phrase into the phrasebook in pen*. From then on, every future visitor with the same request is helped instantly from the book — no more phone calls. The phrasebook quietly gets better every time it is found lacking, and the expensive phone call happens only once per phrase.

We tested this self-teaching directly. We took three Arabic technical terms that we had *verified were missing* from the dictionary — the Arabic words for **entropy**, **latent**, and **logistic** — and asked about each one. Each started at recall **0.0** (missing bridge, empty list, as expected). After a single self-teaching event each one jumped to recall **1.0** and stayed there, fast and model-free, on every subsequent ask.

Three guard-rails keep this self-teaching honest and safe, and they are simple:

- It **never overwrites** a human-curated entry. The hand-checked terms are protected; the machine can only add new words, never corrupt the trusted ones.
- It **rejects junk**. It refuses to learn a “term” shorter than three characters (too short to be a real word) or a “translation” longer than sixty characters (that is a sentence, not a term — something went wrong).
- It **stops growing at 5,000 learned terms**, so the dictionary file cannot balloon without limit and eat the device’s storage.

And here is the conceptual point I want to hammer, because it is the same spine as the rest of the paper. **This is learning without retraining the AI.** Nothing about any neural network changes — not one weight, not one number inside any model. What changes is a small, plain-text dictionary file that a human could open and read. This makes the learning *auditable* (you can see exactly what was learned), *reversible* (you can delete a wrong entry by hand), *crash-proof* (it survives restarts because it is just a file on disk), and immune to the way AI models sometimes “forget” old skills when they learn new ones (there are no weights to overwrite, so nothing old can be clobbered). It is fairness improving over time as a property of ordinary file-based engineering, not as a training run.

7.8 The honest caveat: the meaning-matcher already does this

Now the most important honesty in this Part — and I want to state it as bluntly as the scientific paper does, with no softening, because the temptation to oversell an Arabic-fairness result is exactly the trap this whole project refuses to fall into.

Remember there were *two* librarians: the literal word-matcher and the modern meaning-matcher. Everything above is about repairing the literal word-matcher, who was stranded at zero. But **the meaning-matcher was never stranded**. The modern meaning-matcher, which matches by fingerprint-of-meaning rather than by letters, *already* crosses scripts perfectly well. We measured it: the meaning-matcher finds the right English book for an Arabic question at recall **1.00**, all on its own, with no glossary at all. We even tested a much bigger, heavier meaning-matcher (a 1.8-gigabyte model called LaBSE), and it also reaches **1.00**.

So I must say plainly: **at the level of “did we find the right book,” the glossary’s extra benefit over the modern meaning-matcher is essentially nil.** It is simply false to claim “modern AI search fails Arabic across scripts.” It does not fail. It already gets there. If a system already runs the modern meaning-matcher, the glossary does not improve the chance of finding the right book, because that chance is already 1.00 and you cannot beat perfect.

Here is the comparison laid out side by side:

Method	Arabic→English	English→Arabic	Cost per question	AI model?
Word-matcher, no glossary	0.00	1.00	tiny	none

Method	Arabic→English	English→Arabic	Cost per question	AI model?
Word-matcher + glossary (our fix)	1.00	1.00	+48 microseconds	none
Meaning-matcher (the one we deploy)	1.00	1.00	~16 milliseconds	yes (~118-million-piece model)
Meaning-matcher (heavy, LaBSE)	1.00	1.00	~22 milliseconds	yes (~471-million-piece, 1.8 GB)

So if the glossary does not win on finding-the-right-book, why build it at all? Because its real value lies in four other places, and they are genuine:

1. **Speed and cost.** The glossary takes about 48 microseconds and loads no model. The meaning-matcher takes about 16 to 22 *milliseconds* per question — that is hundreds of times slower per question — and on top of that it must do a one-time job of computing a fingerprint for every page in the whole library before it can search at all, which took about 92 seconds for the smaller meaning-matcher and about 453 seconds (more than seven minutes) for the heavy one. On a modest offline laptop with no cloud to fall back on, “free and instant and needs no model in memory” is worth a great deal.
2. **It feeds the literal half of the search.** Recall there are two librarians and their two lists get blended. Even when the meaning-matcher would have found the book anyway, having the word-matcher *also* contribute a real, correct candidate makes the blended list sturdier. The system does not have to lean its entire weight on the meaning-matcher for every cross-script question; the literal half is pulling honestly too.
3. **You can read it and edit it.** The glossary is a plain list of word pairs. A human can open it, check it, correct a mistranslation, or add a term by hand. The meaning-matcher is a black box of millions of numbers that no human can inspect or hand-edit. For a deployed system serving students, that transparency is a real, practical good.
4. **Tenant-specific words.** Different libraries use different specialist vocabulary, and the self-teaching loop quietly tailors the glossary to the terms each library’s users actually ask about.

So the honest one-sentence summary of Fix #2 is: **the glossary does not beat modern AI search at finding the right book — it ties it — but it does so far faster, for free, transparently, and while teaching itself new words, which makes it the right tool for a cheap offline device even though it is not a recall miracle.** That is the truth, and we say it plainly rather than dressing up a tie as a win.

Part 8 — Fix #3: The Per-Library Shelf (How a Small Arabic Library Avoids Being Buried)

8.1 A third problem, and the most subtle one

We now reach the third and final fix. Like the cross-script problem of Part 7, this one is *not* about the score-squeezing that is the spine of the paper. And like the cross-script problem, it is not, strictly speaking, a problem *about Arabic* at its root — it is a problem about *small libraries that share vocabulary with big ones*, and it just so happens that in our deployment the small library is often the Arabic one. I want to be honest about that framing from the very start, because overstating it as “an anti-Arabic bug” would be wrong, and this paper does not do that.

But the harm is real, it lands on Arabic users in practice, and it must be fixed *before* the other two fixes can even do their jobs. So let me build it up carefully.

8.2 What a “tenant” is, and how the shelves are shared

Our system serves many separate users, and each user has their own private library. In the technical world each such private library is called a **tenant** — think of it as one renter who has their own locked room of books, completely walled off from every other renter. One student’s library is one tenant. Another student’s library is another tenant. A user can only ever see and search their own tenant’s books; strict walls keep them apart.

So far so good. The walls between tenants are solid. The problem is not that the walls leak. The problem is hidden in *how the literal word-matcher counts*, and to see it you have to understand one more thing about how that librarian decides which words are important.

8.3 The hidden trap: how the word-matcher decides what is “important”

Recall the literal word-matcher (BM25) from Part 7. She ranks pages by shared words, but she is cleverer than a plain word-counter in one specific way: **she gives more weight to rare, distinctive words and less weight to common ones**. The word “the” appears on nearly every page, so matching it tells her almost nothing. The word “entropy” appears on very few pages, so matching it is a strong, distinctive signal. She decides how rare a word is by looking across *the whole collection* and asking: “in what fraction of all the pages does this word appear?” A word in very few pages is treated as precious; a word in very many pages is treated as cheap.

Now here is the trap. **In the original design, “the whole collection” meant every page of every tenant mashed together into one giant shared pile**, and only *after* searching that giant pile did the system filter down to the one tenant who actually asked. Two librarians-worth of cleverness, one giant shared shelf, filter at the very end.

That sounds harmless. It is not, and here is exactly why.

8.4 How the giant library buries the small one

Picture two tenants sharing that one giant pile. One tenant is huge — thousands of pages. The other is tiny — a few dozen pages. In our real measured deployment, one tenant owned **98.5%** of all the pages and the other owned just **1.5%** (that is 3,725 pages versus 58 pages — a staggering imbalance). Now suppose, as is common with academic books, the two libraries *share a lot of vocabulary* — both contain the words “function,” “model,” “equation,” “variable,” and so on, because they are both technical libraries.

When a user from the *tiny* library asks a question, here is the sequence of harm:

First harm — the rarity-judgment gets hijacked. The word-matcher judges how “precious” each word is by looking across the whole giant pile. But the giant pile is 98.5% owned by the big tenant, so it is the *big* tenant’s books that decide what counts as common or rare. A word that is rare and distinctive *within the small Arabic library* might be common *across the giant English-dominated pile*, so the matcher treats it as cheap and down-weights it — even though, for the small library’s user, it was a strong, distinctive signal. The big neighbour’s statistics have effectively taken over the dictionary’s sense of what matters. In technical terms this is called **statistics capture** — the dominant tenant captures the corpus-wide rarity counts. In plain terms: the big library gets to decide which words are “interesting,” and it decides based on its own books, not the little one’s.

Second harm — the small library’s pages get crowded out before the filter even runs. The system searches the whole giant pile first and keeps the top results, *then* filters down to the asking tenant. But because the big tenant owns 98.5% of the pages and shares vocabulary with the small one, the big tenant’s pages flood the top of the combined results. By the time the system filters down to “only the small tenant’s pages, please,” most of the small tenant’s correct pages have already been shoved off the end of the list and thrown away. They were discarded *before* anyone checked whose library they belonged to. This is called **candidate crowding** — the big tenant’s sheer volume crowds the small tenant’s pages out of contention before the filter ever runs.

An everyday analogy. Imagine a library where one enormous donor’s collection and one tiny local collection are shelved together on one set of shelves, sorted purely by some global popularity ranking. When you ask for the ten most relevant books and *then* say “oh, but only show me the ones from the tiny local collection,” you find that the giant donor’s near-duplicates filled all ten slots, and the local collection’s books — which were genuinely relevant — never made the shortlist, because they were ranked against the donor’s mountain and lost. You filtered too late. The relevant local books existed; they were simply buried before you got to look for them.

8.5 Measuring the burial: 0.913 collapses to 0.461

We measured exactly how badly the small library gets buried as the big one grows. The measure we use is called **overlap-at-5**: of the five pages the small library *should* have surfaced for a question (the five genuinely best ones in that small library), how many did the shared-pile search actually recover before throwing the rest away? A

score of 1.0 means all five — perfect. A score of 0.5 means only half of the right pages survived; the other half were buried.

Here is what happens as the big tenant’s share of the pile climbs from half to almost everything:

Big tenant owns...	Right pages recovered (of 5)	Overlap-at-5
50%	4.72	0.913
80%	4.33	0.833
90%	3.53	0.707
95%	3.11	0.631
98%	2.29	0.483
98.5% (our real deployment)	2.11	0.461

Read that bottom row slowly. At the real skew in our actual deployment, the small library recovered only **2.11 of its 5 best pages** — an overlap of **0.461**, meaning **less than half** of the small library’s genuinely-best pages survived to be considered. More than half of the correct evidence for that user’s questions was thrown in the bin *before* the smart reranker, the gate, or anything else ever got to look at it. The small (often Arabic) library’s user was being quietly served the wrong half of their own books, and no amount of clever scoring downstream could rescue pages that had already been discarded.

Notice the trend, too: when the two libraries were the same size (50/50), overlap was a healthy 0.913 — the burial barely happened. It is *specifically the growing imbalance* that drives the collapse from 0.913 down to 0.461. The bigger the neighbour, the deeper the burial.

8.6 Proving it is vocabulary, not language, and not size alone

Here is where I must be scrupulously honest, exactly as the scientific paper is. It would be tempting to shout “the system buries Arabic libraries!” But that is not quite the true cause, and we ran a careful control test to find the *real* trigger.

We re-ran the whole experiment with a small library that did **not** share vocabulary with the big one — a small library on a totally different subject, using totally different words. And we watched what happened as the big tenant grew to dominate.

The result: the distinct-vocabulary small library **barely suffered at all**. Its overlap stayed essentially flat — around 0.968 down to 0.914 — even as the big tenant swelled to 98.5%. It was *not* buried.

This is the crucial control, so let me spell out what it proves. The small library that *shared* vocabulary with the big one collapsed to 0.461. The small library that did *not* share vocabulary held firm near 0.9. The only difference between them was shared vocabulary. Therefore:

- **The cause is shared vocabulary, not language.** A small library is buried only when it competes for the same words as the giant. A small library with its own distinct words is safe.

- **The cause is not merely being small.** Both small libraries were equally tiny. Only the vocabulary-sharing one was buried. Smallness alone does not do it; smallness *plus shared words* does.

So the honest framing is this: **this is not an anti-Arabic bug in the scorer. It is a “small-library-that-shares-words-with-a-big-one gets buried” bug, and in our bilingual deployment the small library that shares academic vocabulary with the dominant English one is, very often, the Arabic one.** The Arabic library suffers *in practice* because of where it sits — small, and sharing the universal vocabulary of mathematics and science with a much bigger English neighbour. We fix it because it harms our Arabic users, but we describe its true cause accurately rather than dressing it up as something it is not.

8.7 The fix: give every library its own little shelf

The fix follows directly and almost obviously from the diagnosis. The whole problem came from searching *one giant shared pile first* and filtering to the right tenant *last*. So we simply flip the order: **filter to the right tenant FIRST, then search.**

Instead of one giant shared index over everybody’s pages, each tenant gets its own small private index built over *only its own pages*. The technical name is a **per-tenant sub-index**, but the idea is just: give every library its own little card catalogue covering only its own books.

Once you do this, both harms vanish at once:

- **The rarity-judgment is now honest.** Because the index covers only the small library’s own pages, the word-matcher judges how “precious” each word is using *the small library’s own books* — not the giant neighbour’s. The big tenant’s statistics can no longer hijack the sense of what is interesting. Statistics capture is gone.
- **There is no crowding.** Because the search only ever looks at the small library’s own pages, the big tenant’s pages are simply not in the room. They cannot crowd out anything, because they were never candidates in the first place. The “top results” *are* the small tenant’s top results, by construction.

And the result is exactly what you would hope: the burial disappears completely.

Small-library overlap-at-5: 0.461 → 1.000, at every level of imbalance.

A perfect 1.0 — every one of the small library’s five best pages is recovered, whether the big tenant owns 50% or 98.5% or 99.9% of the overall corpus. The imbalance simply stops mattering, because each library is searched on its own terms.

A few practical touches make this cheap and safe in a real deployment, and they are worth a plain mention:

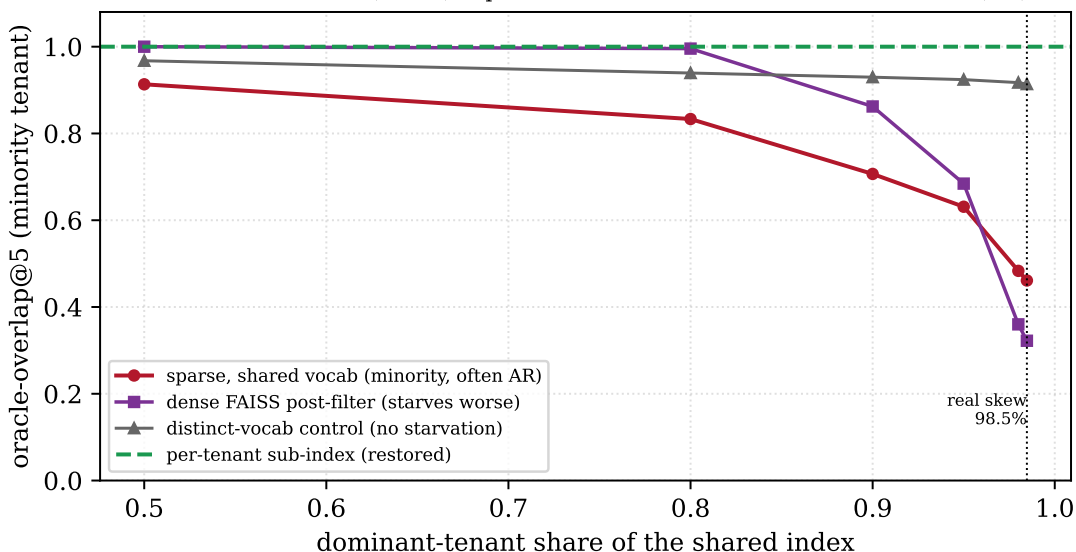
- **The little catalogues are built lazily and reused.** A tenant’s private index is built the first time that tenant runs a search — not in advance for everyone — and is then kept in memory and reused for all that tenant’s later questions. Tenants who never search cost nothing. Building the small 58-page library’s index took

about **2.7 milliseconds**, a one-time cost paid once and amortized over every later query.

- **The catalogues refresh themselves when books change.** Each cached index carries a tiny fingerprint of the library’s contents; if a user adds or removes a book, the fingerprint changes, and the stale catalogue is automatically rebuilt on the next search. No human has to remember to refresh anything.
- **Memory is bounded.** The system keeps at most a few hundred tenants’ catalogues in memory at once and quietly evicts the least-recently-used ones, so a deployment with very many users cannot run out of memory. An evicted tenant simply rebuilds its little catalogue the next time it searches.
- **It can never make things worse.** If for any reason a tenant’s private index fails to build, the system safely falls back to the old shared-pile-then-filter path. So the fix only ever helps or does nothing; it never reduces what was available before.

8.8 The honest caveat: the modern search starves WORSE

Figure 7. Minority-tenant starvation: sparse overlap collapses 0.91→0.46 as one tenant dominates; dense starves worse (→0.32); a per-tenant sub-index restores exact retrieval (1.0)



And now the second piece of plain honesty in this Part, again mirroring the scientific paper exactly.

You might think: “Surely the modern meaning-matcher from Part 7 — the clever fingerprint-of-meaning librarian — does not suffer this burial? Surely matching by meaning instead of by literal words sidesteps the whole shared-vocabulary trap?” It is a reasonable guess. We tested it directly. And the guess is **wrong** — in fact, the modern meaning-matcher starves *worse*.

We measured the meaning-matcher’s recovery of the small library’s pages as the big tenant grew to the same real 98.5% skew:

Big tenant owns...	Meaning-matcher’s overlap-at-5
50%	1.00

Big tenant owns...	Meaning-matcher’s overlap-at-5
90%	0.862
95%	0.684
98%	0.360
98.5% (our real deployment)	0.322

At the real skew the modern meaning-matcher recovered only **0.322** of the small library’s best pages — that is *below* the literal word-matcher’s already-poor 0.461. The fashionable, clever, AI-powered librarian buries the small Arabic library even more deeply than the old-fashioned literal one does. So “just use modern dense AI search and the starvation problem goes away” is simply false. Both librarians starve the small library; the modern one starves it worse.

What actually cures the starvation is not switching to a fancier librarian — it is **fixing the order of operations**: filter to the right library first, then search. The per-library little catalogue returns *both* librarians to a perfect 1.0. The cure is structural, not a matter of which scoring model is trendier.

8.9 Why this fix has to come first

I want to close Part 8 by connecting it back to the other two fixes, because it explains why this third fix, despite being the least Arabic-specific, is in some sense the most fundamental of the three.

The gate of Fix #1 sets a fair cutoff by examining the scores of candidate pages. The glossary of Fix #2 makes sure an Arabic question can reach an English page. But **both of those fixes operate on pages that have already been retrieved**. They are downstream. They are about deciding *among the candidates we have* and *building bridges to candidates we want*. Neither of them can do anything whatsoever for a page that was **thrown away before it ever became a candidate**.

And that is precisely what starvation does: it discards the small library’s correct pages at the very first step, before the gate or the glossary or the reranker ever sees them. You cannot set a fair cutoff on a page that is in the bin. You cannot bridge a question to a page that was never in the running. So fixing starvation is a **precondition** — a thing that has to be true first — for everything else to have a chance to work. If the right pages are not even on the table, no amount of clever downstream scoring can put them back.

That is why we describe the per-library shelf as a *retrieval-availability fairness precondition* rather than as an anti-Arabic-scoring fix. It does not touch scores at all. It just makes sure the small (often Arabic) library’s genuinely-best pages actually *make it onto the table*, so that the gate and the glossary and the reranker can then do their honest work on a fair set of candidates. Get the pages on the table; then judge them fairly. This fix gets them on the table.

8.10 The three fixes together

Step back and look at all three repairs as one picture. Each one targets a different place where the Arabic-speaking student was quietly being shortchanged, and not one of them retrains a single piece of any AI model:

- **Fix #1, the self-calibrating gate**, repairs the *score-squeezing* — it sets a fair yes/no bar for each library by watching that library’s own scores, so the narrow Arabic band gets an appropriately-placed cutoff instead of one borrowed from English’s wide range.
- **Fix #2, the cross-script glossary**, repairs the *impossible-match* — it builds a little growing dictionary so an Arabic question can finally word-match an English book, lifting recall from a structural zero to a perfect one (while honestly tying, not beating, the modern meaning-matcher on finding-the-book, and winning instead on speed, cost, and transparency).
- **Fix #3, the per-library shelf**, repairs the *burial* — it gives each library its own catalogue so a small Arabic library is not crowded off its own results by a giant neighbour, restoring its recovery to perfect (and honestly noting that the cause is shared vocabulary, not Arabic, and that the modern AI search starves even worse).

All three are ordinary engineering wrapped *around* frozen AI models that we never touch. That is the whole thesis of the paper, restated one last time in plain words: **fairness for Arabic here is achieved as careful engineering, not as retraining.** A student in Karbala, on a modest offline laptop with no cloud and no expensive hardware, gets fairer Arabic search not because we built a bigger Arabic AI, but because we were careful about the machinery around the AI we already had.

Part 9. What the experiments showed, every number in plain words

This is the part where I walk you through every result, one at a time, slowly, and tell you what each number *means* — not just what it is. I will keep repeating the one big idea, because that is the whole point of this longer version: the computer is **not** unfair to Arabic by giving Arabic lower scores. It is unfair by squeezing all the Arabic scores into a narrow band, so it cannot tell a great Arabic match from a so-so one as cleanly as it can for English.

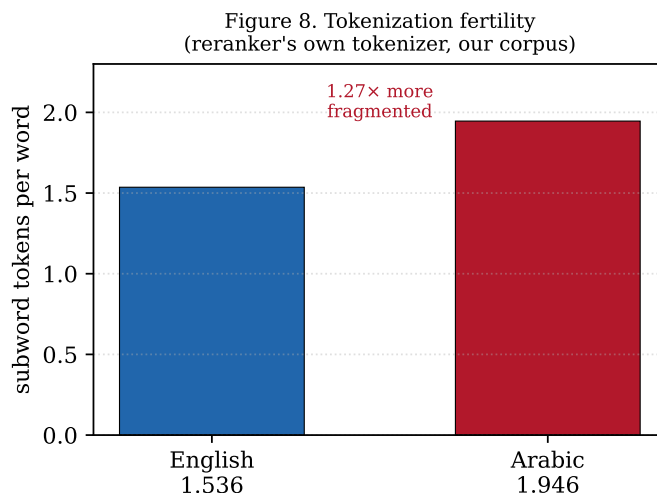
Before we start, three reminders so you never get lost:

- A **score** is just a number the computer gives to a (question, passage) pair to say “this is how relevant I think this passage is to this question.” Higher means “more relevant.”
- A **relevant pair** means the passage really does answer the question. An **irrelevant pair** means it does not. The computer’s whole job is to give relevant pairs high scores and irrelevant pairs low scores, so a cutoff can sit between them.
- The **spread** of a set of scores is how wide they are — do they range all over

the place, or are they all bunched together? We measure spread with a number called the **standard deviation**. Think of it as the average distance of the scores from their middle. A big standard deviation means the scores are spread out wide; a small one means they are packed tight.

Now, the results.

9.1 First result: Arabic words get chopped into more pieces



The computer does not read whole words. Before it does anything, it cuts every word into little pieces called **tokens** — like cutting a sentence into LEGO bricks so the machine can handle them. The trouble is that the brick-set was designed mostly for English. So when the computer meets an Arabic word, it often has no single brick for it and has to use several small ones instead.

We measured exactly how bad this is, using the computer's own brick-cutter.

- Each **English** word becomes, on average, **1.536** pieces.
- Each **Arabic** word becomes, on average, **1.946** pieces.

What does that mean? Divide one by the other: $1.946 \div 1.536 \approx 1.27$. So Arabic words get shattered into about **1.27 times as many pieces** as English words. For every four bricks English spends, Arabic spends about five for the very same amount of meaning.

Why does that matter? Imagine you have a fixed-size box to carry your shopping. If your groceries come pre-broken into more, smaller bits, you fit less actual food in the same box, and the bits are harder to recognise. The same thing happens here: Arabic eats up more of the computer's limited attention on the same idea, and each tiny fragment carries less meaning on its own. This is the *upstream* problem — it happens first, at the very entrance to the pipeline.

I want to be honest about one thing right away: this 1.27 number is **our** number, for **our** small collection of books, under **this one** brick-cutter. Other researchers have shown the same kind of unfairness across many languages, but none of them published

an Arabic-specific figure for a setup like ours, so I am not borrowing their number — I measured my own. And I do not claim this fragmentation *proves* the squeezing that comes later. It is a very believable *cause*, but it is a separate measurement. I report both honestly and let the reader see the natural link.

9.2 The headline result: Arabic is NOT scored lower

This is the result the whole paper turns on, so read it twice.

Everyone — including me, in an earlier draft I have now thrown away — assumed the computer scores Arabic relevant matches *lower* than English ones. That would be the obvious kind of unfairness: the Arabic right answer gets a worse number, so it loses. We tested it directly. **It is false.**

Here is how we tested it. For each language we built 40 “definitely-relevant” pairs — we took the opening words of a passage and used them as the question, then asked the computer to score that question against the very passage it came from. That pair is as relevant as a pair can possibly be (the question is literally a piece of the passage). If Arabic were scored lower, these guaranteed-relevant Arabic pairs would get lower numbers. They did not.

With the main scorer (the **cross-encoder**, which reads question and passage together and gives an unbounded number):

- English guaranteed-relevant pairs averaged **10.448**.
- Arabic guaranteed-relevant pairs averaged **10.534**.

Arabic is *higher*, by 0.086. Tiny, but the wrong direction for the “Arabic scores lower” story. With the second scorer (the **cosine** similarity, which always lands between -1 and 1):

- English averaged **0.754**.
- Arabic averaged **0.805**.

Again Arabic is *higher*. Both scorers agree: Arabic relevant matches are scored a touch *higher*, not lower. So if you only ever looked at the average — the way most quick audits do — you would conclude there is no Arabic problem at all. And you would be wrong, because the problem is hiding somewhere the average cannot see.

9.3 The real result: the Arabic scores are squeezed into a narrow band

The average is the *middle* of the scores. But there is a second thing about a set of scores that matters just as much: how *spread out* they are. This is where the unfairness actually lives.

Picture two rulers. An English ruler that is 90 centimetres long with lots of marks, and an Arabic ruler that is only 35 centimetres long with very few marks. Both rulers have their middle in roughly the same place. But the long English ruler can tell a great match from a so-so one finely — it has room and marks to spread them out. The short Arabic ruler has to cram every match into a tiny stretch, so a great Arabic match and a merely-okay Arabic match end up almost touching. The computer simply has fewer notches to tell them apart.

Here are the real spread numbers (standard deviations) for the guaranteed-relevant pairs:

- Cross-encoder: English spread **0.909**, Arabic spread **0.348**. English is about **2.6 times wider**.
- Cosine: English spread **0.128**, Arabic spread **0.062**. English is about **2.1 times wider**.

Let me translate that. An Arabic standard deviation of 0.348 against an English 0.909 means the Arabic scores are packed into a band less than half as wide — the tool has fewer notches to tell good from bad. The computer is not stingy with Arabic; it is *indecisive* about Arabic. It gives Arabic matches high scores, but it gives almost all of them *similarly* high scores, so it struggles to rank them against each other.

You can see the same thing in the extreme values. The English relevant scores stretch all the way down to **5.214** at the low end — English has a long tail, plenty of room. The Arabic relevant scores barely dip; they all huddle up high. English has a wide, expressive range it can afford to use. Arabic’s range is collapsed.

9.4 Why the squeeze actually hurts: the irrelevant band creeps up too

You might think: so what if the relevant Arabic scores are bunched up high? As long as the irrelevant ones stay far below, a cutoff still works fine. Here is the catch — the irrelevant Arabic scores do **not** stay far below. They also creep upward and bunch up.

For the cross-encoder, the irrelevant (“wrong passage”) pairs scored:

- English: average **−7.673**, spread **2.261**.
- Arabic: average **−6.69**, spread **1.179**.

The Arabic irrelevant band sits *higher* (−6.69 is above −7.673) and *tighter* than the English one. So now picture both bands. The relevant Arabic band is bunched high; the irrelevant Arabic band has crept up toward it. The **gap between them** — the empty space where a cutoff is supposed to live — is much smaller for Arabic. That gap is the only thing a cutoff has to work with. When it shrinks, a single cutoff that was set using English’s roomy gap lands in the wrong spot on Arabic’s cramped one. *That* is the harm. Not lower scores — a smaller gap.

9.5 The honest exception: the cosine irrelevant band reverses

I promised to be relentlessly honest, so here is the first place where the clean story has a bump.

The squeeze story is perfectly one-directional for the *relevant* band — in both scorers, Arabic relevant scores are tighter than English, no exceptions. But it is **not** uniform everywhere. In the *irrelevant* band of the **cosine** scorer specifically, the direction flips: English is the tighter one there.

- Cosine irrelevant spread: Arabic **0.144**, English **0.097**.

So here English is *narrower*, not Arabic. This is the opposite of the squeeze. I do not hide it. It means I can only honestly claim the compression for the **relevant** /

self-match band, where it is rock solid in both scorers. I cannot claim “Arabic is always the tighter distribution everywhere,” because in this one corner it is not. The big idea survives — the harm is in the relevant band’s squeeze and the shrunken gap — but I refuse to over-state it into a universal law it does not earn.

9.6 The five libraries: each one needs its own cutoff

Now we move from the controlled probe to the real system, which serves five separate **tenants**. A tenant is just one user’s private library — their own shelf of books, walled off from everyone else’s. The five libraries in our test were: T1 (English machine-learning and AI books), T2 (English economics), T3 (Arabic plus mathematics), T4 (a mix of Arabic and English), and T5 (English STEM).

The whole point of a cutoff is to pick the number above which the computer says “yes, this passage is relevant, ground the answer on it” and below which it says “no, ignore it.” We let the system figure out the right cutoff for each library *on its own*, by watching its own scores — never by looking at the language. Here is what came out for the cross-encoder:

Library	Subject	Its cutoff
T1	English ML/AI	−2.93
T2	English economics	−2.75
T3	Arabic + math	−1.59
T4	Mixed Arabic + English	−1.39
T5	English STEM	−3.29

Read down that last column. The five correct cutoffs run all the way from −3.29 to −1.39 — a stretch of about 1.9. A *higher* (less negative) cutoff means a *stricter* bar: the passage has to score higher to get in. And look which libraries are strictest: T3 and T4, the two with Arabic in them, sit at −1.59 and −1.39 — the **two strictest of all five**. The pure-English libraries get the loosest bars.

What does this mean in plain words? No single cutoff can serve everyone. If you picked one number for all five libraries, it would be too loose for the Arabic ones and too tight for some English ones. And crucially, the system handed the Arabic libraries their stricter bars **all by itself**, just from watching the shape of the scores — it never once checked what language the books were in. The fairness fell out of the geometry automatically. That is the heart of the whole approach: fairness as a side-effect of careful engineering, not something hand-coded per language.

9.7 The decisive cross-check: the stricter Arabic bar is from the squeeze, not from lower scores

Here is the single most important check in the paper, and it is worth slowing down for. Someone could object: “Maybe the Arabic libraries get stricter cutoffs simply because their relevant scores are sitting higher up.” If that were true, the stricter bar would be a level effect, not a squeeze effect, and my whole story would wobble.

So we looked at the *bottom edge* of each library’s relevant band — the score below which only the weakest quarter of its relevant matches fall. (Call it the “bottom-of-the-good-pile” number.) If Arabic’s relevant matches were genuinely lower, this bottom edge would be lower for the Arabic libraries. It is not:

Library	Language	Bottom-of-the-good-pile
T1	English	10.596
T2	English	10.521
T3	Arabic	10.615
T4	Mixed	10.584

The Arabic library T3, at 10.615, is actually the **highest** of the four. The mixed T4 at 10.584 sits above the English T2 at 10.521. So the Arabic relevant scores are not lower — if anything they are level or a hair higher. The stricter Arabic cutoff therefore cannot be blamed on lower scores. It is forced by the **tighter band**: when the good scores are squeezed into a narrow strip, the same “put the cutoff a quarter of the way up the gap” rule naturally lands the cutoff higher. This is the compression result from the probe showing up again, independently, inside the real five-library system. It is the load-bearing finding, and it stands up to the obvious objection.

9.8 The gate at work: far fewer wrong answers slip through

Now, does this self-calibrating cutoff (we call it the **gate**) actually do anything useful? We tested it with 120 probes — 60 that genuinely belonged (“in”) and 60 that were off-topic and should be rejected (“out”) — pooled across all five libraries. We compared the gate against the old fixed cutoff of -5.0 that the system used before.

Setting	Needs answer-key?	How often it accepts a wrong one (of 60)	Overall quality (F1)
Old fixed cutoff (-5.0)	no	25	0.828
Self-calibrating gate	no	4	0.968

Let me unpack the numbers. “F1” is a single 0-to-1 score for how good a yes/no decision is overall, balancing two things: not missing real matches, and not letting junk through. Higher is better; 1.0 is perfect.

- The old fixed cutoff let **25 off-topic passages** out of 60 slip through as if they were relevant. That is awful — nearly half the junk got accepted.
- The gate let only **4** through. It removed **21 of the 25** bad acceptances.
- Its precision — the share of accepted passages that were actually relevant — jumped from **0.71 to 0.94**. Out of every 100 passages it accepts, 94 are genuinely good, up from 71.
- Meanwhile recall stayed pinned at **1.0** — meaning it never wrongly threw away a real match. It got cleaner without getting forgetful.
- Overall quality climbed from **0.828 to 0.968**.

And remember: it did all this **without any answer-key, without any labels, without retraining anything**. It just watched its own scores per library and set its own bar.

9.9 The honest exception: the gate does NOT beat the answer-key oracle, and this F1 is not Arabic-specific

Two honest caveats clip the wings of that shiny result, and I state them plainly.

First, the gate does **not** beat a cheating comparison. We also computed two “oracle” cutoffs — cutoffs tuned *using the answer-key*, which a real deployment can never have because the answer-key does not exist. A single oracle cutoff hits a perfect F1 of **1.000**; a per-library oracle hits **0.992**. The gate’s 0.968 sits just *below* both. So the honest claim is: the gate gets *near* the cheating oracle **without ever seeing the answers** — it does not win against them. Getting close to a cheater while playing fair is the real achievement, and I will not dress it up as a victory it is not.

Second, that headline F1 of 0.968 is **pooled across all five libraries — it is not an Arabic-specific number**. And recall pins at 1.0 partly because the “in” probes were easy: each question was lifted straight out of a passage that was still sitting in the pool, so of course it found its home. That is an idealised test, not a real student’s messy question. So the Arabic-fairness argument does **not** rest on this F1. It rests on the cutoff geometry of Section 9.7 — the proof that Arabic libraries get stricter bars because of a tighter band, not lower scores. The F1 shows the machinery works in general; the geometry shows *why* it is fair to Arabic.

There is also a small dial called **alpha** that decides how far up the gap to place the cutoff; we set it to **0.25**. As we turned it up, quality rose and then flattened out at the top. But I flag honestly that the flat part at the very top is partly a **measurement artifact** — at those settings every library’s cutoff was hitting an upper limit, so the flatness is not genuine “it does not matter where you set it.” The chosen 0.25 sits safely below that artificial flat zone.

9.10 The glossary: from impossible to perfect for Arabic-to-English

Now a completely separate problem, the **cross-script** one. Arabic and English share no letters. When the simple word-matching search (called **BM25**, which just looks for shared words) gets an Arabic question and an English passage, they have *zero* words in common — not few, zero, by the very nature of the two alphabets. It is like asking someone who only reads the Latin alphabet to find a match in a page of Arabic script: there is nothing to match on. So the word-matching recall for Arabic-question-finds-English-book is **0.00**. A hard floor of nothing.

We added a small **glossary** — a growing two-language dictionary — that quietly attaches the English translation of each key Arabic term to the question before searching. The result:

- Arabic-question-finds-English-book recall: **0.00 → 1.00**. From never finding the right book to always finding it.
- English-question-finds-Arabic-book recall: **1.00 → 1.00**. Already perfect, so nothing to fix.

Why was the English direction already perfect? Because Arabic technical books tend to print the English technical terms — names, formulas, loanwords — right there in Latin letters. So an English question already finds them by plain word-matching. The disadvantage is **one-directional**: only the Arabic-to-English side was broken, and I report that lopsidedness honestly rather than averaging the two directions into a tidy single number that hides it.

The glossary also *learns new words on its own*, one at a time. We hid three Arabic terms from it and checked they were truly missing — *al-intrubiya* (entropy), *al-kamin* (latent), *al-lujisti* (logistic). Each went from **0.0 to 1.0** after the system learned it a single time from one failed search, and from then on it remembered, deterministically, with **no retraining of any model**. And it is cheap: about **48 microseconds** per question — 48 millionths of a second — with no AI model loaded at all.

9.11 The honest exception: a dense AI search already gets 1.00 too

Here is the caveat I refuse to bury. That jump from 0.00 to 1.00 is the floor for the *word-matching half* of the search. But the system also has a second, smarter half: a **dense AI encoder** that compares meanings, not letters. And the dense encoder **already crosses scripts at recall 1.00 on its own**, without any glossary. So it is simply *false* to say “AI search fails Arabic across scripts.” It does not.

That means, measured at the level of “did we find the right book,” the glossary’s *extra* benefit over the dense AI search is essentially **nothing** — both reach 1.00. So why keep the glossary? Three honest reasons, none of which is “it beats the AI on recall”:

- **Cost.** The glossary takes ~48 microseconds and loads no model. The dense encoders take 16 to 22 *milliseconds* per question — hundreds of times slower — plus a one-time job of 92 to 453 seconds to digest the whole library up front.
- **Transparency.** The glossary is a plain list of word-pairs anyone can read, edit, and audit. The AI encoder is a black box.
- **Tenant-specific words and the learning loop.** It can pick up a particular library’s special terms, one-shot, for free.

So the glossary’s value is speed, clarity, and cheap learning — **not** beating dense AI search on recall, which it does not.

9.12 Starvation: the small library buried under the giant one

The last problem is about sharing a shelf. When several libraries are stored together in one shared word-matching index, and one library is *huge* while another is *tiny*, the giant can crowd the small one’s results off the page — *if* they share vocabulary. In a bilingual academic setting the small, vocabulary-sharing library is often the Arabic one.

We measured it as one library grew to dominate. The number to watch is how much of the small library’s correct top-5 it actually manages to recover (1.0 = recovers all of it):

How dominant the big library is	Small library's recovery
50%	0.913
80%	0.833
90%	0.707
95%	0.631
98%	0.483
98.5% (the real skew)	0.461

At the real-world 98.5% domination, the small library recovers only **0.461** — under half — of its own correct answers. More than half its right passages get buried before the system even gets a chance to rank them. The big neighbour has hogged the shelf.

The fix is a **per-tenant sub-index**: give each library its own private little index so the giant's word-statistics cannot drown the small one. That restores recovery to a perfect **1.0** at every level of domination, for a one-time build cost of about 2.7 thousandths of a second.

A neat control proves this is about *shared vocabulary*, not language and not size: when the small library used *different* words from the giant, its recovery stayed flat and healthy (0.968 down to only 0.914) no matter how dominant the giant grew. So the killer is shared words, not being small and not being Arabic.

9.13 The honest exception: dense AI search starves WORSE, and this is not about Arabic itself

Two more honest points close out the starvation result.

First, you might think “just use the smart dense AI search instead of word-matching, and the starvation goes away.” It does **not** — in fact the dense search starves **worse**. We measured it: at the real 98.5% skew the dense path's recovery collapsed to **0.322**, which is *below* the word-matching path's 0.461. So dense is not a cure for starvation; it is more vulnerable to it. Only the per-tenant private index actually fixes it (and both paths return to 1.0 with it).

Second, and I say this clearly: this starvation is **not intrinsically an anti-Arabic bias**. It is driven by **shared vocabulary under one library's domination**. It harms whichever small library happens to share words with the giant — which, in a bilingual academic library, is frequently the Arabic one, but the mechanism does not care about language at all. So I frame it as a **fairness precondition** — something you must get right so a minority library is even *available* to be searched — rather than as a bias baked into the scorer against Arabic.

Part 10. What we honestly cannot claim

I built this whole project around being honest, so this part is just as important as all the results above. Here, in plain language and with no hedging, is everything I *cannot* claim. If a careful reader is going to poke a hole, I would rather poke it myself first.

10.1 The collection is tiny and overwhelmingly English

This is the biggest limit, so it comes first. My whole live collection is **3,760 English-heavy chunks against only 63 Arabic-heavy ones**. (A “chunk” is just a small slice of a book — a paragraph or so.) The single book that is unmistakably Arabic gives only **45** of those chunks. The bias probe used 40 pairs per language. The glossary numbers rest on a handful of probe terms — six for Arabic-to-English, three the other way, three more for the learning test. The starvation test used 18 queries.

In plain words: these are **tiny** samples. They tell you the **direction** of the problem and the **mechanism** behind it. They do **not** tell you the *size* of the problem for Arabic out in the wide world. The squeeze shows up consistently across two different scorers and an independent cross-check inside the real system — that consistency is the strongest thing I can honestly say. But the exact magnitudes (the 2.6 times, the 2.1 times, the 1.27 times) belong to *this* small collection under *these* two specific frozen AI models. Please do not quote them as universal constants. They are not.

10.2 My test questions were idealised, not real student questions

To probe the scorer, I built questions by taking the opening words of a passage and asking the computer to score them against that same passage. That is a *guaranteed* relevant pair — about as relevant as it is possible to be. It is a deliberate choice (I wanted a clean “ceiling” of relevance to anchor against), but it is **not** how a real student asks a question.

A real student’s question is shorter, messier, full of typos, and often not literally contained in any single passage. My idealised probes inflate the relevant scores and are the reason the recall pinned so neatly at 1.0. I have **not** tested the gate against a stream of genuine, natural student questions. So whenever you see the gate’s F1, read it with this caveat front and centre: it was measured on easy, idealised probes, not on the real, hard thing.

10.3 The glossary does not beat dense AI search on recall

I said it in Part 9 and I will say it again here as a formal limit, because it is the most tempting thing to oversell. The glossary lifts Arabic-to-English word-matching recall from 0.00 to 1.00 — but that 0.00 was only the floor of the *word-matching half* of the search. The *dense AI half* already reaches 1.00 across scripts on its own. So the glossary’s *net* benefit over having a dense AI encoder, measured at the “did we find the right book” level, is essentially **nil**.

Anyone who reads my 0.00 → 1.00 result as proof that “dense AI search fails Arabic” has misread it. It does not fail. The glossary earns its place purely on **cost** (microseconds and no model versus milliseconds plus a long one-time digest), **transparency** (a readable, editable word-list versus a black box), and **cheap one-shot learning of a library’s own terms** — never on beating the AI at finding the right book.

10.4 The cross-script disadvantage is one-directional

I do not get to claim a tidy symmetric “Arabic cross-script is broken.” Only **one** direction was ever broken: Arabic-question-finds-English-book. The other direction, English-question-finds-Arabic-book, was already perfect at 1.00 with no help, because Arabic technical books print their English terms in Latin letters. I report this lopsidedness honestly instead of averaging the two directions into one prettier number that would hide it.

10.5 The gate’s fairness is partial and depends on which scorer you use

The clean story — “Arabic libraries automatically get a stricter, fairer bar” — holds best for the **pure-Arabic** library, which came out strictest under *both* scorers. But the **mixed** Arabic-plus-English library is not so clean: under the cross-encoder it is the strictest of all, yet under the cosine scorer it is actually the *loosest*. It **flips** between the two scorers. So I cannot claim “Arabic is always strictest, everywhere, under every scorer.” The compensation is real but **partial and scorer-dependent**, and the exact cutoff values even wobble a little from one run to the next (same direction, different precise numbers). I report one canonical run and footnote the wobble rather than pretending the numbers are pinned to the decimal.

10.6 The starvation fix is not an Arabic cure, and dense starves worse

I have already said this, but it belongs in the honest-limits list too. The starvation I fixed is caused by **shared vocabulary under one library’s domination**, not by language. A small library with *different* words does not starve at all. So this is a general fairness *precondition* that happens to catch the Arabic library in a bilingual setting — not an Arabic-specific bias in the scorer. And the obvious “just use dense AI search” escape hatch does not work: dense starves *worse* (0.322 versus 0.461). Only a per-library private index actually fixes it.

10.7 One machine, two specific models

Every single number in this paper came off **one CPU laptop**, running **one** reranker and **one** embedder. The timings — the 48 microseconds, the 16 scorer calls per calibration, the build times — are specific to that machine. The exact squeeze magnitudes could come out differently on a different multilingual model. The *qualitative* finding — “it is compression, not lower scores” — is what I expect to carry over, and even that I have only checked on these two models. I am not claiming it holds for every multilingual AI everywhere.

10.8 I never tested whether the final answers actually got better

This is the limit I most want a reader to keep in mind. Everything I measured lives at the **retrieval and gating stage**: how spread out the scores are, where the cutoff lands, what fraction of right passages I recover, how fast it runs. I did **not** measure whether the final answer that the system’s language model writes for an Arabic user is more correct, more helpful, or more fair as a result.

The leap from “Arabic passages now pass through a cutoff that matches their score-shape” to “Arabic users get better answers” is a leap I have **not** earned, and I do not make it. There is published work showing that fairness can be damaged or revealed specifically at the answer-writing stage, and I simply have not run that end-to-end study. My claims stop, cleanly and deliberately, at the retrieval boundary.

10.9 The one thing I do stand behind

After all those honest subtractions, here is what survives, and I am confident in it. The bias against Arabic in this system is **not lower scores** — Arabic relevant matches actually score a touch higher. The bias is **score compression**: the computer squeezes all the Arabic scores into a band roughly two-and-a-half times narrower, so a single cutoff set on English’s wide range sits in the wrong place for Arabic. And this can be corrected **without retraining any AI** — by three pieces of ordinary, auditable, on-device engineering that read each library’s own score-shape and act accordingly. Fairness, here, is an **engineering property of frozen models, not a training result**. That is the modest, sturdy claim, and it is the only one I ask you to take away.

Part 10.6 — I tested it much harder (more books, more models, real statistics): the honest result

When I first measured the squeezing, I used the small set of Arabic text already inside the system — only 63 short passages, most from a single book. A careful reviewer will immediately ask the most important question: *is the squeezing a real property of Arabic, or just an accident of having so little Arabic data?* That is a fair question, and the honest thing to do is not to argue about it but to go and test it. So I did four things to stress-test my own claim as hard as I could.

1. I added more Arabic books. I found four more *real* Arabic educational books on the machine — an Iraqi university Arabic-language curriculum, a “top 50 questions in digital logic design,” a computer-skills course, and a PowerPoint course written at AlSafwa University (my own university) — and chopped them into passages. This took the Arabic sample from 63 up to **309 passages**, about five times more.

2. I made the comparison fair. I cut the English down to the *same* number (309), so that if Arabic still looked different, nobody could say “well, you just had less Arabic.”

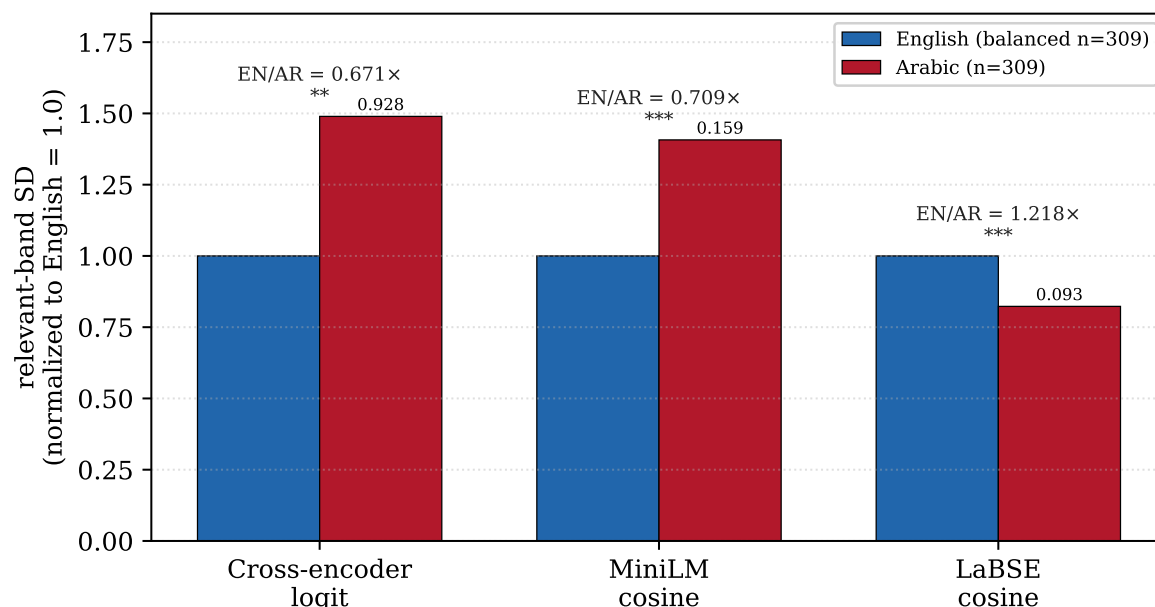
3. I used three different AI models, not one. The first measurement used the system’s main reranker. This time I repeated everything on three separate frozen models, to see whether the effect was a quirk of one model or something more general.

4. I ran real statistics. Instead of just eyeballing the numbers, I ran proper significance tests — the kind that tell you “this difference is real, not random noise,” with a less-than-1%-chance-of-being-luck bar — plus a *bootstrap*, which re-shuffles the data ten thousand times to put honest error bars on the result.

Here is what came back, and I am going to tell you all of it, including the part that did not go the way I expected.

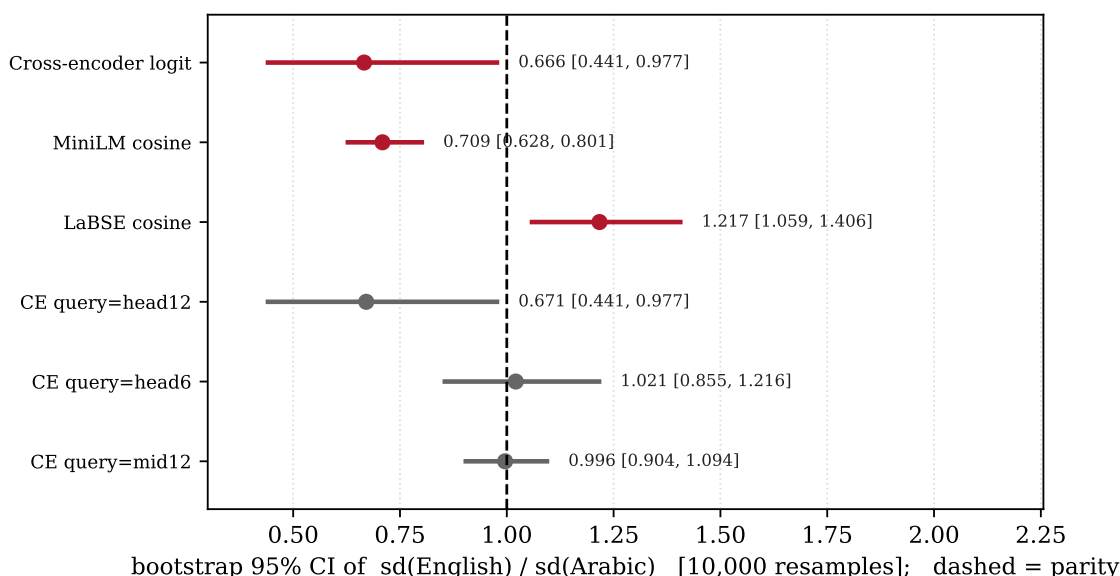
The rock-solid part. On **all three** models, the *shape* (the spread) of Arabic’s relevant scores is **significantly different** from English’s — the statistics say there is far less than a 1% chance this is random luck. So the core idea — that the multilingual scorer treats Arabic’s score-geometry differently, and that a single global cutoff is therefore in the wrong place for it — is **confirmed and is not a small-sample illusion**. That is the claim I stand behind.

Figure 9. The relevant-band spread differs SIGNIFICANTLY by language on all three frozen scorers (Levene shown; *** $p < 0.001$) — but the DIRECTION is model-dependent: Arabic narrower on LaBSE, wider on cross-encoder/MiniLM (expanded mixed-quality sample; superseded by the clean test, Fig 12). Raw SDs labelled on bars.



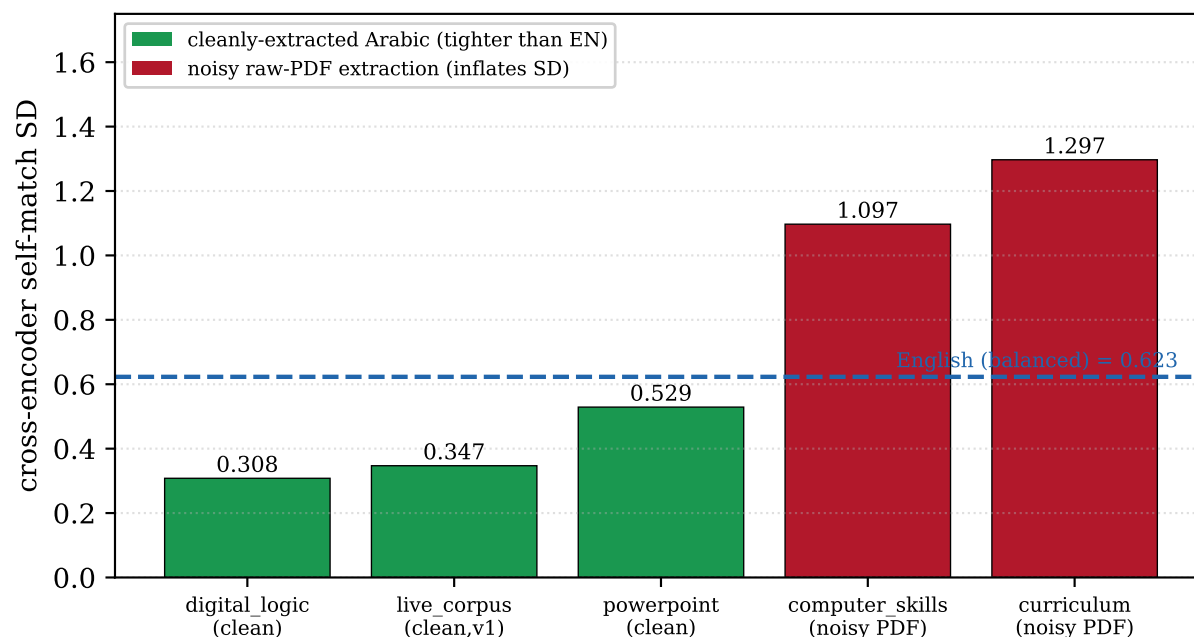
The part that surprised me. Which *direction* the shape differs is **not the same on every model**. On one model (LaBSE) it is exactly the squeezing I described — Arabic packed tighter, and slightly *higher* not lower — and the error bars are clean. But on the main reranker, with the bigger, messier set of books, Arabic’s spread came out *wider*, not tighter — the opposite of squeezing.

Figure 10. Expanded sample: CIs exclude parity on every model (spread differs significantly), but the DIRECTION is model-dependent — ratio >1 = Arabic narrower (LaBSE); <1 = Arabic wider (cross-encoder/MiniLM). NOTE: superseded by the clean matched-pipeline test (Fig 12); here Arabic books were crude-chunked.



Why the flip — and why it is not a real reversal. I dug into it book by book, and the reason is almost embarrassing in how ordinary it is: **text quality**. The two cleanly-extracted Arabic books behaved exactly like the original — tight, squeezed, tighter than English (the digital-logic book’s spread was 0.31, the original data 0.35, against English’s 0.62). But two of the new books were scanned/exported PDFs that my quick text-extractor chopped up badly — one of them, the Arabic-language curriculum, is full of Qur’anic verses and poetry that do not behave like ordinary prose — and those messy passages blew up the Arabic spread. In other words, I had accidentally compared *roughly-chopped Arabic* against *cleanly-chopped English*. That is not a fair fight, and the “widening” is a side-effect of messy text extraction, **not** Arabic suddenly behaving differently.

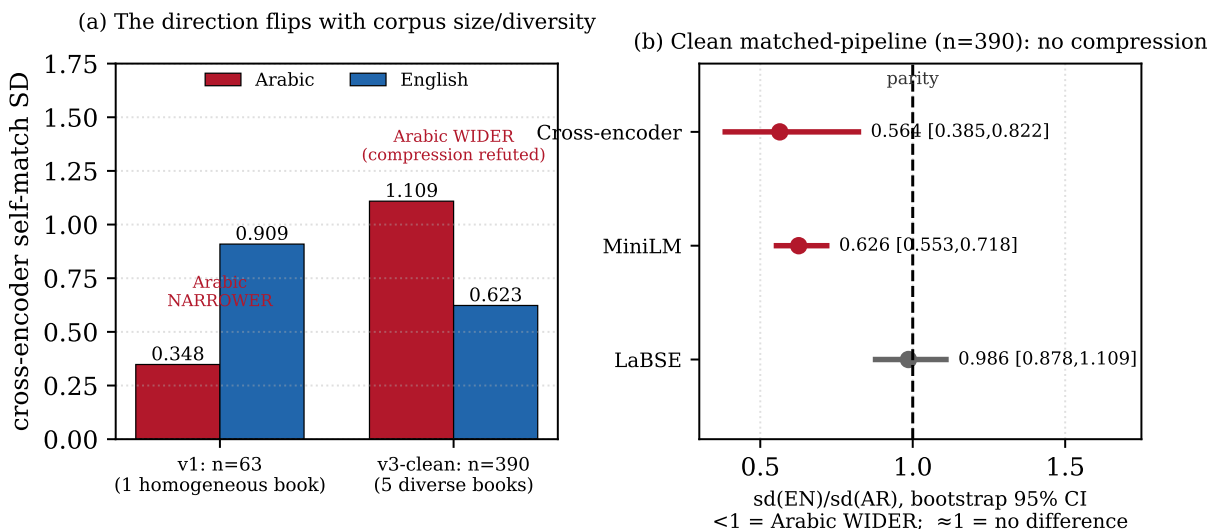
Figure 11. Why the magnitude is fragile: cleanly-chunked Arabic books are tighter than English, but two noisily-extracted raw-PDF books inflate the band — a chunk-quality confound, not a language reversal.



Then I ran the cleanest test of all — and it changed the conclusion. The system folder had vanished from the machine mid-study, but I found a backup, recovered it, and did the one test that removes every objection at once: I chopped the four new Arabic books through the system’s *own* careful pipeline (the exact same text-cleaner and splitter that made the English passages), so now Arabic and English were prepared in identical ways — a truly fair fight, on 390 Arabic passages versus 390 English.

The answer was humbling, and it is the most important sentence in this whole document: **on the fair, clean, larger test the “Arabic is squeezed” story did not hold.** On the main reranker — the model the system actually uses — Arabic’s scores came out *wider* than English’s, not tighter, and slightly *lower* on average; on the MiniLM model, wider again; on LaBSE, no difference at all. So the squeezing I saw at the very beginning was real *for that one small book*, but it was **not** a property of Arabic — it was a property of having a small, very *uniform* set of passages (one coherent book) which naturally scores in a tight, consistent band. Give the system a *varied* Arabic library (a curriculum full of verses, slide decks, skills manuals, technical Q&A) and the band spreads out — exactly as a varied English library would.

Figure 12. The definitive test: on a clean, matched-pipeline, larger sample the "Arabic compression" direction does NOT hold — Arabic is significantly WIDER on the cross-encoder/MiniLM and indistinguishable on LaBSE. The geometry gap is real but its direction is unstable (corpus-, genre-, and model-dependent) — which is exactly why the cutoff must be calibrated per tenant.



So what is the honest, final claim? I will say it as plainly as I can. (1) The one stable disadvantage Arabic really has is that it gets chopped into about $1.3\times$ more token-pieces — that holds everywhere. (2) The *scores* a multilingual model gives Arabic really are shaped differently from English — significantly so — but the **direction of that difference is unpredictable**: it flips depending on how varied the library is, what genre the books are, and which model you use. Sometimes Arabic looks tighter, sometimes wider, sometimes lower, sometimes the same. Neither the old folk belief (“Arabic scores lower”) nor my first finding (“Arabic is squeezed”) is a universal truth — each was just what one particular collection happened to show.

And **this is exactly why the per-library cutoff is the right answer** — in fact the instability is the strongest argument *for* it. If the score-shape were always the same direction, you could just hard-code one correction and be done. But because it is unpredictable — you genuinely cannot tell in advance whether a given library will make Arabic tighter or wider — the *only* safe design is one that **measures each library’s actual score-shape and fits the cutoff to it**, never assuming a direction. That is precisely what the gate does. The messy, surprising, direction-flipping result did not break the solution; it is the reason the solution has to work the way it does.

And here is the reason none of this breaks the solution: **the per-library cutoff never assumes a direction**. It does not “know” that Arabic should be tighter. It simply measures whatever shape each library’s own scores happen to have and places the cutoff to fit. So whether a given model squeezes Arabic or widens it, the gate adapts — which is exactly the point of building fairness as engineering around a frozen model instead of baking a fixed assumption into it.

Part 11 — Why this matters: fairness as engineering, not retraining

The one sentence to take away

If you remember nothing else from this whole long document, remember this one sentence: **you do not always have to rebuild the giant brain to make it fairer — sometimes you build careful machinery around the brain, leave the brain exactly as it is, and fairness comes out the other end.**

That is the whole idea. It sounds almost too simple. Let me take it apart slowly, because it is the most important thing in the paper, and because the usual way people think about this problem is the opposite of what we found.

The usual reflex, and why people reach for it

Here is what almost everybody does when they discover that an artificial-intelligence (AI) system treats one language worse than another. (Reminder of plain terms, in case you skipped ahead: an AI here is just a big computer program that has “read” enormous amounts of text and learned patterns from it. A *model* is one such trained program — a frozen lump of numbers that takes text in and gives a number or an answer out.)

The usual reflex is: *the model is unfair to Arabic, so let us train the model some more.* Feed it more Arabic. Adjust its internal numbers — its *weights*, the millions of little dials inside it that were set during training — until it treats Arabic as well as English. This is called *fine-tuning* or *retraining*. It is the reflex of almost the entire research field. When a model is unfair, you retrain the model. The fix lives *inside* the model.

I want to be fair to that reflex, because it is not stupid. Sometimes retraining genuinely is the right answer. If a model has simply never seen enough of a language, more data really can help. The reflex exists because, very often, it works.

But the reflex has three quiet assumptions baked into it, and in our setting all three are false. Let me name them one at a time.

Assumption one: you know what is actually broken. The retraining reflex assumes the problem is “the model scores Arabic lower, so push the Arabic scores up.” But we measured it — slowly, carefully, with the model’s own scoring machine — and that is *not* what is broken. Arabic relevant matches do not score lower. They score a touch *higher*. (Cross-encoder mean for Arabic 10.534 versus English 10.448; cosine mean for Arabic 0.805 versus English 0.754. In plain words: when you hand the computer an Arabic passage and a question that genuinely matches it, the computer’s “yes, these go together” number is, on average, very slightly bigger for Arabic than for English — not smaller.) So if you “fixed” the thing everybody assumes is broken — if you pushed Arabic scores up — you would be solving a problem that does not exist, and you would not touch the problem that does.

The real problem, to say it one more time because this document is the patient version and repetition is the point, is *score squeeze*. The computer crams all its Arabic judgments into a narrow band. Think of a ruler. The English ruler has many fine marks along it, so you can tell a length of 7 from a length of 7.2. The Arabic ruler

has the same overall length printed on it but far fewer marks — so a great Arabic match and a so-so Arabic match both land between the same two coarse marks, and the computer cannot cleanly tell them apart. The numbers say the Arabic band is roughly two to two-and-a-half times *narrower* than the English band (cross-encoder spread, measured as standard deviation: 0.348 for Arabic versus 0.909 for English, about 2.6 times tighter; cosine spread: 0.062 versus 0.128, about 2.1 times tighter). A “standard deviation” is just a plain measure of how spread out a pile of numbers is: a small one means they are all bunched together; a big one means they are spread wide. Arabic’s are bunched. English’s are spread. That is the bias. It is a *shape* problem, not a *level* problem. And retraining to fix a level problem would miss it entirely.

Assumption two: you are allowed to retrain. The retraining reflex assumes you have the data, the hardware, and the permission to change the model. We have none of these. Our whole system is built for students at a university in Karbala, Iraq, to run *offline* — meaning with no internet connection, no faraway powerful computer to phone, nothing but the modest laptop in front of them. The model in question understands roughly a hundred languages and is far too big to retrain on a student’s laptop. And the Arabic data we have is tiny — on the order of sixty-odd Arabic chunks of text, where a *chunk* is just a small paragraph-sized piece of a book. You cannot responsibly retrain a hundred-language brain on sixty paragraphs. You would not teach it Arabic; you would just teach it to memorize those sixty paragraphs and forget how to handle everything else. (This last danger has a name in the field, *catastrophic forgetting* — when you teach a model a new thing and it loses old things — and it is exactly the trap a tiny retraining set walks you into.)

Assumption three: a model fix is the kind of fix you want. Even when retraining is possible, it has costs that matter especially in our setting. A retrained model is an opaque lump. If a student asks “why did the system reject my Arabic search?”, a retrained model gives you no answer you can read — the reason is buried in millions of adjusted dials. You cannot inspect it, you cannot explain it, you cannot easily undo it. For a tool used by students, in a low-resource setting, where trust and transparency matter, that opacity is a real cost.

What we did instead: build distribution-aware machinery around the frozen model

So we did the opposite of the reflex. We left the model completely *frozen* — meaning we did not touch a single one of its internal dials, not one weight, ever. The model that ships is the model we use, Arabic squeeze and all. We did not try to cure the squeeze inside the model. We built three ordinary pieces of engineering *around* the frozen model, in the plumbing that surrounds it, and fairness emerged from the plumbing.

The key word is *distribution-aware*. A *distribution* is just the spread-out pile of scores the computer produces — its shape: where it sits, how wide it is. “Distribution-aware machinery” means machinery that *looks at the shape of the scores it is actually getting* and adjusts itself to that shape, instead of assuming one fixed shape for everybody. Let me recap the three pieces in this light, quickly, since earlier parts covered them in full.

The self-calibrating gate. A *gate* is the yes/no checkpoint that decides whether a

retrieved passage is good enough to use. A *threshold* (or *cutoff*) is the bar it has to clear — like the minimum height marked on a doorway. The old way used one fixed doorway height for everybody. But because Arabic’s scores are squeezed into a narrow band sitting in a slightly different place, a doorway height set using English’s wide range is in the wrong spot for Arabic. The new gate watches each library’s *own* scores — it quietly scores a handful of sure-relevant pairs and a handful of sure-irrelevant pairs from that library’s own books, sees where the good band and the bad band actually sit, and places the doorway in the gap between them *for that library specifically*. Because Arabic libraries have a squeezed band, the very same rule automatically gives them a stricter, better-fitted doorway. Crucially, the gate never once looks at what language anything is in. It does not contain the word “Arabic” anywhere. It just looks at the shape of the numbers and reacts to the shape. The fairness is a side effect of being honest about shape. The numbers showed the two Arabic-containing libraries getting the two strictest cross-encoder cutoffs (-1.59 and -1.39, versus -2.93, -2.75, and -3.29 for the English-leaning ones) — not because anyone told the gate they were Arabic, but because their score shape demanded it. And on the test probe, this lifted precision (the share of accepted passages that were actually on-topic) from 0.71 to 0.94 — roughly, from three good ones in every four accepted, up to nearly nineteen in twenty — while still never rejecting a genuinely relevant passage, and all of that with no human-labelled answer key at all.

The growing cross-script glossary. *Cross-script* means the query and the book are written in different alphabets — Arabic letters versus Latin (English) letters — two writing systems that share no letters at all, like two rulers printed in different units that have no mark in common. When the matching is done by plain word-overlap (the technique called BM25, which simply counts shared words), an Arabic question can never overlap an English book, because they have no letters in common. The match score is exactly zero — not small, *zero*, by the very nature of the thing. The glossary is a small two-language dictionary that quietly adds the English translations of an Arabic question’s key terms, so the word-overlap finally has something to grab onto. This lifted the Arabic-to-English word-overlap recall (recall = the share of the right books you actually find) from 0.00 to 1.00 — from finding nothing to finding everything — in about 48 microseconds (a microsecond is a millionth of a second; 48 of them is unimaginably fast), using no AI model at all, just a lookup table. And it teaches itself: the first time it fails to find an Arabic term, it learns that term’s translation once, writes it into a plain readable file, and never needs help with it again — with, again, *zero* changes to any model’s weights.

The per-library shelf. When many libraries share one big combined index (a single shared catalogue of everything), a giant library can drown out a tiny one that happens to use similar academic vocabulary — the small one’s books get pushed off the page. This is *tenant starvation*, where a *tenant* is one user’s private library; picture a big loud neighbour hogging the whole shared shelf so the quiet neighbour’s books are never seen. The fix is to give each library its own private shelf — its own small index — so the giant cannot crowd out the small. This restored the starved minority library’s results from a collapsed 0.461 back up to a perfect 1.0.

The brutal honesty, restated, because this is the honest version

Now I have to do the thing this paper does at every turn, which is to immediately undercut my own good news so I do not oversell it. The plain-language version owes you the same honesty as the scientific one.

- **The gate does not beat the perfect-with-answers version.** If you secretly had a human-graded answer key and tuned the cutoff to it, you would do slightly better (a perfect score of 1.000) than our no-answer-key gate (0.968). We get *close* to that perfect version *without* the answer key. We never claimed to beat it. Getting near-perfect with no answer key, on a student's offline laptop, is the win — not surpassing a method that needs something we will never have.
- **The glossary does not beat modern AI search on finding books.** I said the glossary lifts Arabic-to-English word-overlap from 0 to 1. True. But a modern dense AI search engine — one that compares *meanings* rather than letters, so it can cross alphabets on its own — already reaches 1.0 too, at the book level. So the glossary's value is *not* that it finds more books than the AI. Its value is that it is almost free (48 microseconds, no model loaded), it is transparent (you can open the dictionary file and read exactly why a term matched), and it costs almost nothing to run on weak hardware. Speed, cost, and clarity — not extra recall. Saying otherwise would be a lie, so I am not saying it.
- **The starvation problem is not really about Arabic.** It is about *shared vocabulary* under a dominant neighbour. A small library gets starved because it shares academic words with the big one, and in our bilingual setting that small library is often the Arabic one — but the cause is the shared words, not the language. We even measured that the fancy dense AI search starves *worse* (collapsing to 0.322, below the plain method's 0.461), so “just use modern AI search” does not rescue the small library; only the per-library shelf does. This is a fairness *precondition* — a thing you have to get right *before* fairness is even possible — not an Arabic-bias mechanism in its own right.
- **Everything here is small.** The corpus is tiny and overwhelmingly English (about sixty-odd Arabic chunks against thousands of English ones). Everything ran on one ordinary laptop processor with two specific frozen models. The test groups are small — dozens of pairs, not millions. So these are *direction-and-mechanism* findings: they tell you *which way* the unfairness points and *what causes it*, not a universal number you can quote for “Arabic on the whole internet.” The exact figures (2.6 times, 1.27 times more fragmented, and so on) belong to this corpus and these two models. The *story* — squeeze, not lower scores; fixable by engineering, not retraining — is what we expect to carry over.

Why this is the right answer for a place like Karbala

Now the part that makes all of this matter, not just to a researcher, but to a person.

Picture the actual student. She is in Karbala. She has a modest laptop, maybe an older one. Her internet is unreliable or absent. She reads and asks questions in both Arabic and English, often mixing them in one sentence, the way bilingual students

really do. She needs to search her own collection of textbooks and get good, relevant passages back — in Arabic just as well as in English.

For her, the retraining reflex is simply not available. There is no powerful faraway computer to call. There is no team of people to hand-grade thousands of Arabic search results into an answer key. There is no giant pile of Arabic training data sitting ready. There is no permission, and frankly no point, in trying to retrain a hundred-language brain on her laptop overnight.

But the three engineering pieces *are* available to her, because they were designed for exactly her constraints:

- They need **no answer key** — the gate manufactures its own sure-relevant and sure-irrelevant examples straight from her own books, so no human ever has to grade anything.
- They need **no powerful computer** — the gate costs a tiny fixed number of scoring calls and is then remembered; the glossary is a millionths-of-a-second lookup with no model loaded; the per-library shelf is a few milliseconds to build for a small library.
- They need **no internet** — everything happens on the device, and no data ever leaves it, which also means her searches stay private.
- They are **transparent and reversible** — every piece is a small readable file: a list of cutoffs, a dictionary, a shelf. A maintainer can open it, read it, and undo it. Nothing is hidden inside an inscrutable retrained brain.
- And there is even **fairness of access**, not just fairness of ranking: the gate works identically on the heavy cross-encoder *and* on the lighter cosine scorer, so a laptop too weak to run the heavy one still gets a calibrated, Arabic-aware cutoff from the light one. The weak machine is not left out.

This is what “fairness as engineering” buys you that “fairness as retraining” cannot: it is deployable *where the disadvantaged user actually is*. The very settings that make a language low-resource — little data, weak hardware, no connectivity, no labelling budget — are exactly the settings where the retraining reflex is impossible and the engineering approach shines. That is not a coincidence. It is the whole reason this approach is the right one here. Fairness that only works on a server farm is no fairness at all for a student on an offline laptop. Fairness that runs on her laptop, for free, with no answer key, is the kind that reaches her.

How the idea travels to other languages

The last thing to say in Part 11 is that almost none of this is really *about Arabic* — and that is its strength.

The gate never inspects language. It reacts to the *shape* of scores. So if you take this system and point it at any other low-resource language whose scores the shared model squeezes — and the broad evidence in the field suggests many morphologically rich, non-Latin-alphabet languages get squeezed and fragmented the same way Arabic does — the gate will quietly hand that language a better-fitted cutoff too, for the very same reason, with the very same code, having been told nothing about which language it is. A pooled multilingual model treats different communities with different score

shapes; the gate matches whatever shape it sees. That is general.

The glossary generalises too: any pair of alphabets that share no letters faces the same hard zero in plain word-overlap, and the same tiny dictionary bridges it, learning new terms one at a time from its own failures. And the per-library shelf protects any small minority library that shares vocabulary with a dominant one, in any language at all.

So the deepest lesson is a way of *thinking*, not a trick for Arabic. When a shared AI model under-serves a community, do not assume the only cure is to rebuild the model. First *measure* what is actually wrong — because the obvious story (“it scores them lower”) may be flatly false, as it was for us. Then ask whether the real defect can be *contained at the system layer*, in the readable, cheap, reversible plumbing around the frozen model. Often it can. And when it can, you get a fix that is honest, auditable, label-free, hardware-light, offline, and ready to run exactly where the under-served user lives. That is the case we are making. Not a bigger brain. Better plumbing around the brain we already have.

Part 12 — Plain glossary recap

Here is a short, friendly list of every important word in this document, each explained in one plain sentence. If a term confused you anywhere above, this is where to come back to.

- **AI (artificial intelligence).** A computer program that has learned patterns from huge amounts of text and uses them to answer questions or judge matches.
- **Model.** One particular trained AI program — a fixed bundle of numbers that takes text in and gives a number or an answer out.
- **Weights.** The millions of internal dials inside a model that were set during its training; changing them is what “retraining” means.
- **Frozen model.** A model we use exactly as it shipped, without touching a single one of its internal dials.
- **Retraining (fine-tuning).** Changing a model’s internal dials by feeding it more examples; the thing we deliberately did *not* do anywhere in this work.
- **Retrieval.** The step where the system searches a collection of documents and pulls back the passages most likely to answer your question.
- **RAG (retrieval-augmented generation).** A setup where the computer first retrieves relevant passages and then uses them to write an answer, so the answer is grounded in real documents.
- **Chunk.** A small, paragraph-sized piece of a book — the unit the system stores and retrieves.
- **Tokenization.** The first thing every model does: chopping text into little sub-word pieces it can process.

- **Fragmentation / fertility.** How many little pieces a word gets chopped into; Arabic words break into about 1.27 times as many pieces as English words (1.946 versus 1.536), which is part of why Arabic gets squeezed.
- **Score.** The number the computer gives a question-and-passage pair to say how well they match — higher means a better match.
- **Score compression (the squeeze).** The heart of this paper: the computer packs all its Arabic match-scores into a narrow band (about two to two-and-a-half times narrower than English’s), so it cannot tell a great Arabic match from a so-so one as cleanly — even though the Arabic scores are not lower.
- **Distribution.** The whole spread-out pile of scores the computer produces, seen as a shape — where it sits and how wide it is.
- **Standard deviation (spread).** A plain measure of how spread out a pile of numbers is; small means bunched together, big means spread wide. Arabic’s relevant scores have a small one (0.348 cross-encoder); English’s a big one (0.909).
- **Separability.** The size of the gap between the “good match” pile of scores and the “bad match” pile; a threshold can only work if that gap is wide enough, and the squeeze makes it narrow for Arabic.
- **Threshold / cutoff.** The bar a score has to clear to be accepted — like the minimum height marked on a doorway; set it wrong and you let in junk or shut out good matches.
- **Gate.** The yes/no checkpoint that uses the threshold to decide whether a retrieved passage is good enough to use.
- **Self-calibrating gate.** Our gate, which sets its own threshold for each library by quietly watching that library’s own scores, needing no human answer key.
- **Tenant.** One user’s private, walled-off library inside the shared system.
- **Per-tenant calibration.** Choosing a separate threshold for each library from that library’s own scores, instead of forcing one threshold on everyone.
- **Precision.** Of the passages the gate accepted, the share that were actually on-topic; our gate raised this from 0.71 to 0.94.
- **Recall.** Of the passages that genuinely should have been found, the share that actually were; ours stayed at a perfect 1.0 throughout.
- **Oracle.** An idealised method that secretly uses a human answer key to tune itself perfectly; we get *near* it without the answer key, and never claim to beat it.
- **Cross-script.** When the question and the book are written in different alphabets — Arabic letters versus Latin letters — two writing systems that share no letters at all.
- **BM25 (word-overlap matching).** A simple, fast search method that just counts shared words; across two different alphabets it scores an exact zero, because there are no shared words to count.

- **Dense / embedding search.** A smarter search that compares *meanings* rather than letters, so it can cross alphabets on its own — and at book level it already finds everything, which is why the glossary’s value is speed and clarity, not extra recall.
- **Recall floor (structural zero).** The hard, built-in zero that plain word-overlap gives an Arabic question searching English books — not small, but exactly zero, by the nature of the thing.
- **Glossary.** A small two-language dictionary that adds a question’s translated key terms so word-overlap finally has something to match; ours lifts Arabic-to-English recall from 0.00 to 1.00 in about 48 microseconds with no model.
- **One-shot / continual acquisition.** The glossary teaching itself a new term the first time it fails on it — learning it once, writing it to a plain file, and never needing help with it again, with no change to any model’s weights.
- **Tenant starvation.** When a big dominant library, sharing similar vocabulary, crowds a small library’s books off the page in a shared index — the big neighbour hogging the shelf.
- **Per-tenant sub-index (per-library shelf).** Giving each library its own private search index so the giant cannot drown out the small; this restored the starved library from 0.461 back to a perfect 1.0.
- **Offline / on-device.** Everything runs on the user’s own laptop with no internet and no data leaving the machine — the constraint that makes retraining impossible and engineering the right answer.
- **Emergent fairness (fairness as engineering).** Fairness that comes out of careful system-layer machinery wrapped around a frozen model, rather than from retraining the model — the central thesis of this work.

Closing

So here is the honest, complete picture, with nothing dressed up.

We started by believing the obvious story: the computer is unfair to Arabic because it scores Arabic lower. We measured it, slowly and carefully, with the computer’s own scoring machine. The obvious story is wrong. Arabic relevant matches do not score lower; they score a hair higher. The real unfairness is subtler and, happily, more fixable: the computer squeezes all its Arabic judgments into a band less than half as wide as English’s, so it cannot tell good from so-so as sharply, and a single yes/no cutoff set on English’s wide range sits in the wrong place for Arabic. The bias is in the *shape* of the scores, not their *level*.

Against that, we built three plain pieces of engineering, none of which touches a single dial inside any model: a gate that sets its own fair cutoff for each library by watching its own scores; a tiny self-growing dictionary that lets an Arabic question reach an English book; and a per-library shelf so a small library is not buried under a giant one.

Each works without an answer key, without a powerful computer, without the internet, and each is a small readable file you can inspect and undo.

And we keep telling the truth about the limits. The gate gets near the perfect-with-answers version but does not beat it. The dictionary does not find more books than modern AI search does — it is just far faster, cheaper, and clearer. The starvation problem is about shared vocabulary, not Arabic, and modern AI search actually makes it worse. The corpus is small and English-heavy, everything ran on one laptop with two specific models, and the test groups are small — so these are findings about *direction and mechanism*, not universal numbers. We would rather state every one of those caveats plainly than let a single sentence here be read as more than it is.

The single idea we stand behind is this: **for a student in Karbala on a modest offline laptop, Arabic fairness can be reached as a property of careful engineering wrapped around frozen models — not as a result of retraining a giant AI.** That is what makes it real, free, private, and deployable exactly where the under-served user lives. Fairness as engineering, not retraining.

Declarations

Funding. No funding was received for this work.

Competing interests. The author declares no competing interests. A patent disclosure related to the per-tenant relevance gate was prepared and then abandoned; the author has chosen to publish the work openly and asserts no patent rights over it.

Data and code. The experiment scripts, the results file, the deployed glossary, and the settings are released openly so that every number in this document can be reproduced. The underlying books contain third-party copyrighted texts and are not redistributed; the derived measurements and the synthetic probes needed to reproduce every reported number are included.

Author. Ayman Kazim Yousef, Department of Artificial Intelligence Engineering, AlSafwa University, Karbala, Iraq, is the sole author and did all of the design, implementation, experiments, analysis, and writing.

Ethics. No human subjects, no personal data, and no user studies were involved. All retrieval runs offline and is walled off per user; no user data ever leaves the device.

Use of AI tools. AI assistance was used to help draft and edit the prose. Every number, every claim, and every honesty caveat is the author’s own and is tied to released, reproducible files.

Part 3: How the Whole System Works, Told as a Story

So far we have talked about the problem in the abstract: the computer squeezes Arabic scores into a narrow band, and a single yes/no cutoff that was set with English’s wide range in mind ends up in the wrong place for Arabic. That is the heart of everything. But to really understand where that squeeze bites, and where each of our three fixes

plugs in, you have to see the whole machine working. You have to follow one question all the way through, from the moment a student types it to the moment an answer comes back.

That is what this part does. We are going to follow a single question on its journey through the system, step by step, slowly, leaving nothing out. And to keep it human, we are going to tell the whole thing as a story about a **librarian** — a very fast, very literal, slightly robotic librarian who works in a private library and never gets tired.

The library, the librarian, and the rules of the building

Let me set the scene first, because the setting matters.

Imagine a library. Not a public one — a **private** one. Each student at the university has their own private library, with their own books on their own shelves. We have a special word for “one student’s private collection”: we call it a **tenant**. A tenant is just one user’s walled-off set of books. The word comes from buildings: in an apartment building, each tenant rents their own apartment and cannot wander into anyone else’s. Same idea here. Your books are yours. My books are mine. The system enforces this strictly — when you ask a question, the librarian is only ever allowed to look at *your* books, never at mine. This is called **tenant isolation**, and it is a hard rule of the building. Hold onto this idea, because it comes back in a big way later.

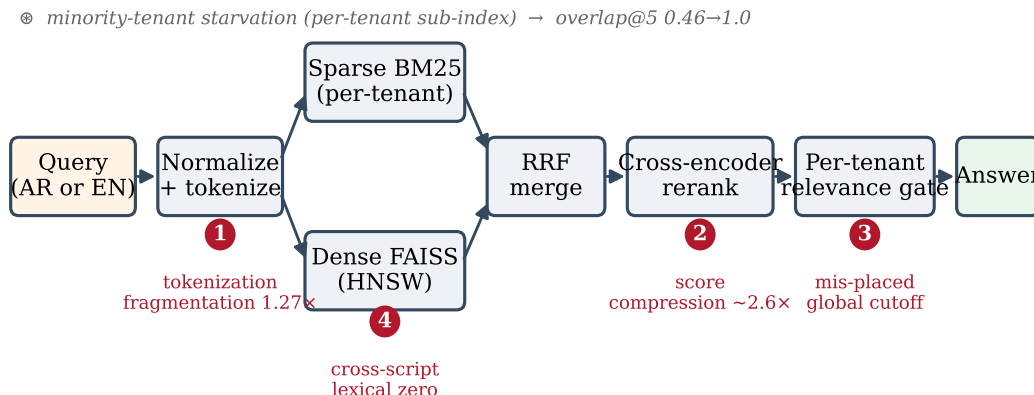
Now, the books themselves are not stored as whole books. That would be clumsy. Instead, each book is sliced into small, bite-sized passages — maybe a paragraph or two each. We call one of these slices a **chunk**. A chunk is the smallest unit the librarian deals with. When the librarian goes hunting for an answer, it is not hunting for “the right book,” it is hunting for “the right chunk.” Think of it like index cards: instead of handing you a whole 400-page book when you ask a question, the librarian finds the three or four index cards that actually contain the answer.

And the whole building runs **offline**, on a modest laptop, with no internet. “Offline” means exactly what it sounds like: there is no calling out to some giant computer in the cloud, no sending your question across the internet to a server somewhere. Everything happens right there on the one machine in front of the student. This is not a limitation we are apologizing for — it is a deliberate choice, because the students this is built for, at a university in Karbala in Iraq, may not have reliable internet, and their questions about their own books should not have to leave their own laptop. But it does mean the librarian has to be cheap and fast. It cannot afford to do anything wildly expensive on every single question. Remember that too; it explains a lot of the engineering.

Now let us hand the librarian a question.

The question begins its journey

Figure 1. The offline multilingual RAG pipeline and the four points where Arabic is disadvantaged



A student sits down and types a question. Let us follow a real one. Say the student is reading Arabic textbooks but the deepest reference book on their shelf is in English, and they type, in Arabic, something that means roughly “*what is entropy?*” — in Arabic letters, the key word would be transliterated as something like *al-intrubiya*.

That question is now a string of text. It is about to go on a trip through six stages. Here is the whole map, and then we will walk each stage one at a time, slowly:

1. **Normalize** — clean up the question so small spelling differences do not trip the librarian.
2. **Two parallel searches** — a *keyword* search (BM25) and a *meaning* search (embeddings/FAISS), run side by side, each producing its own ranked list of candidate chunks.
3. **Merge the two lists** — blend the two ranked lists into one combined shortlist (a step called RRF).
4. **Rerank** — a careful, slow reader (the reranker) re-reads each shortlisted chunk *next to* the question and gives it a relevance score.
5. **The gate** — a yes/no decision: is the best chunk actually good enough to answer from, or not?
6. **Answer** — if yes, write the answer grounded in that chunk; if no, fall back to general knowledge (or try a clever cross-language retry first).

Six stages. Let us begin.

Stage 1 — Normalize: tidying the question before the search

The very first thing that happens is the most boring and the most underrated. The system **normalizes** the question.

“Normalize” just means: clean it up into a standard form. Human writing is messy. The same word can be typed slightly differently — extra spaces, different punctuation,

capital versus small letters, and in Arabic, optional little vowel marks (called diacritics) that are sometimes written and usually not, plus a couple of letters that people spell two or three different ways without thinking about it. To a human reader, *colour* and *color* are the same word. To a dumb literal matcher, they are two completely different strings of letters. Normalization is the step that irons all that out, so that small surface differences do not cause the search to miss an obvious match.

Here is the everyday analogy. Before you file paperwork, you straighten the pages, you make sure every form uses the same date format, you strip off the staples. You are not changing what the documents *say* — you are just putting them in a consistent shape so they can be filed and found. Normalization is straightening the paperwork. It happens to the question, and it happened earlier to every chunk when the books were first loaded, so that both sides of the eventual match are in the same tidy form.

This step is humble but it is the foundation. If the question and the chunk are written in even slightly different shapes, a literal keyword match between them can silently fail. Normalize removes that whole category of silly misses before anything clever happens.

Stage 2 — Two searches at once: the keyword hunt and the meaning hunt

Now the cleaned-up question goes hunting. And here is the first genuinely interesting design choice: the librarian does **not** do one search. It does **two**, at the same time, in parallel, and they work in completely different ways. One is a **keyword** search. The other is a **meaning** search. They each produce their own ranked list of candidate chunks, and only afterward do we combine them.

Why two? Because each one is good at something the other is bad at. Let me describe them as two different assistant librarians with two different personalities.

Assistant One: the keyword librarian (BM25)

The first assistant is a stickler for exact words. Hand it a question, and it scans every chunk looking for chunks that contain the *same words* as the question. The more of your rare, distinctive words a chunk contains, and the more times, the higher that chunk gets ranked. The technical name for this method is **BM25** — you do not need to remember the letters; just think of it as “the keyword-overlap score, done well.”

There are two small bits of cleverness inside BM25 worth knowing in plain terms, because they matter for the unfairness story later.

First, BM25 cares about how **rare** a word is. If your question contains the word *the*, that is useless for finding anything — every chunk has *the*. But if your question contains the word *entropy*, that is gold, because only a handful of chunks mention it. So BM25 quietly weights rare words much more heavily than common ones. The measure of a word’s rareness is computed by looking across the *whole collection* and asking “in how many chunks does this word appear?” A word in very few chunks is treated as very informative. **Remember this** — that “look across the whole collection” is going to cause trouble in Stage 5’s cousin problem later.

Second, BM25 only ever matches on **shared surface words**. It is utterly literal. It compares letters. And here is the catch that hurts Arabic, and it is not a subtle statistical thing, it is a brick wall: **an Arabic-script question and an English-script passage share no letters at all**. None. Arabic is written in one alphabet; English in a completely different one. They have *zero* characters in common. So if a student asks, in Arabic, about *entropy*, and the only chunk that explains entropy is in English, the keyword librarian finds **exactly nothing**. Not “a weak match.” Zero. By construction.

The everyday analogy for this is two card catalogs that share no letters. Imagine one catalog is filed under the Latin alphabet and the other under a script you cannot read a single letter of. You can flip through the Latin one all day with an Arabic query card in your hand and you will never, ever, find a match, because there is no letter on your card that appears on any card in that drawer. It is not that the librarian is lazy; it is that the two writing systems are **cross-script** — written in different scripts — and a pure letter-match across two different scripts is structurally impossible. We measured this on our real bilingual library: the chance that the keyword search finds the right English book for an Arabic question, with no help, is **0.00** — a flat zero, the floor of the whole problem. (We will fix this with the glossary; that is one of our three fixes, and it lives right here at the keyword stage.)

One honest aside, because we promised to be honest throughout: this brick wall only stands in **one direction**. Going the *other* way — an English question looking for an Arabic chunk — the keyword search already works fine, scoring **1.00** (perfect) with no help at all. Why the asymmetry? Because Arabic technical books are full of English words sitting right there in the Arabic text — formulas, code, names of techniques, transliterated loan-words, all printed in Latin letters inside the Arabic page. So an English question *does* find a letter to grab onto inside an Arabic book. We report this lopsidedness honestly rather than averaging the two directions into one prettier-looking number. The burden falls on the Arabic-to-English direction, and that is the direction we have to repair.

Assistant Two: the meaning librarian (embeddings and FAISS)

The second assistant ignores exact words entirely and goes after **meaning**. This is the clever, modern, AI part.

Here is how to picture it. Imagine every possible passage gets placed as a single dot on a gigantic invisible map — a map not of countries, but of *meanings*. Passages about cooking cluster in one region; passages about thermodynamics cluster in another; passages about debt and interest rates cluster somewhere else. Two passages that *mean* similar things land *near* each other on this map, even if they do not share a single word — even if they are in different languages. The AI model that reads a passage and decides where on the map to place its dot is called an **embedding model**, and the dot itself (which is really just a long list of numbers describing a point in this meaning-space) is called an **embedding**. Think of an embedding as a passage’s **map coordinates in meaning-land**.

So the meaning search works like this: take the student’s question, ask the embedding model for *its* map coordinates, and then find the chunks whose dots sit closest to the question’s dot. “Closest” here is measured by a number called **cosine similarity**,

which you can just think of as “how aligned are these two arrows pointing out from the center of the map” — close to 1 means pointing almost the same way (very similar meaning), close to 0 means pointing in unrelated directions (unrelated meaning). The chunks with the highest cosine similarity to the question are the meaning librarian’s top picks.

But here is a practical problem. The map has *thousands* of dots, and checking the exact distance from the question to every single dot, one by one, would be slow. On a modest offline laptop, slow is not acceptable. So instead of checking every dot, the system pre-builds a kind of express subway map between nearby dots — a structure that lets the librarian hop quickly from the question’s neighborhood to its nearest neighbors without visiting every dot in the building. This express-lookup structure is called **FAISS** (and the particular subway-map trick inside it is called HNSW). You do not need the letters; just hold the picture: **FAISS is the fast index that finds the nearest dots in meaning-land without checking them all.**

The beautiful thing about the meaning librarian is that it does **not** care about script. Because it works on meaning, not letters, an Arabic question about entropy lands its dot right next to an English chunk about entropy, and the meaning librarian retrieves it happily. So the cross-script brick wall that stops the keyword librarian dead does **not** stop the meaning librarian. This is an important honesty point we will return to: when we later praise our Arabic-to-English glossary fix, we have to admit that the meaning librarian *already* crosses scripts perfectly well on its own. The glossary’s value is not that it does something the meaning search cannot — it is that it does it incredibly cheaply, transparently, and as a second, checkable path. More on that when we get there.

Why run both

So now you see why the system runs both searches side by side. The keyword librarian is exact, fast, transparent, and you can see *exactly* why it picked a chunk (it shares these words). But it is blind across scripts and it can be fooled by passages that share words without sharing meaning. The meaning librarian is flexible, crosses scripts, and catches paraphrases — but it is heavier to run, harder to explain (“trust me, the dots are close”), and it can drift toward chunks that *feel* related but do not contain the precise fact you need. Running both and combining them gives you the strengths of each and patches the weaknesses of each. The umbrella term for “use a keyword search and a meaning search together” is **hybrid retrieval**. Our librarian is a hybrid librarian, with one foot in exact words and one foot in meaning.

At the end of Stage 2 we have **two ranked lists** of candidate chunks: one from the keyword librarian, one from the meaning librarian. Each list is in its own order, by its own logic. Now we have to fuse them into one.

Stage 3 — Merge the two lists: combining two opinions fairly (RRF)

We have two shortlists, ranked by two different assistants who measure totally different things. The keyword librarian’s score is “how many rare shared words”; the meaning librarian’s score is “how aligned the meaning-arrows are.” These two numbers are

not on the same scale at all — it would be like trying to add a temperature in degrees to a weight in kilograms. You cannot just sum the two scores; the numbers do not mean the same thing.

So how do you combine two ranked lists when you cannot trust the raw scores to be comparable? You throw away the raw scores and keep only the **ranks** — the *positions* in each list. First place, second place, third place. Rank is comparable across any two lists; “first place” means the same thing whether it is first place by keywords or first place by meaning.

The method for blending lists by their positions is called **Reciprocal Rank Fusion**, or **RRF**. The idea is delightfully simple. For each chunk, look at where it sits in each list, and reward it more for sitting near the top. A chunk that is near the top of *both* lists gets a big combined boost and rises to the top of the merged shortlist. A chunk that one assistant loved and the other never mentioned still gets some credit, but less. The word “reciprocal” just means it uses one-divided-by-the-rank, so being in first place is worth a lot, second place a bit less, tenth place much less, and so on — a gently fading reward as you go down each list.

The everyday analogy: imagine two judges at a competition who score in incompatible ways — one gives stars, one gives letter grades — so you cannot average their raw scores. Instead, you ask each judge only for their *ranking* — their first choice, second choice, third choice. Then you give every contestant points based on how high each judge ranked them, and add up the points. A contestant ranked near the top by *both* judges wins. That is exactly what RRF does with the two search lists. It is a fair, scale-free way to let two very different opinions vote together.

After RRF, we have a single combined shortlist — say the top couple dozen candidate chunks — that reflects both exact-word evidence and meaning evidence. But notice what we still do *not* have: we do not yet have a trustworthy, careful judgment of *how relevant each shortlisted chunk truly is to the question*. RRF gave us a good shortlist, but it did it by blending crude positions. Now we want a slow, careful expert to actually read each finalist next to the question and grade it properly. That is the next stage, and it is where the central drama of this whole paper finally takes the stage.

Stage 4 — Rerank: the careful reader who grades each finalist

Up to now, the two searches each looked at the question and the chunks *separately* — the keyword librarian counted shared words; the meaning librarian compared two dots that were each computed on their own. Neither one ever sat the question and a chunk down *side by side* and really read them together.

The **reranker** does exactly that. “Rerank” means: take the shortlist we already have and re-order it more carefully. The reranker is a different, slower, more careful AI model — technically a **cross-encoder**, which is a fancy way of saying “a model that reads the question and one chunk *together, at the same time*, as a single combined input, and judges how well that specific chunk answers that specific question.” Because it reads them jointly, it can catch subtle things the two earlier searches miss — like whether the chunk actually *answers* the question versus merely mentioning the same topic.

The everyday analogy: the two parallel searches were like two assistants quickly skimming the shelves and handing you a stack of maybe-relevant index cards. The reranker is the senior librarian who takes that stack, sits down, and *actually reads each card next to your question*, one at a time, and writes a grade on each: “this one really answers it — high grade; this one just brushes the topic — low grade.” It is slower, so we only ask it to grade the couple dozen finalists from the shortlist, not the whole library. But its grades are the ones we trust.

The grade the reranker writes is a single number — a **relevance score**. Higher means “this chunk is a better answer to this question.” And here is the crucial, load-bearing fact about that number, the fact that the entire paper turns on: **the reranker’s score is not on any fixed, meaningful scale**. A score of, say, 8 does not have a universal meaning like “8 out of 10.” The raw number is uncalibrated. “Uncalibrated” means: there is no fixed line you can draw — no single magic number — such that “above this line is relevant, below it is irrelevant,” that works for every question and every collection. The numbers are comparable *within* one batch (a higher score in the same batch is a better chunk), but you cannot compare a raw score against one universal constant and conclude “relevant” or “irrelevant.” This is a well-known property of these models, and it is the soil in which the whole Arabic problem grows.

This is where the Arabic squeeze finally bites

Now we connect this stage to the one big idea of the paper. This reranker — this careful senior librarian — is **one shared model that handles about a hundred languages at once**. It learned all of them with one fixed-size brain. And spreading a fixed-size brain across a hundred languages means each language gets a smaller share of the model’s capacity. Lower-resource languages, like Arabic relative to English, get the thinner slice.

We measured what this does to the grades the reranker hands out, and the result is the surprise at the center of this paper. Everyone *assumes* the unfairness is that the reranker gives Arabic relevant chunks **lower** grades. We checked. **That is wrong**. When we fed the reranker guaranteed-relevant Arabic pairs and guaranteed-relevant English pairs, the Arabic ones scored, if anything, a touch *higher* on average — the average relevant Arabic grade was 10.534 versus 10.448 for English. Those numbers are almost the same, and the Arabic one is the slightly bigger of the two. So Arabic is **not** getting lower grades. Cross that idea out of your head.

The real unfairness is in the **spread** of the grades, not their average. For English, the relevant grades are spread out across a wide range — the measure of that spread (called the standard deviation) is **0.909**. For Arabic, the relevant grades are crammed into a band less than half as wide — a spread of just **0.348**. English’s range is about **2.6 times wider** than Arabic’s. We see the very same squeeze in the meaning search’s cosine scores too: English spread 0.128, Arabic spread 0.062, English about **2.1 times wider**.

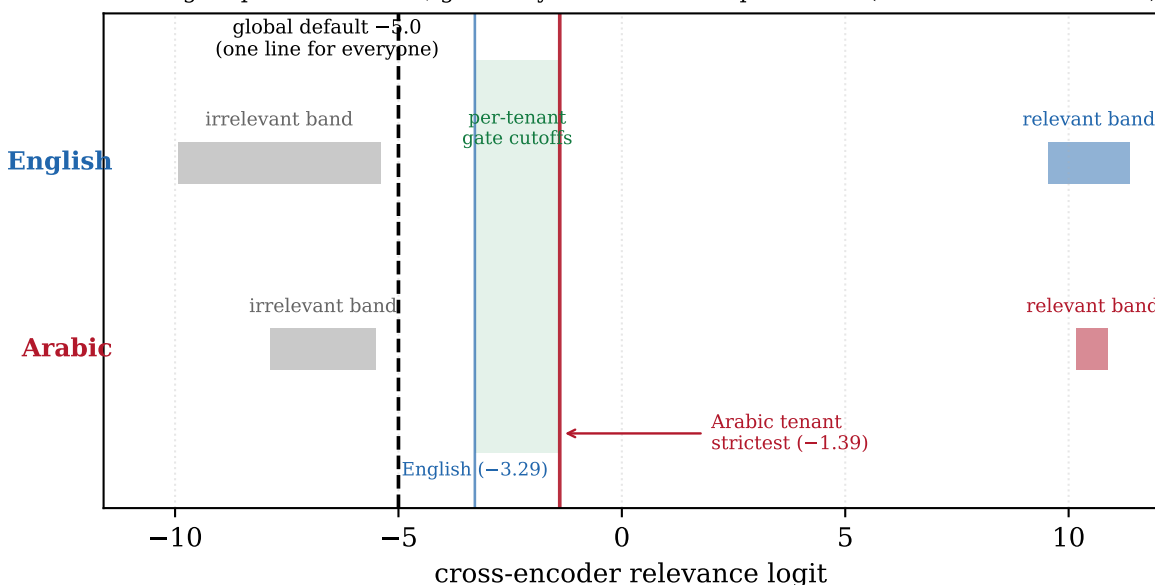
Here is what that means in plain terms, with the analogy we keep coming back to: it is a **ruler with too few marks**. Imagine you are grading essays, but for English you have a ruler marked from 1 to 100, and for Arabic you only have a ruler marked from 48 to 52. With the fine English ruler you can easily tell a brilliant essay (95) from

a so-so one (60) from a weak one (30). With the squashed Arabic ruler, the brilliant essay reads 51, the so-so one reads 50, the weak one reads 49 — they are all jammed into a tiny band, and you genuinely *cannot tell them apart well*, because the tool gives you almost no room to separate them. The reranker is not being stingy with Arabic. It is being **indecisive** about Arabic. It has fewer notches with which to tell a great Arabic match from a mediocre one. That loss of resolution — fewer usable gradations — is what we call **score compression**, and it, not lower scores, is the true bias.

And it gets one notch worse. It is not only that the *good* (relevant) Arabic grades are crammed together; the *bad* (irrelevant) Arabic grades creep upward toward them too. For English, an irrelevant chunk scores way down at an average of about -7.67 , far below the relevant band, leaving a big comfortable gap in between. For Arabic, the irrelevant chunks score up around -6.69 — higher, closer to the relevant band, and tighter. So the **separation** between “relevant” and “irrelevant” — the gap a yes/no decision has to exploit — is *smaller* for Arabic on both ends at once: the good band is squeezed down toward the middle, and the bad band is pulled up toward the middle. The two bands huddle closer together. That shrinking gap is the whole problem, and it sets up the final, decisive stage.

Stage 5 — The gate: deciding yes or no, and why one fixed answer fails Arabic

Figure 3. One global cutoff cannot fit both: Arabic's relevant and irrelevant bands sit closer together, so the gate places a stricter, geometry-matched cutoff per tenant (Arabic tenants strictest)



We now have, from the reranker, a single best chunk and its relevance grade. One question remains, and it is the most important question in the whole pipeline: **is that best chunk actually good enough to answer from?**

This is the job of the **gate**. The gate is a doorway with a height bar. If the best chunk's grade clears the bar, the gate **accepts** it — the chunk passes through and becomes the basis for the answer. If the best grade is below the bar, the gate **rejects**

it — the chunk is turned away, because grounding an answer in a chunk that is not actually relevant would produce a confident, wrong, made-up answer, which is the worst possible outcome. The height of the bar is called the **threshold**.

The everyday analogy is a **doorway with a minimum-height bar**, like the “you must be this tall to ride” sign at a fair. Set the bar at the right height and you let the genuinely relevant chunks through while turning away the junk. Set it wrong and you either turn away good chunks (too high) or wave through nonsense (too low).

Now here is where the squeeze from Stage 4 turns into visible, real-world unfairness, and it is the cleanest way to see the whole point of this paper.

Suppose — as the system *originally* did — you use **one fixed bar height for everybody**. One single threshold, hand-picked once, applied to every tenant and every language. Where would you set it? Naturally, you would tune it on the data you have most of, which is English, with its nice wide range of grades. You would find a height that sits comfortably in the big gap between English’s relevant band and English’s irrelevant band. Good. For English, that bar is perfectly placed.

But now run an Arabic question through that exact same bar. Remember Arabic’s grades are squeezed into a narrow band, and the irrelevant Arabic grades have crept up close to the relevant ones. The gap is small and it sits at a different height. A bar that was set to live inside English’s *wide* gap is now in the wrong place for Arabic’s *narrow* gap. It is a doorway height set for one room being used in a differently-shaped room. The relevant Arabic chunks and the irrelevant Arabic chunks are huddled so close together that a bar placed using English’s geometry slices through the wrong spot — letting Arabic junk through, or blocking Arabic gold, depending on exactly where it landed. **The gate never even looked at the language.** It just used one English-shaped bar for a room that is not English-shaped. That is the unfairness, made concrete.

We proved this is real by measuring the *correct* bar height for each of five different private libraries (tenants) drawn from the same corpus. The right heights were all over the place: for the two English libraries the correct bar sat around -2.93 and -2.75 ; for the Arabic-containing libraries it had to be **stricter**, up at -1.59 and -1.39 ; and another English STEM library wanted -3.29 . Those numbers span almost two full points. **No single bar height sits correctly inside all of them.** One fixed bar literally cannot be right for everyone — which is exactly why the original one-fixed-bar design was failing Arabic.

The fix that needs no retraining: let the bar set itself, per library

Here is our first and most important fix, and it lives right here at the gate. Instead of one fixed bar for everyone, we let **each library set its own bar by quietly watching its own grades**. And — this is the elegant part — it does this *without ever being told what language the library is in, and without anybody labeling a single example as relevant or not*. No retraining of any AI model. Just careful engineering wrapped around the frozen models.

How can the bar set itself with no labels? With two cheap tricks that the librarian can do all by itself inside one private library.

First, it manufactures a handful of **guaranteed-relevant** pairs. It takes a chunk, snips off its own opening words, and uses those opening words as a pretend question — then asks the reranker to grade that pretend question against the very chunk it came from. Of course they match; the question is *literally the start of the chunk*. This is the most relevant pair that can possibly exist, so the grade it earns marks the top of the “relevant” band. (The analogy: to see how your bathroom scale reads a known weight, you step on it yourself — you already know the answer, so the reading calibrates the instrument.)

Second, it manufactures **guaranteed-irrelevant** pairs. It takes that same pretend question and grades it against a chunk from a *completely different book* in the library. Those almost certainly do not match, so the grades they earn mark the top of the “irrelevant” band.

Now the librarian has read, with its own eyes, where this *particular* library’s relevant grades sit and where its irrelevant grades sit — and it sets the bar in the gap between them, a little above the irrelevant band so it does not turn away anything good. Because it built both bands using the *same* reranker on the *same* library, any quirk in how that reranker treats this library’s language is baked into *both* bands automatically and gets absorbed into where the gap falls. So in an Arabic-heavy library, where the grades are squeezed and the gap is narrow and sits higher, the self-setting bar lands *higher and stricter* — correctly — all on its own. The squeeze is handled not because anyone wrote “if Arabic, be stricter,” but because the librarian calibrated to the *shape* it actually saw. **The fairness falls out of the geometry.** That is the whole thesis of this paper in one sentence: fairness as a side effect of careful engineering around frozen models, not as a result of retraining anything.

And we have the receipts that this is really about the squeeze and not about lower scores. We checked the *bottom edge* of each library’s relevant band: for the Arabic libraries it was 10.615 and 10.584; for the English ones, 10.596 and 10.521. The Arabic relevant scores are **not lower** — the Arabic one is actually the highest of the four. So the stricter Arabic bar is forced purely by the *narrower* band, exactly the compression story, and not by any lower scores. This is the same finding from Stage 4, reappearing independently inside the live five-library system.

When we tested the self-setting gate against the old one-fixed-bar approach on a mixed batch of 60 on-topic and 60 off-topic questions, the precision — the fraction of accepted chunks that were actually relevant — jumped from **0.71 to 0.94**, while recall stayed pinned at a perfect **1.0** (it never wrongly turned away a good chunk). In plain terms: the old fixed bar waved through **25** off-topic chunks; the self-setting gate waved through only **4** — it caught 21 of the 25 mistakes — and it did all this **with no labels and no retraining**.

Two honest notes, because we promised honesty. First, this self-setting gate does **not** beat a “cheating” bar that was tuned using real relevance labels — that idealized cheater scored a perfect 1.000; our gate reached 0.968 and got *near* it without the labels, which is the point. We do not claim to beat the labeled cheater; we claim to come close to it for free. Second, that headline number is a pooled, all-languages test on a small library, so it is not by itself an “Arabic” result; the genuinely Arabic-fairness finding is the bar-height geometry — that the Arabic libraries automatically get the

stricter, correctly-placed bars.

Stage 6 — Answer: and the clever cross-language retry

We are at the end of the journey. The gate has made its call.

If the gate **accepts** — the best chunk cleared the bar — the system hands that chunk to a local language model (a small AI that runs on the same offline laptop) and asks it to write an answer *grounded in that chunk*. “Grounded” means the answer is built from the retrieved passage, not spun from the model’s vague general memory. This is the good case: a real answer, backed by a real passage from the student’s own books, that the student could go and verify.

If the gate **rejects** — nothing cleared the bar — the system does *not* bluff. It is far better to say “I could not find this in your books” than to invent a confident, wrong answer. So a rejection normally falls back to answering from general knowledge (clearly, that is weaker), or to admitting it found nothing.

But there is one more clever move, and it ties the whole story together with our Arabic student from the very start. Remember her: she asked, in Arabic, *what is entropy?*, and the only chunk that truly explains entropy is in English. Walk her question through the pipeline and watch what happens. The keyword librarian hits the cross-script brick wall and returns nothing. The meaning librarian might still find the English chunk by meaning — but suppose it does not surface it cleanly, or suppose the reranker’s squeezed Arabic grades leave the best candidate just under the bar. The gate rejects. A naive system would shrug and answer from general knowledge.

Our system does something smarter, and it pays for the cleverness *only when it is actually needed*. A rejection on an Arabic question is treated as a **calibrated same-language miss** — the system tried hard *in Arabic*, using a properly-placed Arabic bar, and genuinely came up empty. *That* is the signal, and only that signal, that triggers a single cross-language retry: translate the question once into the library’s other language (here, English), and run the whole pipeline one more time. Now the English version of “what is entropy?” sails through the keyword librarian, finds the English chunk, clears the bar, and the student gets her answer. Crucially, this expensive translate-and-retry happens *only* after a calibrated miss, never on every question — which matters enormously on a modest offline laptop where doing a translation on every single query would be far too slow.

And this same moment is where our **glossary** fix quietly does its work and even *learns*. The glossary is a simple two-way dictionary of technical terms — Arabic word on one side, English word on the other — that gets *added* to the question before the keyword search, so that an Arabic question carries its English equivalents along and can finally match an English chunk by letters. It is astonishingly cheap: about **48 microseconds** per question (a microsecond is a millionth of a second — so this is effectively free) and it loads **no AI model at all**. It lifts that structurally-zero Arabic-to-English keyword match from **0.00 to 1.00**. And when the cross-language retry above is forced to translate a brand-new term the glossary did not yet know — say *al-intrubiya* / *entropy* — the system **writes that new pair into the glossary**, so the very next time anyone asks, the keyword search already knows the word and never needs the expensive

translation again. The dictionary teaches itself from its own misses, one word at a time, forever — and, importantly, *nothing in any AI model is changed*; only a small, readable dictionary file grows.

We must say the honest thing here, as we did earlier: the *meaning* librarian could already cross scripts on its own, so the glossary’s worth is **not** that it does something impossible without it. Its worth is that it does the crossing for almost no cost, with no model loaded, in a way a human can open up and read and edit, and as a second independent path feeding the merge step. The benefit is cost, speed, and transparency — not a recall miracle. We will not overclaim it.

The cousin problem: the big library that hogs the shelf

There is one last thing to add to the story, and it is a problem that does not happen *inside* one library — it happens *between* libraries, back at the very first hunt in Stage 2, and it is sneaky enough that it deserves its own scene. It is also where our third fix lives.

Recall from Stage 2 that the keyword librarian decides how “rare and therefore valuable” a word is by looking across *the whole collection*. Now recall the rule of the building from the very start: each student has their own private library (tenant), and the librarian is only allowed to return *your* chunks. The trouble is *how* that rule was originally enforced. The old way was: search across the big combined pile of everyone’s chunks first, grab the global top results, and *then* throw away everyone else’s and keep only yours.

See the trap? Suppose one student has a huge library — thousands of chunks — and another student, often the one with the small Arabic library, has only a few dozen chunks, and the two libraries happen to share a lot of the same academic vocabulary (both talk about *gradient*, *function*, *probability*, and so on). When the librarian searches the big combined pile, the giant library’s chunks, simply because there are so many of them, fill up *all* the top slots. By the time the librarian filters down to the small library’s chunks, the good small-library chunks were already pushed off the bottom of the list and thrown away. The small library gets **starved** — its own relevant chunks never even make it to the shortlist.

The everyday analogy is a shared shelf with a pushy big neighbor. Two of you share one display shelf. Your neighbor brings a thousand books; you bring twenty. The shelf only shows the “top” books by some global rule, and your neighbor’s thousand books crowd the whole shelf. Then someone says “now show only *this* customer’s books” — and yours are nowhere to be seen, because they were elbowed off the shelf before the filter ever ran. You did nothing wrong; you were simply out-numbered. We measured this: as one library grows to dominate **98.5%** of the combined pile, the small library’s ability to recover its own correct top chunks collapses from **0.913 down to 0.461** — it loses more than half of its rightful results, before the careful reranker or the smart gate ever get a chance to help. Those later stages cannot rescue a chunk that was thrown in the bin at the very first step.

The fix is almost embarrassingly simple once you see the problem: **filter to the student’s library first, then search**. Build each library its own small, private

keyword index, and search only inside it. No pushy neighbor, because the neighbor is not even in the room. With its own private shelf, the small library recovers its correct chunks **perfectly — back up to 1.0** — at every level of dominance. We call this the **per-tenant sub-index**: each tenant gets its own dedicated keyword index. It is built quickly and cheaply the first time a small library is queried (a few milliseconds) and then kept ready for next time.

And here come two honest notes, exactly as the paper insists. First, **this starvation is not really about Arabic**. It is about *shared vocabulary under a lopsided size difference*. We checked: a small library with totally *distinct* vocabulary does **not** get starved — it stays flat and fine. So the real culprit is sharing words with a giant neighbor, not being Arabic. It just happens that, in our setting, the small library sharing academic words with the big one is often the Arabic one — so fixing the starvation is a *fairness precondition* that protects them, not a claim that the search is biased against Arabic on purpose. Second, do **not** assume the fancy meaning search escapes this trap. We measured it: under the same lopsided dominance, the meaning librarian starves **worse**, collapsing to **0.322** — even lower than the keyword librarian’s 0.461. The meaning search is not a free cure here. Only giving each library its own private index actually fixes it, and it fixes both the keyword and the meaning path back to perfect.

The whole journey, in one breath

Let us replay the entire trip one final time, fast, so the shape of it sticks.

A student types a question. We **normalize** it — straighten the paperwork so small spelling differences do not cause silly misses. We run two searches in parallel: the **keyword librarian** (BM25), exact and transparent but blind across scripts (Arabic-to-English keyword match is a flat zero by construction — the glossary’s home), and the **meaning librarian** (embeddings found fast via FAISS), flexible and script-crossing but heavier and harder to explain. We **merge** their two ranked lists fairly by position, not by incomparable raw scores, using **RRF**. We hand the merged shortlist to the **reranker**, the careful senior reader who grades each finalist by reading it next to the question — and this is where Arabic’s grades come out squeezed into a narrow band (spread 0.348 versus English’s 0.909, a ruler with less than half the marks), with the irrelevant band creeping up close, so the separating gap shrinks even though Arabic is *not* scored lower (10.534 versus 10.448 — a touch higher, in fact). Then the **gate** decides yes or no by clearing a height bar — and instead of one English-shaped bar for everyone (which mis-fits Arabic’s narrow gap and waved through 25 off-topic chunks), each library quietly **sets its own bar by watching its own grades**, with no labels and no retraining, so Arabic-heavy libraries automatically get a stricter, correctly-placed bar and the off-topic mistakes drop to 4. If the gate accepts, we **answer**, grounded in the real passage; if it rejects, we do not bluff — and a calibrated Arabic miss triggers exactly one cheap cross-language retry, where the self-growing **glossary** bridges the script gap for almost nothing and learns any new term forever. And underneath it all, the **per-tenant sub-index** makes sure a small, often-Arabic library is not starved off the shelf by a giant neighbor before the journey even begins.

Three small, ordinary, reversible pieces of engineering — a self-setting gate, a self-

growing glossary, a per-library index — wrapped around two AI models that we never once retrained. The unfairness against Arabic was never that the computer scored it lower. It was that the computer was *fuzzier* about Arabic, and one fixed bar set with English in mind sat in the wrong place for it. We did not fix that by changing the computer’s mind. We fixed it by building the rooms around it to fit the shapes it actually produces. **Fairness as engineering, not retraining.**

A final word on how seriously to take the exact numbers. Every figure in this story comes from a *small* test on *one* modest laptop with *two* specific frozen models, and the Arabic part of the collection is genuinely small — just 63 Arabic chunks against 3,760 English ones. So please read these numbers as showing a clear **direction and mechanism** — *which way* the bias points and *why* — and not as universal, settled measurements that would come out to the same decimals on every library in the world. The story is true. The exact digits are a snapshot.

Part 6 — Fix Number One: The Self-Calibrating Gate, Step by Step

What this part is about, and why it comes first

Let me remind you where we are, because this is the heart of the whole project.

We discovered something surprising in the earlier parts. Everyone *assumes* the computer is unfair to Arabic because it gives Arabic worse scores — lower marks, as if Arabic relevant matches were treated like second-class material. We measured it carefully, and that belief is **wrong**. Arabic relevant matches actually score a *touch higher*, not lower. The cross-encoder (the part of the system that grades how well a passage answers a question) gives Arabic relevant pairs an average of 10.534 versus 10.448 for English. That is a tiny bit *higher* for Arabic. So “Arabic scores lower” is simply false on our data, and we are honest about that from start to finish.

The *real* unfairness is different. It is that the computer **squeezes all the Arabic scores into a narrow band**. For English, the scores spread out across a wide range — there is plenty of room between a great match and a so-so one. For Arabic, the scores are bunched up tightly. The standard deviation (a plain measure of how spread out numbers are) is 0.348 for Arabic versus 0.909 for English on the cross-encoder — that means the English scores are spread across a band about **2.6 times wider** than the Arabic band. On the other scorer, the cosine similarity, it is 0.062 for Arabic versus 0.128 for English, a band about **2.1 times wider** for English.

Here is the everyday picture. Imagine two rulers. One ruler — the English ruler — has a hundred fine marks along its length, so you can tell apart things that differ by a hair. The other ruler — the Arabic ruler — has only forty marks crammed into the same space, so two things that are genuinely different look almost the same on it. The computer is not handing Arabic *shorter* lengths. It is measuring Arabic with a ruler that has **fewer notches**. It cannot tell a great Arabic match from a merely okay one as clearly as it can for English. That loss of fine-grainedness — that fuzziness — is the

true bias. We call it **score compression**.

So now we need a fix. And the kind of fix we want is special: we are **not allowed to retrain the AI**. The students this is built for run it offline, on a modest laptop, with no internet and no powerful graphics card. Retraining a model that knows a hundred languages is out of the question — there is no hardware for it, and there is barely any Arabic data to do it with (we will be honest about that small-data problem all the way through). So our fix has to be a piece of **ordinary engineering that wraps around the frozen AI** — the model stays exactly as it shipped, and we build something clever around it.

This part explains the first of three such fixes. It is called the **self-calibrating per-tenant relevance gate**. That is a mouthful, so let me unpack every word of it slowly, because by the end of this part you should understand it as well as the person who built it.

What is a “gate,” in plain words?

When you ask the library a question, the system finds the best-matching passage it can and scores it. Then it has to make a yes-or-no decision: *is this passage actually good enough to base an answer on, or is it junk that just happened to be the least-bad thing on the shelf?*

That yes-or-no decision is the **gate**. Think of it exactly like a bouncer at a door, or a height bar at a fairground ride. The bar is set at a certain height. If you are taller than the bar, you get in. If you are shorter, you do not. The gate works the same way: the passage gets a score, and there is a **cutoff** — a line in the sand. If the score is above the cutoff, the gate says “yes, use this passage.” If the score is below the cutoff, the gate says “no, this is not relevant enough — better to admit I don’t know than to answer from rubbish.”

That cutoff number — the height of the bar — is the single most important setting in this whole story. In the technical writing it is called the **threshold**, and it gets the symbol τ (the Greek letter “tau”). Whenever you see “threshold” or “cutoff” or “the bar,” it is the same thing: the line a score must clear to be accepted.

Now here is the problem the old system had. It used **one single cutoff for everybody**. The deployed system originally used a hand-picked value of -5.0 on the cross-encoder’s scale. (Don’t worry that the number is negative — the cross-encoder’s scores are unbounded numbers that can be negative or positive; a higher number always means “more relevant,” a lower number “less relevant.” -5.0 is just a particular line on that scale.) One bar, set at one height, for all questions in all languages.

And we already know why that is a disaster for Arabic. The single bar was set looking at English’s *wide* ruler. But Arabic’s ruler is narrow and bunched up. A bar that sits in exactly the right spot in English’s roomy range sits in the *wrong* spot in Arabic’s cramped range. It is like setting a fairground height bar by looking at a crowd of tall adults, and then trying to use the very same bar for a crowd of children of nearly identical heights — the bar that nicely separated the adults will either let almost all the children through or block almost all of them, because the children are all bunched within a few centimetres of each other.

So the obvious idea is: **stop using one bar for everyone. Give each shelf its own bar, set to that shelf’s own scoring range.** That is exactly what the self-calibrating gate does. Now I’ll explain how it sets each bar without ever being told which language the shelf is in — which is the clever and surprising part.

What is a “tenant”? Why “per-tenant”?

One more word to define before the mechanism: **tenant**.

A tenant is just one user’s private library — one walled-off shelf of books that belongs to one person (or one class, or one collection). The system keeps each tenant strictly separate from every other tenant: your books are yours, mine are mine, and the system never mixes them. Think of an apartment building where each tenant has their own locked apartment. Same building, separate homes.

“Per-tenant” simply means we set a *separate* cutoff for *each* of these private shelves, instead of one cutoff for the whole building. One tenant’s shelf might be full of English machine-learning books; another’s might be Arabic mathematics; another might be a mix. Each gets its own bar, tuned to its own scoring range.

Why does this help Arabic specifically? Because the Arabic-heavy shelves are exactly the ones with the squeezed, narrow scoring range. If each shelf gets a bar matched to *its own* range, then the squeezed Arabic shelves automatically get a bar placed correctly for a squeezed range — and the roomy English shelves get a bar placed correctly for a roomy range. The fix to the compression problem falls out for free, **without the gate ever knowing it is looking at Arabic.** Let me show you how.

The big trick: how do you set the right bar with no answer key?

Here is the puzzle. To set a good cutoff, you would normally need labelled examples — a stack of passages where a human has already written down “this one is relevant, this one is not.” With that answer key, you could find the height that best separates the good from the bad. But we have **no answer key**. Nobody has gone through these students’ private Arabic and English passages marking them relevant or irrelevant. Producing such labels is exactly the expensive work that a small, offline, low-resource setting cannot afford. (In the jargon, having no answer key is called being **label-free**, and it is one of the things we are most proud of: the gate needs zero human labels.)

So the gate has to figure out a good bar **from the books alone**, with no human telling it what is relevant. How on earth can it do that?

The answer is beautifully simple, and it is the core idea of this fix. The gate **manufactures its own examples** — it makes two homemade measuring sticks, one for “definitely good” and one for “definitely bad” — and then it knows the right bar must sit somewhere between them. Let me describe each homemade stick.

Homemade stick number one: a guaranteed-GOOD pair

Take any passage from the shelf. Now take just its **opening words** — the first twelve words of that very passage — and pretend those opening words are a question. Then

ask the scorer: “how well does this passage answer this question?”

Well, of course it answers it perfectly — the “question” is literally the passage’s own beginning. A passage is the best possible match for its own opening words. There is no more relevant pair in the universe than a passage paired with itself. So this is a **guaranteed-good pair**, an example of the highest possible relevance, and we did not need any human to tell us it was good — we built it to be good by construction.

The technical name for this is a **self-match**: a passage matched against a slice of itself. Think of it as holding up a photograph next to the actual person — of course they match, that is the whole point. It gives us a reliable reading of “this is what a great score looks like *on this particular shelf, measured by this particular scorer.*”

The gate does this not once but for several passages spread evenly through the shelf (eight of them, in the deployed version), so it gets a small spread of “definitely good” scores rather than relying on a single lucky reading.

Homemade stick number two: a guaranteed-BAD pair

Now the opposite. Take those same opening words (the pretend question), but this time pair them against a passage from a **completely different book** on the shelf — a book about a different subject entirely.

The opening words of a chapter on, say, partial fractions, matched against a passage from a book on economics? Almost certainly irrelevant. A question from one book paired with a chunk of an unrelated book is, by construction, an **almost-surely-bad pair**. Again, no human had to tell us it was bad — we built it to be bad by pairing things that have nothing to do with each other.

The technical name is a **cross-book** pair (or “cross-document”). The everyday picture: it is like asking a question about cooking and being handed a page from a car-repair manual. The mismatch is obvious, and the scorer will give it a low score. This gives us a reliable reading of “this is what a bad score looks like *on this shelf, by this scorer.*”

Putting the two sticks together: find the gap, climb a quarter of the way

So now, for each shelf, the gate has two clusters of scores:

- A cluster of **guaranteed-good** scores (the self-matches), sitting high.
- A cluster of **guaranteed-bad** scores (the cross-book pairs), sitting low.

And the truth — the right place for the bar — must live in the **gap between them**. Above the bad cluster, below the good cluster. That is just common sense: a good cutoff lets the good stuff through and blocks the bad stuff, so it belongs in the no-man’s-land in between.

To be careful and not let one freak reading throw things off, the gate does not use the absolute highest bad score or the absolute lowest good score (a single weird pair could be misleading). Instead it uses **robust edges**:

- The **top edge of the bad cluster** — it takes the level below which three-quarters of the bad scores fall (the 75th percentile of the bad pile). Call this the bad-band

ceiling.

- The **bottom edge of the good cluster** — it takes the level above which three-quarters of the good scores fall (the 25th percentile of the good pile). Call this the good-band floor.

Using these trimmed edges instead of the extremes is like ignoring the single tallest and single shortest person in a room when you describe how tall the room's people are — it stops one outlier from skewing your picture. The cost is a little extra caution, which we happily accept.

Then comes the final placement. The gate puts the bar a **quarter of the way up** from the bad-band ceiling toward the good-band floor. In plain arithmetic: take the gap between the bad ceiling and the good floor, and set the bar one-quarter of the way into that gap, starting from the bottom. The fraction “one quarter” is written as $\alpha = 0.25$ (α is the Greek letter “alpha,” and it is just the dial that says how far up the gap to climb).

Why a quarter, why low down in the gap rather than the middle? Because placing the bar **low** is generous about letting things through. It hugs the bad band closely, sitting just above it. This protects **recall** — recall being the plain idea of “do we catch all the genuinely good stuff?” A bar set low almost never accidentally rejects a real match; it errs on the side of admitting things. We deliberately chose this cautious, recall-favouring setting. (We'll see in the numbers that on our test, recall stayed perfect across every setting of this dial, so we weren't actually giving anything up — but a quarter is the safe, conservative choice.)

And one safety rule: if the shelf is too thin to measure — fewer than two books, or fewer than six passages total — or if the two clusters do not actually separate (the good floor is not above the bad ceiling, so there is no clean gap), the gate refuses to guess. It quietly falls back to a safe default bar rather than inventing a shaky shelf-specific one. **It fails safe**, which is exactly what you want from a careful piece of engineering. Better to use a sensible default than to make up a number from too little evidence.

That is the entire mechanism. Two homemade measuring sticks — a guaranteed-good pair and a guaranteed-bad pair — find the gap, and the bar goes a quarter of the way up. No labels, no human answer key, no retraining, and the model never changes. It is calibration by self-measurement.

Why this automatically gives Arabic a stricter, correctly-placed bar — without ever knowing it is Arabic

This is the most important idea in the whole part, so I am going to say it slowly and then say it again a different way, because it is the bit that makes the fix feel almost magical.

Notice that the gate **never looks at the language**. There is not a single line of code anywhere that asks “is this Arabic?” or “is this English?” It does not check the script, the alphabet, the words — nothing. It only measures scores. It is completely language-blind.

And yet it ends up handing Arabic-heavy shelves a **stricter** bar than English shelves.

How can a language-blind machine end up treating Arabic differently in exactly the right way?

Here is why. Remember the two homemade sticks — the good cluster and the bad cluster — are both built from the **same shelf** and graded by the **same scorer**. So whatever the scorer does to Arabic, it does to *both* clusters on an Arabic shelf. If the scorer squeezes Arabic scores into a narrow band, then *both* the good cluster and the bad cluster on an Arabic shelf are squeezed and sit close together. The compression is baked into where both clusters land, automatically.

Now run the recipe. On a roomy English shelf, the good cluster is high, the bad cluster is far below it, and there is a wide gap between them. A quarter of the way up a wide gap lands the bar at a comfortable, moderate height. On a squeezed Arabic shelf, the good cluster is high but the bad cluster is *also* pulled up closer to it (remember from Part on measurement: the Arabic irrelevant band sits higher and tighter, at -6.69 versus English's -7.673). The whole arrangement is compressed and shifted up. A quarter of the way up *that* gap lands the bar **higher** — that is, **stricter**.

So the squeezed Arabic geometry, all by itself, pushes the bar up. The gate did not decide “Arabic needs a stricter bar.” The *shape of the scores* decided it, and the gate simply read the shape. The fairness correction is an **emergent** property — it falls out of the geometry rather than being programmed in.

We can see this in the real numbers from the five-shelf test system. The gate set these cutoffs on the cross-encoder:

- Shelf 1 (English machine-learning/AI): -2.93
- Shelf 2 (English economics): -2.75
- Shelf 3 (**Arabic** plus mathematics): **-1.59**
- Shelf 4 (**mixed Arabic and English**): **-1.39**
- Shelf 5 (English STEM): -3.29

Remember, higher means stricter (the bar is set higher, so a passage has to score higher to get in). Look at where the two Arabic-containing shelves land: -1.59 and -1.39 are the **two highest** — the **two strictest** — bars of all five. The pure-English shelves got looser bars (-2.93 , -2.75 , -3.29). The Arabic-containing shelves got the strictest bars, automatically, with no language code anywhere.

And here is the clincher — the cross-check that proves this is about the *squeeze* and not about Arabic somehow scoring lower. Recall the “bottom edge of the good cluster” for each shelf, the good-band floor. If Arabic shelves got stricter bars because their relevant scores were *lower*, then their good-band floors would sit lower than the English ones. They do not. The good-band floors are: Shelf 1 (English) 10.596 , Shelf 2 (English) 10.521 , Shelf 3 (Arabic) **10.615** , Shelf 4 (mixed) **10.584** . The Arabic shelf's floor (10.615) is actually the **highest** of the bunch, and the mixed shelf's (10.584) sits above one of the English shelves. So the Arabic relevant scores are **not lower** — if anything they are a hair higher. The stricter Arabic bar comes purely from the **band being tighter**, exactly as our whole thesis says. The same compression we measured in Part on the bias reappears here, independently, inside the real cutoff machinery of a working five-shelf system. That is why we call this the load-bearing result: two completely separate measurements point at the same truth.

Being honest: the fix is real but partial, and it depends on the scorer

I promised relentless honesty, and here is where I deliver it, because it would be easy to oversell this.

The compensation is **real but partial, and it depends on which scorer you use**. It is not a clean universal law that says “Arabic always gets the strictest bar everywhere.” Let me show you the wrinkle.

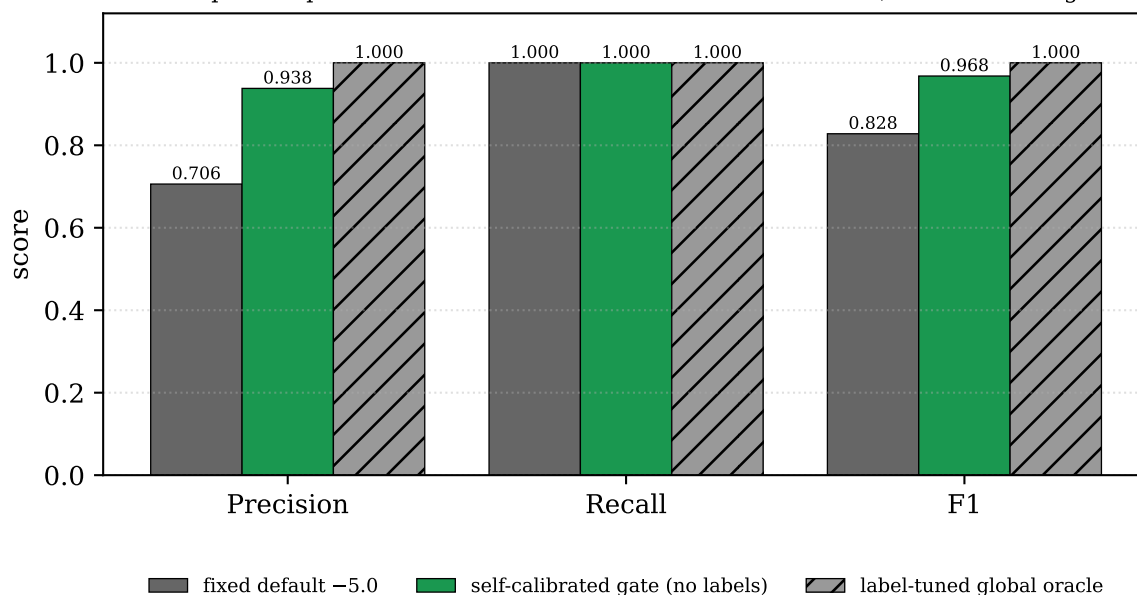
There are two different scorers, and they have different personalities — the cross-encoder (the unbounded-number scorer) and the cosine similarity (a scorer bounded between -1 and 1). The pure-Arabic shelf (Shelf 3) is the strictest in **both** scorers — its cross-encoder bar of -1.59 is the strictest, and on the cosine scorer its bar of 0.409 is also the highest of all. So far so consistent.

But the **mixed** Arabic-and-English shelf (Shelf 4) behaves inconsistently. On the cross-encoder it has the *strictest* bar of all (-1.39 , the highest). But on the cosine scorer it has the *loosest* bar of all (0.302 , the lowest). It flips from strictest to loosest depending on which scorer you ask. We call this a **rank-flip**, and we report it openly rather than hiding it. So the honest claim is: the pure-Arabic shelf is strictest in both scorers, and the Arabic-containing shelves are strictest on the cross-encoder — but it is a *corrective tendency*, not an ironclad rule, and a mixed shelf can behave differently across scorers. We claim a real, helpful, geometry-matched correction. We do **not** claim “Arabic is always the strictest, no matter what.” That would be an overstatement, and the data do not support it.

We are also honest that the exact cutoff numbers wobble a little if you run the experiment again. One run gave the Arabic bars as -1.59 and -1.39 ; a separate run gave -1.79 and -1.27 , with the English ones also shifting slightly. The **direction** is stable (Arabic-containing shelves come out strictest on the cross-encoder every time), but the precise decimal values move a touch from run to run. We report one canonical run throughout and simply note this wobble rather than pretend the numbers are carved in stone.

Walking the numbers: what the gate actually buys us

Figure 5. The label-free gate lifts precision 0.71→0.94 (F1 0.83→0.97), recall pinned at 1.0; off-topic acceptances fall 25→4 — near the oracle it never sees, without beating it



Now let me show you, in concrete terms, how much better the gate is than the old one-bar-for-everyone approach. We tested it on a fair little exam: 60 questions that genuinely have a good answer in the library (call these the “should-accept” cases), and 60 questions that are off-topic and have no good answer (the “should-reject” cases). A perfect gate accepts all 60 of the first kind and rejects all 60 of the second kind.

First, two plain-language measures you need:

- **Precision** answers: “of all the passages the gate accepted, what fraction were actually good?” High precision means the gate is not letting junk through. Low precision means it is waving rubbish past the bar.
- **Recall** answers: “of all the genuinely good passages, what fraction did the gate accept?” High recall means it is not accidentally rejecting good stuff.

Now the results.

The old fixed bar (one cutoff of -5.0 for everyone): precision **0.71**. That means barely seven out of every ten accepted passages were actually good — the other three were junk that slipped through. Recall was a perfect 1.0 (it caught all the good stuff), but only because the single bar was set so low and loose that almost everything got in, junk included. Out of the 60 off-topic questions that should have been firmly rejected, this loose bar wrongly **accepted 25** of them. Twenty-five pieces of off-topic rubbish waved straight through the door. Its overall balanced score (the F1, a single number that blends precision and recall) was **0.828**.

The self-calibrating gate (a bar per shelf, set by the homemade sticks): precision jumps to **0.94**. Now more than nine out of ten accepted passages are genuinely good — the junk is mostly shut out. Recall **stays at a perfect 1.0** — crucially, tightening up the bar did **not** cause it to start wrongly rejecting good passages. It is more selective

without becoming trigger-happy. And the off-topic rubbish it wrongly accepted fell from **25 down to just 4**. It slammed the door on 21 of the 25 pieces of junk the old bar had let through. Its overall balanced score rose from 0.828 to **0.968**.

Let me translate those numbers into what they *mean*, because the digits alone are dry. The old loose bar was like a careless bouncer who, afraid of turning away a real guest, let in roughly a third of the gate-crashers too. The new self-calibrated bar is a sharp bouncer who turns away the gate-crashers while still letting in every single real guest. Same door, same crowd, no retraining of anybody — just a smarter, shelf-by-shelf reading of where to set the bar. And it did all of this with **zero human labels**: nobody told it which passages were good or bad. It worked it out from the books themselves.

The honest ceiling: it does NOT beat the answer-key bars

Here is the honesty that matters most, and I want to be very plain about it. We compared the gate against two “cheating” bars — bars that were allowed to peek at the answer key (the human labels) to set themselves perfectly. These are called **oracles**, which just means “a method that gets to cheat by seeing the answers.” An oracle is not something you can actually deploy — in the real world there is no answer key — but it tells you the best you could possibly do.

- A single global bar tuned *on the answers* scored a perfect 1.000.
- A per-shelf bar tuned *on the answers* scored 0.992.

Our label-free gate scored 0.968. So the gate sits **just below** these answer-key bars. It does **not** beat them, and we never claim it does. What we claim — and it is the genuinely useful claim — is that the gate gets **remarkably close to the cheating bars without ever seeing a single answer**. It is “near-oracle without labels.” That is the honest framing: not a victory over having the answer key, but a way to get *almost* the same result *without* one, which is exactly what you need when no answer key exists.

A note on the quarter-of-the-way dial

I mentioned the gate climbs a quarter of the way up the gap ($\alpha = 0.25$). You might wonder: is the whole thing fragile, balanced on a knife’s edge at exactly one-quarter? It is not. We tried many settings of that dial — 0.00, 0.10, 0.25, 0.50, 0.75, 1.00 — and the balanced score climbed and then settled into a plateau (0.736, 0.845, 0.968, 0.992, 0.992, 0.992), with recall staying perfect at 1.0 the whole way. So the choice is not a hair-trigger; a range of settings work well.

But I owe you one more piece of honesty here. The flat plateau at the high end (the 0.75 and 1.00 settings) is **partly an artifact**, not genuine robustness. At those high settings the bar tries to climb so far up that it bumps into a built-in ceiling (a “clamp” that stops the bar from going absurdly high), and all five shelves end up pinned against that ceiling — so they all look the same. The numbers being equal there is the clamp talking, not the method being magically insensitive. We chose 0.25 precisely because it sits comfortably below that ceiling region, in the honest part of the curve. We report the artifact rather than dressing the plateau up as evidence of stability.

And one more honest scope note on that headline

That balanced score of 0.968 is a **pooled** result across all five shelves on a small, English-dominated test corpus — it is **not** an Arabic-only number. And recall pins to a perfect 1.0 partly because the “should-accept” test questions were the easy passage-derived self-matches with their source passage left sitting in the pool to be found. So the real Arabic-fairness evidence does **not** rest on this 0.968 headline. It rests on the **cutoff geometry** — the fact that the Arabic-containing shelves got the strictest bars purely because their bands are tighter, proven by the good-band floors not being any lower. That geometry result is the load-bearing one. The F1 is supporting colour, and we are careful not to let it carry more weight than it can bear.

The translate-only-on-a-miss fallback, in plain words

There is one last piece bolted onto the gate, and it is a tidy bit of thrift. It handles the case where you ask a question in one language but the only book that can answer it is in the *other* language.

Translating every single question into the other language would be wasteful — translation is slow and expensive on a modest offline laptop, and most of the time it is unnecessary because the answer is right there in the same language as your question. So the system is lazy in a good way. It works like this:

1. You ask your question in, say, Arabic. The system searches the Arabic side, finds the best passage, and scores it.
2. The gate checks that score against the shelf’s bar. If it **clears the bar**, great — accept, answer, and **never translate**. No cost paid. Most questions stop here.
3. Only if the best score **misses the bar** — a genuine, calibrated “nothing good enough here in your language” — does the system bother to translate your question **once** into the other language and try again. Just one retry. The translation is cached (saved) so it is never redone, and it is wrapped in a hard time limit so it can never hang.
4. If that cross-language retry now finds something that clears the bar, it accepts that. If it still finds nothing, it gives up gracefully rather than serving junk.

The everyday picture: you ask a librarian for a book, and they check your usual shelf first. Only if it is genuinely not there do they walk over to the foreign-language section and look again — they do not march across the building on every single request “just in case.” Translation is paid for **only when the calibrated bar tells us we truly missed**, and even then at most once. This thrift is possible *because* the gate gives us a trustworthy “did we miss?” signal in the first place. The bar is not just a yes/no on the answer — it is also the trigger that decides whether the more expensive cross-language search is even worth attempting. (As a small bonus, this same translate-on-a-miss path is what quietly teaches the growing bilingual dictionary new words — but that is the second fix, and it belongs to the next part.)

Pulling Part 6 together

Let me restate the whole thing in one breath, because repetition is fair in the exhaustive version.

The unfairness to Arabic is not lower scores — it is a **squeezed, narrow scoring band**, a ruler with too few notches, so that one fixed cutoff set on English’s wide range sits in the wrong place for Arabic. The self-calibrating gate fixes this **without retraining the AI**. For each private shelf, it builds two homemade measuring sticks — a guaranteed-good pair (a passage versus its own opening words) and a guaranteed-bad pair (a passage versus a chunk of an unrelated book) — finds the gap between the resulting good and bad score clusters, and sets the bar a cautious quarter of the way up. Because both sticks are made from the same shelf and graded by the same frozen model, the Arabic squeeze is baked into where both clusters land, so the recipe **automatically places a higher, stricter, correctly-matched bar on Arabic-heavy shelves** — even though the gate is completely language-blind and never checks the script. We proved this is the squeeze and not lower scores by checking the good-band floors, which are not lower for Arabic. The payoff is concrete: precision climbs from 0.71 to 0.94, off-topic junk accepted drops from 25 to 4, recall stays perfect at 1.0, and all of it uses **no human labels and no model retraining**. And we are honest: the gate gets *near* the answer-key bars but does **not** beat them; the correction is real but partial and can flip between scorers; the headline balanced score is pooled and not Arabic-specific; and a thrifty translate-only-on-a-miss fallback handles cross-language questions by paying the translation cost only when the calibrated bar says we genuinely missed. This is the first concrete face of our one big idea: **fairness as engineering, not retraining**.